

UNIVERSIDADE FEDERAL DE UBERLÂNDIA
FACULDADE DE CIÊNCIA DA COMPUTAÇÃO
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO



**PADRÕES E DIRETRIZES ARQUITETURAIS PARA
ESCALABILIDADE DE SISTEMAS**

IVENS OLIVEIRA PORTO

Uberlândia - Minas Gerais

2009

UNIVERSIDADE FEDERAL DE UBERLÂNDIA
FACULDADE DE CIÊNCIA DA COMPUTAÇÃO
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO



IVENS OLIVEIRA PORTO

PADRÕES E DIRETRIZES ARQUITETURAIS PARA ESCALABILIDADE DE SISTEMAS

Dissertação de Mestrado apresentada à Faculdade de Ciência da Computação da Universidade Federal de Uberlândia, Minas Gerais, como parte dos requisitos exigidos para obtenção do título de Mestre em Ciência da Computação.

Área de concentração: Redes de Computadores.

Orientador:

Prof. Dr. Pedro Frosi Rosa

Uberlândia, Minas Gerais

2009

Dados Internacionais de Catalogação na Publicação (CIP)

P853p Porto, Ivens Oliveira, 1978-
Padrões e diretrizes arquiteturas para escalabilidade de sistemas /
Ivens Oliveira Porto. - 2009.
161 f. : il.

Orientador: Pedro Frosi Rosa.

Dissertação (mestrado) – Universidade Federal de Uberlândia, Programa de Pós-Graduação em Ciência da Computação.
Inclui bibliografia.

1. Redes de computação - Teses. I. Rosa, Pedro Frosi. II. Universidade Federal de Uberlândia. Programa de Pós-Graduação em Ciência da computação. III. Título.

CDU: 681.3.02

UNIVERSIDADE FEDERAL DE UBERLÂNDIA
FACULDADE DE CIÊNCIA DA COMPUTAÇÃO
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Os abaixo assinados, por meio deste, certificam que leram e recomendam para a Faculdade de Ciência da Computação a aceitação da dissertação intitulada “**Padrões e diretrizes arquitetais para escalabilidade de sistemas**” por **Ivens Oliveira Porto** como parte dos requisitos exigidos para a obtenção do título de **Mestre em Ciência da Computação**.

Uberlândia, 3 de Setembro de 2009

Orientador:

Prof. Dr. Pedro Frosi Rosa
Universidade Federal de Uberlândia

Banca Examinadora:

Prof. Dr. Sergio Takeo Kofuji
Universidade de São Paulo

Prof. Dr. Rivalino Matias Jr.
Universidade Federal de Uberlândia

UNIVERSIDADE FEDERAL DE UBERLÂNDIA
FACULDADE DE CIÊNCIA DA COMPUTAÇÃO
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Data: Setembro de 2009

Autor: **Ivens Oliveira Porto**
Título: **Padrões e diretrizes arquiteturais para escalabilidade de sistemas**
Faculdade: **Faculdade de Ciência da Computação**
Grau: **Mestrado**

Fica garantido à Universidade Federal de Uberlândia o direito de circulação e impressão de cópias deste documento para propósitos exclusivamente acadêmicos, desde que o autor seja devidamente informado.

Autor

O AUTOR RESERVA PARA SI QUALQUER OUTRO DIREITO DE PUBLICAÇÃO DESTE DOCUMENTO, NÃO PODENDO O MESMO SER IMPRESSO OU REPRODUZIDO, SEJA NA TOTALIDADE OU EM PARTES, SEM A PERMISSÃO ESCRITA DO AUTOR.

Dedicatória

Dedido este trabalho à minha esposa Adriana, pelo apoio incondicional, compreensão e paciência.

Agradecimentos

Agradeço a todos do corpo docente e do corpo administrativo da Faculdade de Computação da Universidade Federal de Uberlândia pela oportunidade de realizar este trabalho e por toda contribuição em minha formação acadêmica e profissional, e todo o apoio para conclusão deste trabalho.

Aos meus pais que sempre me apoiaram e me guiaram a fazer as coisas certas.

Ao meu professor, parceiro, orientador e amigo Dr. Pedro Frosi Rosa, por sempre acreditar em meu trabalho e minha capacidade. Obrigado por este trabalho, espero que seja apenas mais um de muitos outros.

A todos que direta ou indiretamente me ajudaram para conclusão deste trabalho.

Obrigado a todos.

“Gatinho de Cheshire”, começou, muito timidamente, por não saber se ele gostaria desse tratamento: ele, porém, apenas alargou um pouco mais o sorriso.

“Ótimo, até aqui está contente”, pensou Alice.

E prosseguiu: “Você poderia me dizer, por favor, qual o caminho para sair daqui?”

“Depende muito de onde você quer chegar”, disse o Gato.

“Não me importa muito onde...” foi dizendo Alice.

“Nesse caso não faz diferença por qual caminho você vá”, disse o Gato.

“...desde que eu chegue a algum lugar”, acrescentou Alice, explicando.

“Oh, esteja certa de que isso ocorrerá”, falou o Gato, “desde que você caminhe o bastante.”

- Lewis Carrol.

Resumo

Com o uso da computação em praticamente todas as áreas de atividades, os sistemas (software) destinados a prover grande capacidade de armazenamento/processamento/acessos passaram a ser concebidos como sistemas distribuídos. Para esses, escalabilidade tornou-se uma importante propriedade em seu projeto e arquitetura, fazendo com que tenham de lidar com cargas de trabalho, de dados e de acessos cada vez maiores enquanto exige-se que apresentem desempenho satisfatório.

Uma questão ainda não totalmente explorada é: como arquitetar um sistema escalável? Existem trabalhos que discutem princípios e técnicas gerais de escalabilidade, especialmente sobre melhoria de desempenho. Entretanto, essas informações estão desorganizadas, e desestruturadas.

Ao iniciar o projeto de um sistema, sempre há o questionamento sobre quais camadas, módulos, objetos e relacionamentos o projetista deve considerar como ponto de partida. Entretanto, os sistemas distribuídos, por mais particulares que sejam, sempre apresentam algumas similaridades quanto ao acesso a dados, processamento distribuído, compartilhamento de contextos, etc.

Esta dissertação objetiva identificar, catalogar e discutir as diretrizes e técnicas arquiteturais para auxiliar nos projeto e construção de sistemas escaláveis horizontalmente desde sua concepção, transformando a escalabilidade de uma propriedade de sistema em um aspecto fundamental da sua arquitetura.

A idéia básica é oferecer aos projetistas, um conjunto de padrões arquiteturais que ele possa instanciar à medida que ele os detecte na análise do sistema. Por exemplo, se houver massivo acesso a dados e, portanto, haja necessidade de particionar o banco de dados, o padrão arquitetural aplicável (*Sharding*) especificará quais camadas e subcamadas devem constar no sistema.

Para que os projetistas possam identificar os requisitos de escalabilidade e estabelecer as nuances de escalabilidade, os padrões arquiteturais são relacionados em uma linguagem de padrões. Esta linguagem pode ser utilizada como uma ferramenta durante o projeto de um sistema escalável. Ressalte-se a apresentação de diretrizes para se alcançar escalabilidade na construção de sistemas.

As diretrizes e técnicas se preocupam, fundamentalmente, com a escalabilidade horizontal, que torna possível a execução de um sistema em vários nós de processamento. Ao aumentar a quantidade de nós, o sistema aumenta, ou mantém, seu desempenho de maneira satisfatória. As diretrizes e padrões apresentados neste trabalho são aplicáveis particularmente a aplicações *web* e a sistemas distribuídos que trabalham com dados armazenados.

É apresentada a arquitetura de um sistema escalável e discutido quais padrões e diretrizes foram utilizados, como foram aplicados e quais decisões levaram a sua aplicação no projeto do sistema. Um estudo em laboratório permite verificar a eficácia da proposta.

O trabalho tem como principal resultado a apresentação de padrões arquiteturais, de uma linguagem de padrões e das diretrizes a serem utilizadas por arquitetos de software na construção de sistemas escaláveis.

Palavras chave: arquitetura, escalabilidade, desempenho, padrões, sistemas distribuídos, engenharia de software, teorema cap.

Abstract

With the use of computation in practically all areas of work, the systems (software) being used to provide great capacity of storage/processing/accesses are conceived as distributed systems. To those, scalability has become an important property to its project and architecture, making them deal with ever growing workloads of data and accesses while demanding satisfactory performance.

An issue not fully explored is: how to build an architecture for a scalable system? There are works that discuss principles and general techniques for scalability, specially about performance improvement. However, this information is disorganized and unstructured.

When beginning a system project there is always the question about which tiers, modules, object and relationships the designer must consider as a starting point. However, distributed systems, as particular as they may be, always present some similarities regarding data access, distributed processing, context sharing, etc.

This master's thesis objective is to identify, catalog and discuss, architectural guidelines and techniques to help in designing and building horizontally scalable systems since its conception, transforming scalability from a system property to a fundamental aspect of their architecture.

The basic idea is to offer to designers a set of architectural patterns that he can instantiate as he detects them during the systems analysis. For example, if there is massive access to the data, and thus the need to partition the database, the applicable architectural pattern (*Sharding*) specifies which tiers and sub-tiers must be part of the project.

To make possible for designers to identify the scalability requirements and establish the scalability nuances, the architectural patterns are related into a pattern language. This language can be used as tools during the design of a scalable system. It should be noted the presentation of guidelines for achieving scalability in building systems.

The guidelines and techniques are concerned, fundamentally, with horizontal scalability, that makes possible the execution of a system with several processing nodes. By increasing the number of nodes, the system increases, or maintains, its performance satisfactorily. The guidelines and patterns presented in this work are particularly applicable to web applications and distributed systems that deal with stored data.

The architecture of a scalable system is presented and the applied patterns and guidelines are discussed along with how they were applied and which decisions lead to its use. A laboratory study allows the verification of the proposal effectiveness.

This work has as its main result the presentation of architectural patterns, a pattern language and the guidelines to be used by software architects while building scalable systems.

Keywords: architecture, scalability, performance, patterns, distributed systems, software engineering, cap theorem.

Sumário

Lista de Figuras	xix
1 Introdução	21
1.1 Contribuições	24
1.2 Organização da Dissertação	25
2 Posicionamento de Contexto em Escalabilidade de Sistemas	27
2.1 Definições Preliminares	27
2.2 Definições de Escalabilidade	28
2.2.1 Escalabilidade Vertical	31
2.2.2 Escalabilidade Horizontal	32
2.2.3 Categorias de Escalabilidade	34
2.3 Escalabilidade e Desempenho	37
2.4 O Teorema CAP (<i>Consistency, Availability, Partition Tolerance</i>)	38
3 Padrões Arquiteturais para Escalabilidade	41
3.1 Padrões: Definição e Aspectos Relevantes	41
3.1.1 Estrutura e Descrição de Padrões	44
3.1.2 Formato da Descrição dos Padrões	45
3.2 Padrão: Arquitetura <i>Shared Nothing</i>	47
3.3 Padrão: <i>Sharding</i>	58
3.4 Padrão: BASE (<i>Basically Available, Soft state, Eventual consistency</i>)	77
3.5 Padrão: Sagas	94
3.6 Padrão: Camada de Caches Distribuídos	104
3.7 Uma Pequena Linguagem de Padrões	118
4 Diretrizes Arquiteturais para Escalabilidade	121
4.1 Diretrizes	121
5 Exemplo de uma Arquitetura de um Sistema Escalável	129
5.1 Requisitos Funcionais	129
5.2 Requisitos Não Funcionais	130

5.3	Arquitetura e Funcionamento	130
5.4	Aplicabilidade dos Padrões e Diretrizes	133
5.4.1	Aplicabilidade das Diretrizes	133
5.4.2	Aplicabilidade dos padrões	136
5.5	Análise da Escalabilidade	140
6	Conclusão e Trabalho Futuros	145
6.1	Conclusão	145
6.2	Trabalhos Futuros	146
	Referências Bibliográficas	149
A	Sagas	155

Lista de Figuras

2.1	Gráfico da escalabilidade de um sistema	31
2.2	Escalabilidade linear	35
2.3	Escalabilidade sublinear	36
2.4	Escalabilidade superlinear	36
3.1	Padrão fachada	42
3.2	Aumento da complexidade do sistema com adição de novas instâncias	49
3.3	Possível solução <i>shared nothing</i>	51
3.4	Melhoria na solução <i>shared nothing</i>	52
3.5	Particionamento funcional de um SNA	53
3.6	SNA com banco de dados único	53
3.7	SNA com <i>sharding</i>	54
3.8	Solução <i>shared nothing</i>	55
3.9	Objetivo do <i>sharding</i>	60
3.10	Arquitetura de um sistema com <i>sharding</i>	61
3.11	<i>Sharding</i> vertical	63
3.12	<i>Sharding</i> Diagonal	64
3.13	Dinâmica do <i>sharding</i>	65
3.14	Modelo de dados da tabela de consulta	66
3.15	Estrutura para consultas paralela em shards	71
3.16	Dinâmica das consultas paralelas em shards	72
3.17	Estrutura de uma arquitetura BASE	81
3.18	Estrutura de uma arquitetura BASE com cache distribuído	82
3.19	Dinâmica de uma arquitetura BASE	83
3.20	<i>Shards</i> do exemplo	84
3.21	Modelo de dados do exemplo	85
3.22	Modelo de domínio de Sagas	96
3.23	API para Sagas	97
3.24	Estrutura de caches distribuídos e particionados	106
3.25	Caches em <i>sideline</i>	107
3.26	Caches como uma camada de abstração	107

3.27	Dinâmica de uso de cache em <i>sideline</i>	108
3.28	Dinâmica de caches como uma camada de abstração aos dados	109
3.29	Escrita <i>write-through</i>	110
3.30	Escrita <i>write-back</i>	110
3.31	Linguagem de padrões	119
5.1	Arquitetura do sistema de exemplo	131
5.2	Construção de <i>pools</i> de instâncias para o sistema exemplo	136
5.3	Organização do módulos do sistema	141
5.4	Avaliação do tempo de resposta	142
5.5	Resumo dos resultados da primeira avaliação	142
5.6	Variação do tempo de resposta por vazão, 1 nó	143
5.7	Variação do tempo de resposta por vazão, 2 nós	143
5.8	Resumo dos resultados da segunda avaliação	143

Capítulo 1

Introdução

É cada dia mais difícil encontrar um área de atividade que não utilize a computação como meio, como ferramenta. Muitas atividades podem ser sistematizadas através da computação, sendo que neste caso, o acesso às atividades é provido a seus usuários por meio de serviços. Em muitos casos, a diferença reside na amplitude da utilização. A quantidade de acesso a um serviço definirá o poder de processamento do hardware a ser utilizado por um servidor (provedor do serviço).

Nos dias atuais, várias áreas de atividades contam com inúmeros usuários e, para estas áreas, o processamento requerido pode demandar máquinas de alto desempenho ou, até mesmo, várias máquinas. Na realidade, a quantidade de processamento requerido por um servidor pode ser provido por máquinas de alto desempenho ou provido por aglomerados (*Cluster*) de máquinas de processamento regular.

Aglomerados podem ser interessantes por três aspectos: i) podem ser compostos de máquinas regulares, cujos preços são muito inferiores a certas máquinas de alto desempenho; ii) podem crescer à medida que aumenta a necessidade de processamento; e, iii) podem oferecer capacidade de processamento variável (em função da demanda), permitindo melhor gerenciamento do uso de energia.

Para estes casos, escopo desta dissertação, escalabilidade tornou-se uma importante propriedade de sistemas (de software) ¹. Doravante, todas as considerações arquiteturais tomarão por base os sistemas de software desenvolvidos para atender a grandes volumes de acessos a serviços.

Os software construídos hoje em dia, por exemplo para provimento de serviços em áreas tais como Internet ou Telecomunicações, devem lidar com um volume crescente de usuários e dados, e, no entanto, devem continuar a funcionar com desempenho satisfatório. Este cenário se torna mais complexo se se considerar que as dimensões das junções semicondutoras dos circuitos integrados aproximam-se de seus limites físicos. Há um limite de processamento para

¹Neste trabalho os termos “sistema” e “software” são tratados como sinônimos e intercambiáveis. Eventualmente, em situações específicas “sistema” se referirá à combinação de software e hardware, nestes casos este fato será deixado explícito.

as máquinas de alto despenho, após o qual será necessário introduzir mais máquinas. Deste modo, escalabilidade torna-se uma necessidade desafiadora para arquitetos ou desenvolvedores de software.

Um sistema não escalável pode trazer sérias conseqüências às empresas, pois elas poderão não ser capazes de atender a seus clientes. Além disso, é provável que tenham algum prejuízo em sua imagem, considerando que os usuários tendem a não utilizar um sistema que não é capaz de atendê-los - podendo haver perda de confiança na empresa ou no serviço (um ponto importante já que cada vez mais as pessoas armazenam seus dados nos computadores de empresas), ou, pior ainda, os usuários poderão utilizar os serviços de concorrentes.

Apesar da escalabilidade ter se tornado uma propriedade importante na especificação de sistemas, pois é quase uma constante entre os requisitos não funcionais, não há uma definição única, amplamente aceita e formal do que seja escalabilidade. Após Hill [Hill 1990] ter colocado o desafio de definir rigorosamente o que é escalabilidade ou parar de usá-la para descrever sistemas, foram feitas várias definições, formais e informais, entre elas [Bondi 2000] [Weinstock e Goodenough 2006], [Brataas e Hughes 2004], [Steen et al. 1998].

Algumas maneiras de medir e prever a escalabilidade de sistemas também foram desenvolvidas [Duboc et al. 2007], [Jogalekar e Woodside 2000]. O que ficou claro a partir destes estudos é que escalabilidade é um assunto complexo, com mais de uma dimensão. Ter uma definição clara do que é escalabilidade ajuda a entendê-la e faz com que sejam possíveis afirmações como “o sistema é escalável”, “o sistema não é escalável” ou “o sistema escala desta maneira ...”. Mesmo assim, ainda é muito comum fornecedores descreverem que um produto (software) possui “alta escalabilidade” sem que se saiba exatamente do que se está falando. O mesmo se aplica aos consumidores, é comum clientes, ao contratarem fábricas de software para desenvolvimento de software, exigirem sistemas escaláveis sem que saibam o que isso significa.

Uma questão sobre escalabilidade, que não foi totalmente explorada, é como arquitetar um sistema escalável. Existem trabalhos que discutem princípios e técnicas gerais [Neuman 1994], [Steen et al. 1998], [Weinstock e Goodenough 2006], e inúmeros trabalhos com um escopo menor, especialmente sobre melhoria de desempenho [Rosenthal 2003], [Anderson 1999], [Bertolino e Mirandola 2004].

Existem trabalhos, em sua grande maioria produzidos pela indústria, que tratam de como construir sistemas escaláveis, sendo, todavia, com um foco menor e quase sempre tratam de tecnologias específicas como Java, Ruby, PHP, Linux. Além disso, estas informações estão espalhadas em muitos artigos, *web sites*, *blogs*, relatórios técnicos e muitas vezes apenas nas mentes de alguns poucos e experientes engenheiros.

Ressalte-se ainda que, geralmente, estas informações não estão detalhadas, estruturadas e não são discutidas o suficiente. Um arquiteto de sistemas, com a responsabilidade de projetar um sistema escalável, tem à sua disposição uma fonte desorganizada e pobre de conhecimento para apoio ao seu trabalho.

Escalabilidade é considerada uma propriedade difícil de ser contemplada, pois não é algo

que pode ser pensado ou feito após a implementação. Depois de pronto, é muito mais difícil escalar o sistema, sendo que em alguns casos, o custo (de alterações) pode ser proibitivo. Deve-se, portanto, projetar e construir um sistema com a preocupação da escalabilidade desde seu início.

O objetivo principal deste trabalho é identificar, catalogar e discutir diretrizes e técnicas arquiteturais para auxiliar arquitetos de sistemas a projetar e construir sistemas escaláveis, desde a sua concepção. Deste modo, este trabalho pretende transformar a escalabilidade de uma propriedade de sistema em um aspecto fundamental da sua arquitetura.

As diretrizes e técnicas se aplicam durante a fase de projeto, para que o software seja escalável a partir de sua concepção, evitando os erros, comuns, de abordar a escalabilidade como um item a ser tratado mais tarde (quase sempre quando já é tarde demais) ou como um processo de tentativa e erro.

Um outro objetivo deste trabalho é compartilhar a experiência adquirida no desenvolvimento de sistemas escaláveis para empresas de internet e de telecomunicações, através do uso das diretrizes e técnicas. A partir destas diretrizes e técnicas, será possível para projetistas, arquitetos e desenvolvedores de sistemas, confrontarem o desafio de construir sistemas escaláveis com ferramentas melhores e mais adequadas. Como não há uma única estratégia ou diretriz que solucione todos os problemas relacionados a escalabilidade, portanto, é de grande importância que arquitetos de sistemas tenham à sua disposição pré-projetos utilizáveis.

Para se atingir o objetivo principal deste trabalho, são apresentadas as técnicas, sempre que for factível, como padrões arquiteturais (*architectural patterns*), que são um tipo específico de padrões, com escopo mais amplo do que os padrões de projeto (*design patterns*). Assim, a aplicação das técnicas no projeto de uma arquitetura de sistema torna-se mais fácil, rápida e sistemática, levando aos projetistas de sistemas todos os benefícios de padrões de projeto.

A catalogação das técnicas será feita em formato de padrões arquiteturais, pois, até onde foi possível pesquisar, não foi encontrado trabalho com esta abordagem e, além disso, percebeu-se que é factível a estruturação de técnicas arquiteturais desta maneira. Foi notado, ainda, que há problemas recorrentes em contextos diferentes, relacionados à escalabilidade, que podem ser resolvidos de forma similar. Há vários trabalhos de padrões aplicados a arquitetura de sistemas, como por exemplo [Schmidt et al. 2000] [Kircher e Jain 2004] [Buschmann et al. 2007a], e a proposta é que a mesma abordagem pode de ser aplicada ao quesito escalabilidade.

Um exemplo de desafios de escalabilidade que este trabalho se propõe a ajudar e, eventualmente, a solucionar: suponha que exista um sistema *on-line* que atenda requisições de vários clientes e devido a uma nova funcionalidade do sistema, de um dia para o outro, a carga de trabalho imposta ao sistema triplica; o que deve ser feito para que o sistema seja capaz de atender a esta nova carga de trabalho? O sistema é capaz de atender à nova carga oferecendo tempo de resposta aceitável? Ele está preparado para crescer visando a atender a nova carga de trabalho? Como o sistema deve crescer para suportar a nova carga?

Um outro exemplo, este já elaborado com algum conhecimento sobre escalabilidade, é como

projetar um sistema para atender ao seguinte requisito: “o sistema a ser construído deve suportar entre 100 e 10.000 requisições por segundo, sempre respondendo em menos de 2 segundos às requisições”.

Este trabalho propõe diretrizes e técnicas que abordam, fundamentalmente, a escalabilidade horizontal, tornando possível a execução de um sistema em vários nós de processamento de tal modo que ao aumentar a quantidade de nós o sistema aumente a capacidade de processamento e mantenha, ou aumente, seu desempenho de maneira satisfatória. Escalabilidade horizontal é muito interessante, pois atualmente é possível contar com a disponibilidade de uma gama considerável de hardware a preços acessíveis.

Geralmente os sistemas escalam bem em apenas um nó de processamento (escalabilidade vertical), mas quando atingem o limite de processamento do hardware não há outra alternativa a não ser expandir o sistema para outros nós (escalabilidade horizontal). As técnicas arquiteturais para escalabilidade abordadas aqui não têm como objetivo ser uma lista exaustiva de todas as técnicas para construir sistemas escaláveis, pois isso ocuparia o espaço de várias dissertações, mas trata-se aqui daquelas que possuem impacto na escalabilidade horizontal.

Especificamente, as diretrizes e técnicas apresentadas neste trabalho são aplicáveis a sistemas *web* e a sistemas distribuídos em rede. Assim como foi feito em [Gamma et al. 1995], os padrões descritos aqui não são novas (e não testadas) criações. Tratam-se de técnicas aplicadas e testadas em sistemas reais, que não foram devidamente documentadas e estruturadas, mas, uma vez documentadas, possibilitam o compartilhamento da experiência de muitos anos adquirida por arquitetos de sistemas. Deste modo, outro objetivo do trabalho é discutir diretrizes arquiteturais conhecidas, e amplamente aplicadas, e trazer à tona seus relacionamentos com a escalabilidade.

1.1 Contribuições

Os objetivos descritos anteriormente permitem vislumbrar as seguintes contribuições deste trabalho:

- Identificação e catalogação de diretrizes e técnicas arquiteturais para construção de sistemas escaláveis horizontalmente;
- Estruturação de técnicas para construção de sistemas escaláveis horizontalmente como padrões arquiteturais;
- Identificação das forças e os aspectos do problema que devem ser considerados na solução, referentes a vários problemas de escalabilidade, como consequência da estruturação dos padrões arquiteturais;
- Diretrizes de implementação para a solução dos problemas de escalabilidade discutidos nos padrões arquiteturais;

- Discussão do relacionamento de diretrizes arquiteturais, e boas práticas de construção de sistemas já conhecidos, com a escalabilidade horizontal;
- Contribuição de experiências profissionais com a aplicação das diretrizes arquiteturais; e
- Fornecimento de uma implementação de Sagas [Garcia-Molina e Salem 1987] de código fonte livre.

1.2 Organização da Dissertação

Esta dissertação está organizada da seguinte maneira. No capítulo 2 será apresentado um posicionamento de contexto em escalabilidade de sistemas. Definições de escalabilidade serão apresentadas e discutidas e será feita a categorização da escalabilidade em função do fator de escalabilidade.

No capítulo 3 discute-se o que são padrões arquiteturais, o que os compõe, como devem ser documentados e apresenta-se os padrões arquiteturais para escalabilidade horizontal. Uma pequena linguagem de padrões é elaborada.

O capítulo 4 lista e discorre sobre várias diretivas que podem ser utilizadas para auxiliar na construção de sistemas escaláveis.

No capítulo 5, a arquitetura de um sistema escalável é apresentada e discute-se como os padrões e diretivas deste trabalho foram aplicados e como isto tornou o sistema escalável. Além disso, é feito um estudo da escalabilidade do sistema de exemplo.

Finalmente, no capítulo 6, serão apresentadas conclusões e perspectivas de trabalhos futuros.

Capítulo 2

Posicionamento de Contexto em Escalabilidade de Sistemas

Neste capítulo é apresentado o estado da arte no que diz respeito ao entendimento e definição de escalabilidade. Para a finalidade deste capítulo, serão apresentados quatro aspectos da escalabilidade: o conceito geral do que é escalabilidade; escalabilidade vertical; escalabilidade horizontal; e categorias de escalabilidade. O claro entendimento e a definição do que é escalabilidade mostra a direção correta a ser seguida durante a discussão das diretrizes e dos padrões arquiteturais para construção de sistemas escaláveis.

2.1 Definições Preliminares

Ao longo deste trabalho alguns termos serão frequentemente utilizados e, para a sua compreensão, são apresentados aqui seus significados no que tange este trabalho.

Desempenho: quantidade de trabalho realizado por um sistema comparado ao tempo e recursos utilizados, podendo ser medido através de métricas de desempenho como tempo de resposta ou vazão, sendo que o termo Desempenho, quando utilizado neste trabalho, se referirá não apenas a um, mas a qualquer conjunto de métricas de trabalho realizado por um sistema se comparado ao tempo e recursos utilizados.

Sistema ou Aplicação: software, que pode ser constituído de módulos, subsistemas, componentes, etc.

Instância: sistema, ou alguma de suas partes constituintes, em execução, hospedado em um computador. É possível que um computador hospede mais de uma instância ao mesmo tempo.

2.2 Definições de Escalabilidade

Escalabilidade é um termo usado a bastante tempo para descrever uma propriedade de sistemas. É muito comum ouvir dizer que determinado sistema é, ou não, escalável. Hill [Hill 1990], em 1990, colocou um desafio para definir formalmente o que é escalabilidade ou a parar de usar o termo para qualificar sistemas. Depois deste desafio várias definições de escalabilidade foram feitas, formais e informais, entretanto, ainda hoje não há uma única definição amplamente aceita.

Outro ponto que dificulta uma única definição é que escalabilidade é um tópico multidimensional. Assim como a arquitetura de sistemas é um tópico multidimensional [Clements et al. 2002], a escalabilidade também o é. Não é possível falar de escalabilidade sem considerar outros aspectos como desempenho, manutenibilidade, usabilidade, confiabilidade, segurança, disponibilidade, etc. [Duboc et al. 2007]. Escalabilidade é um conceito que pode ser aplicado a praticamente qualquer aspecto de um software, sendo que pode-se referir a escalabilidade relacionada a desempenho, a capacidade de armazenamento/recuperação de dados, a estrutura, a extensibilidade do software, a processos de desenvolvimento de software, etc. [Bondi 2000]. Neste trabalho tratar-se-á de escalabilidade relacionada ao desempenho de sistemas.

Para podermos formar um conceito geral de escalabilidade é importante conhecer algumas definições existentes. A seguir são listadas algumas definições de escalabilidade.

“O conceito conota a habilidade de um sistema de acomodar um número crescente de elementos ou objetos, para processar crescentes cargas de trabalho graciosamente, e/ou ser suscetível a ser ampliado.” [Bondi 2000]

“Dizemos que um sistema possui escalabilidade de carga se ele tem a habilidade de funcionar graciosamente, i.e., sem atraso indevido e sem consumo improdutivo de recursos ou contenção de recursos com cargas de trabalho leves, moderadas ou pesadas enquanto faz bom uso dos recursos.” [Bondi 2000]

“Representa a habilidade de cumprir requisitos de capacidade dentro de uma faixa desejada, enquanto continua a satisfazer todos os outros requisitos: funcionais, estatísticos, qualidade de serviços, custo de propriedade por unidade, etc.” [Brataas e Hughes 2004]

“Um arquitetura é escalável ... se ela apresenta ... um aumento linear (ou sublinear) no uso de recursos físicos a medida que a capacidade aumenta ...” [Brataas e Hughes 2004]

“Um sistema é dito escalável se ele pode arcar com a adição de usuários e recursos sem sofrer perda de desempenho perceptível ou aumento da complexidade de administração.” [Neuman 1994]

“Escalabilidade $\psi(k_1; k_2)$ de uma escala k_1 para outra escala k_2 é a taxa de eficiência para os dois casos, $\psi(k_1; k_2) = E(k_2) = E(k_1)$. Ela também possui um valor ideal que é unitário.” [Jogalekar e Woodside 2000]

“Escalabilidade significa não apenas a habilidade de operar, mas de operar com eficiência e com qualidade de serviço adequada, dentro de uma faixa de possíveis configurações.” [Jogalekar e Woodside 2000]

“Definimos escalabilidade como uma qualidade de sistemas de software caracterizada pelo impacto causal que aspectos da escalabilidade do ambiente do sistema e seu projeto tem em certas qualidades mensuráveis a medida que estes aspectos variam dentro uma faixa operacional esperada. Se um sistema pode acomodar esta variação de alguma maneira que é aceitável para os interessados, então o sistema é escalável.” [Duboc et al. 2007]

Um sistema é escalável se ele pode “acomodar qualquer nível de desempenho ou número de usuários necessários simplesmente pela adição de recursos ao sistema [...]. Uma forma desejável de escalabilidade é um custo dos recursos que é no máximo linear em relação ao desempenho ou uso do sistema.” [Messerschmitt 1996]

“Escalabilidade é a medida da habilidade de um sistema de, sem modificações e com custo eficaz, prover uma maior vazão, tempo de resposta menor e/ou suportar mais usuários quando mais hardware é adicionado.” [Williams e Smith 2004]

“Em uma arquitetura escalável, o uso de recursos deve aumentar de maneira linear (ou melhor) com a carga, onde carga pode ser medida como o tráfego de usuários, volume de dados, etc. Onde desempenho é considerado como o uso de recursos associados a uma única unidade de trabalho, escalabilidade é como o uso de recursos muda a medida de as unidades de trabalho crescem em número ou tamanho. Dito de outra forma, escalabilidade é a forma da curva desempenho-preço, em contraste ao seu valor em um ponto particular da curva.” [Shoup 2008]

“Escalabilidade é a habilidade de processar uma carga de trabalho maior (sem adicionar recursos ao sistema).” [Weinstock e Goodenough 2006]

“Escalabilidade é a habilidade de processar uma carga de trabalho maior através da aplicação repetida de um estratégia de custo eficaz para aumentar a capacidade do sistema.” [Weinstock e Goodenough 2006]

A partir das definições listadas anteriormente, independentemente de qual dimensão, tipo ou categoria de escalabilidade se esteja tratando, retira-se a idéia geral, o conceito, da propriedade de escalabilidade, que seria:

Escalabilidade: a capacidade de um sistema de acomodar cargas de trabalho variantes, enquanto continua a satisfazer todos os seus outros requisitos: funcionais, não funcionais, etc.

O que caracteriza a propriedade escalabilidade é capacidade de um sistema de “crescer” ou “diminuir” para se adaptar a carga de trabalho. Se a carga de trabalho aumenta o sistema deve permitir, ou ser capaz de, (em ambos os casos através de intervenção manual ou de maneira automática) suportar a carga de trabalho de alguma maneira. Se a carga de trabalho diminui o sistema deve permitir, ou ser capaz de, (em ambos os casos através de intervenção manual ou de maneira automática) ser “reduzido”, de alguma maneira, para atender a carga de trabalho e evitar que recursos de hardware e software fiquem ociosos.

Esta definição de escalabilidade trata apenas do que é a propriedade escalabilidade e não diz nada a respeito de como o sistema pode ser escalado. Consideramos neste trabalho que a maneira, ou estratégia, pelos quais um sistema é capaz de processar cargas de trabalho variantes é uma outra questão, e por este motivo esta definição trata apenas do que é a característica escalabilidade.

Para o restante deste trabalho é utilizada uma definição mais específica de escalabilidade que é a seguinte:

Escalabilidade de desempenho: a capacidade de um sistema de processar crescentes cargas de trabalho aumentando, ou mantendo, seu desempenho.

Essa definição é utilizada pois é o tipo de escalabilidade que interessa a este trabalho. Um exemplo, para processar uma carga de trabalho 10 vezes maior que a atual talvez seja necessário um hardware com mais memória, ou então seja preciso ter várias instâncias do sistema; estas são maneiras pelas quais é possível para o sistema processar cargas cada vez maiores de trabalho. Seguindo o raciocínio de separação entre a característica escalabilidade da estratégia para realizá-la, poderia ter sido feita uma definição mais sucinta: “A capacidade de um sistema de processar crescentes cargas de trabalho.”, entretanto todo sistema é capaz de processar crescentes cargas de trabalho até certo ponto, mas apenas um sistema dito escalável é capaz de fazer isto aumentando, ou mantendo, seu desempenho.

O uso do termo “desempenho” na definição se refere a qualquer métrica, ou conjunto de métricas, que possam ser utilizadas para medir o desempenho de um sistema, como por exemplo, tempo de resposta, vazão, eficiência relativa, etc. Quando um sistema é escalado é preciso saber quais métricas de desempenho estão sendo observadas, pois é possível que escalando uma métrica outra seja prejudicada. Além disso, os requisitos funcionais do sistema, claro, devem continuar a ser satisfeitos.

O motivo pelo qual a definição diz que manter o desempenho também é escalabilidade são situações onde há requisitos de desempenho e escalabilidade deste tipo: o sistema deve ser capaz de responder a todas as requisições em menos de 1 segundo com uma carga de 100 requisições/segundo” e “quando a carga aumentar, até o limite de 300 requisições/segundo,

o tempo de resposta deve continuar sendo menor do que 1 segundo. Ou seja, escalabilidade também é capacidade de um sistema de manter seu desempenho quando confrontado com uma carga de trabalho maior (independentemente de como isto foi feito).

A partir dos exemplos anteriores pode-se ter a impressão de que para ser escalável um sistema deve apenas atender alguns requisitos de desempenho, mas é importante notar que desempenho e escalabilidade tem um relacionamento muito forte [Williams e Smith 2004], e muitas vezes acabam se confundindo. Desempenho é um valor, como o tempo de resposta que é um número, como a vazão de um sistema que é um número relacionado a um período de tempo, já a escalabilidade é o comportamento do desempenho à medida que a carga de trabalho varia. A Figura 2.1 mostra um exemplo do comportamento do desempenho de um sistema à medida que sua carga de trabalho varia. Neste exemplo o desempenho do sistema diminui a medida que a carga de trabalho aumenta.



Figura 2.1: Gráfico da escalabilidade de um sistema

As definições de escalabilidade apresentadas são todas informais, no entanto, definições e medidas formais de escalabilidade foram feitas em [Weinstock e Goodenough 2006], [Luke 1994], [Williams e Smith 2004], [Duboc et al. 2007]. Para o objetivo principal deste trabalho, que são diretivas e padrões arquiteturais para sistemas escaláveis, não há necessidade, no estágio atual do trabalho, de utilizar uma definição formal de escalabilidade, pois não é objetivo do trabalho definir escalabilidade.

A partir do entendimento do conceito de escalabilidade realizado e de suas definições é possível então definir métodos de escalabilidade, onde há uma estratégia para alcançar o aumento da escalabilidade.

2.2.1 Escalabilidade Vertical

A maneira mais simples de aumentar a escalabilidade de um sistema é dar a ele mais recursos de hardware e/ou software, sendo sua definição:

Escalabilidade vertical: a capacidade de um sistema de processar crescentes cargas de trabalho aumentando, ou mantendo, seu desempenho, através da adição de

mais recursos de hardware e/ou software em cada nó de processamento utilizado pelo sistema.

Para escalar um sistema verticalmente pode-se adicionar um processador mais rápido, adicionar discos rígidos mais rápidos, adicionar mais memória, aumentar a quantidade de processos do sistema operacional, aumentar a quantidade de *file descriptors* abertos simultaneamente para cada processo, etc. Escalabilidade vertical também é conhecida como *scale up*.

Apesar de relativamente simples de ser atingida, a escalabilidade vertical possui dois problemas: é limitada, pois o desempenho do sistema pode aumentar apenas até certo ponto por melhor que seja o hardware; e, se torna financeiramente cara, pois hardware (em especial memórias) de alto desempenho podem ser muito caros.

Talvez a restrição mais importante seja o limite que eventualmente se alcança ao escalar verticalmente um sistema, pois se chegará a um ponto onde será usado o melhor hardware possível ou então se chegará a um ponto onde o sistema não será mais capaz de fazer uso de todo o hardware disponível devido a limitações em sua arquitetura e implementação. Estes dois problemas com a escalabilidade vertical exercem uma pressão para que se opte pela escalabilidade horizontal.

Escalabilidade vertical não é o principal interesse deste trabalho devido às limitações apresentadas e às características dos sistemas que se desenvolve hoje em dia, com cargas de trabalho que são facilmente identificadas como além das capacidades de um único nó de processamento e a busca por um custo mais baixo para o sistema como um todo.

É importante ressaltar que este trabalho foi inspirado por situações onde a escalabilidade vertical deixou de ser uma opção, seja por questões de custo financeiro ou por questões de cargas de trabalho muito grandes.

Entretanto, mesmo com os problemas citados acima a escalabilidade vertical não deve ser descartada ou ser usada como uma segunda opção para aumentar a escalabilidade. A escalabilidade vertical é, para muitos sistemas, a maneira mais rápida, fácil e barata de escalar um sistema.

2.2.2 Escalabilidade Horizontal

Escalabilidade horizontal está relacionada à capacidade de crescimento, de expansão, de um sistema. Escalabilidade horizontal pode ser definida como:

Escalabilidade horizontal: *A capacidade de um sistema de processar crescentes cargas de trabalho aumentando, ou mantendo, seu desempenho, através da adição de mais instâncias do sistema.*

Por exemplo, se o sistema é uma aplicação *web*, podemos aumentar a escalabilidade deste sistema utilizando várias instâncias. Escalabilidade horizontal também é conhecida como *scale out*.

A escalabilidade horizontal possui uma vantagem sobre a escalabilidade vertical: se o sistema for capaz, é possível escalá-lo mais do que com o uso da escalabilidade vertical. Teoricamente, é possível construir sistemas que possuem escalabilidade horizontal linear (ver 2.2.3). Por exemplo, suponha uma aplicação *web* (aqui deixamos de lado problemas introduzidos pela escalabilidade horizontal para facilitar o entendimento); esse sistema recebe requisições de clientes, processa as requisições e retorna respostas. Se esse sistema, utilizando determinado hardware, é capaz de processar 100 requisições/segundo, e se for necessário processar 1.000 requisições/segundo, então utiliza-se 10 instâncias do sistema para alcançar a vazão desejada. Note que neste exemplo o sistema precisa ter sido construído para que seja possível ter-se várias instâncias capazes de trabalhar juntas, sem isso não seria possível escalar o sistema na horizontal. Através da adição de mais instâncias o sistema foi escalado horizontalmente, e isto poderia ter sido feito até que fosse atingido o limite do sistema de ser executado como várias instâncias.

Além da vantagem citada acima deve-se lembrar que nos dias de hoje há a disposição hardware com bom desempenho a preço acessível. Muitas vezes pode ser mais barato escalar um sistema na horizontal ao invés de na vertical, especialmente com o uso de virtualização [Borden et al. 1989]. Escalando um sistema verticalmente poderá se chegar a um ponto onde o preço de um determinado computador, com grande poder de processamento, será superior ao preço de vários computadores de menor poder de processamento e mais baratos que poderiam ser usados para escalar na horizontal e atingir os objetivos de escalabilidade desejados. O custo pode ser ainda mais reduzido com o uso de virtualização, a aquisição de uma máquina virtual é mais barata do que uma máquina física. Lembrando que ao se investir na escalabilidade vertical introduz-se a possibilidade de SPOF (*Single Point Of Fail*) e a escalabilidade horizontal, tem como efeito colateral positivo, o aumento da disponibilidade do sistema já que para realizá-la aumentasse a quantidade de instâncias do sistema.

Para que um sistema seja escalado horizontalmente é preciso que o sistema esteja preparado para isto. Por “preparado” quer-se dizer que o sistema deve ter sido projetado e construído de tal maneira que seja possível executar o sistema em vários nós de processamento de modo que o sistema continue a funcionar, sem a necessidade de alterações em seu código fonte. É preciso que o sistema seja capaz de crescer, de ser expandido, através da adição de novos nós de processamento. Muitas vezes, a maneira, ou ainda, a arquitetura como o sistema foi construído impede que ele seja executado em vários nós de processamento ou que seja possível executar o sistema apenas em uma quantidade limitada de nós de processamento. As diretrizes e padrões arquiteturais apresentados nesta dissertação têm como objetivo a construção de sistemas capazes de serem executados em vários nós de processamento para aumentar sua escalabilidade.

Um caso em particular de escalabilidade horizontal é quando se escala na horizontal sem a adição de nós físicos de processamento. Esta situação ocorre quando um sistema composto de um único módulo lógico de processamento que possui limitações em sua arquitetura e/ou implementação, propositais ou não, e é incapaz de utilizar toda a capacidade do hardware disponível.

Nesta situação é possível executar várias instâncias do sistema em uma mesma máquina para realizar a utilização total do hardware pelo sistema. Voltando ao exemplo da aplicação *web* utilizado anteriormente, caso ele seja capaz de processar 100 requisições/segundo mas utiliza apenas 10% da capacidade do hardware, então é possível executar 10 instâncias na mesma máquina para se ter uma vazão de 1.000 requisições/segundo.

A escalabilidade horizontal tem uma grande influência na maneira como deve-se projetar e construir sistemas. Essencialmente o que se deve fazer para que um sistema seja horizontalmente escalável é distribuir o sistema e evitar gargalos (de processamento, de comunicação, etc.)

Assim como a escalabilidade vertical a escalabilidade horizontal possui desvantagens. A medida que aumenta a quantidade de instâncias de um sistema a gerência do sistema se torna mais trabalhosa e cara. Além disso, tem-se aumento do consumo de energia e espaço físico necessário para os computadores.

Depois da discussão sobre as definições de escalabilidade, uma pergunta ainda precisa ser respondida: alterar um sistema para que ele suporte maiores cargas de trabalho é escalar o sistema? Quando um sistema é alterado o que se faz é substituir o sistema atual por outro que possui uma melhor escalabilidade. Assim, neste trabalho consideramos que o ato de alterar um sistema não é escalar, mas é uma maneira de possibilitar que o sistema seja escalado (na vertical ou horizontal). Considera-se aqui que alterar um sistema é modificar seu código fonte, ajustes nas configurações do sistema não são consideradas alterações. O ajuste nas configurações de um sistema é uma maneira válida de adaptar o sistema para atender a uma carga de trabalho, pois o sistema foi projetado e construído para que fosse possível realizar estes ajustes.

2.2.3 Categorias de Escalabilidade

À medida que mais recursos são adicionados, de software ou hardware, é praticamente certo que o recurso adicionado possui ou causará alguma sobretaxa operacional no sistema e isso faz com que apenas parte da capacidade do recurso seja utilizada, quando este é inserido no sistema, para realizar trabalho efetivamente útil aos usuários. Esta medida de quanto da capacidade do recurso será realmente utilizada é chamada de *fator de escalabilidade do recurso* [Tharakan 2007]. Nesta seção serão apresentadas 3 classificações de escalabilidade baseadas no fator de escalabilidade.

Escalabilidade Linear

Escalabilidade linear significa que para cada recurso adicionado a um sistema o desempenho aumenta de maneira diretamente proporcional. Recursos podem ser hardware, software, hardware e software, ou qualquer outro necessário ao sistema. Por exemplo, se uma instância de um sistema é capaz de processar X requisições/segundo, adicionando-se mais uma instância o sistema será capaz de processar $2 * X$ requisições/segundo, se forem adicionadas n instâncias o

desempenho aumentará em $n * X$ requisições por segundo.

A Figura 2.2 mostra graficamente o que isto significa (figura baseada em [Tharakan 2007]).

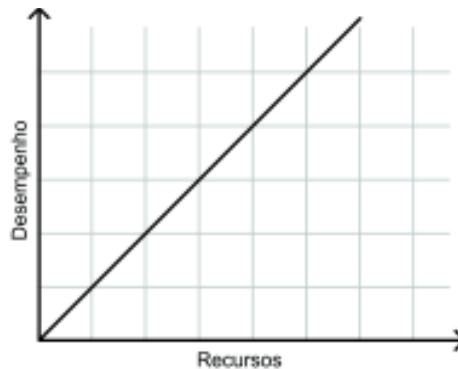


Figura 2.2: Escalabilidade linear

Na escalabilidade linear o fator de escalabilidade é igual a 1,0, o que indica que 100% da capacidade do recurso adicionado é utilizada. Um sistema que é escalável linearmente é extremamente difícil de ser feito na prática devido à complexidade dos sistemas que são compostos de software e hardware. Entretanto, sempre tenta-se construir sistemas que se aproximam da escalabilidade linear.

Com escalabilidade linear, um sistema se torna previsível, sabe-se de antemão qual será o comportamento do sistema à medida que é escalado, quanto será preciso de hardware para atender a determinada carga de trabalho e quanto será o custo financeiro. Os padrões arquiteturais apresentados neste trabalho têm sempre em vista a escalabilidade linear como objetivo.

Escalabilidade Sublinear

Escalabilidade Sublinear significa que para cada recurso adicionado a um sistema o desempenho aumenta de maneira inferior e não proporcional à capacidade individual dos recursos adicionados. Na escalabilidade sublinear o fator de escalabilidade é menor que 1,0, indicando que o sistema não é capaz de fazer uso de 100% da capacidade dos novos recursos. Um modelo formal que descreve a escalabilidade linear é a Lei de Amdahl [Amdahl 1967], [Williams e Smith 2004].

A Figura 2.3 mostra graficamente o que isto significa (figura baseada em [Tharakan 2007]). Na figura a linha pontilhada representa a escalabilidade linear, a linha sólida a escalabilidade sublinear.

A escalabilidade sublinear é o caso de ocorrência mais comum na prática devido a complexidade dos sistemas. Quando se adiciona um recurso a um sistema sempre haverá alguma sobretaxa operacional ou computacional, adicionando-se uma CPU será preciso mais comunicação entre os processadores para manter, por exemplo, a coerência de cache, adicionando-se mais memória o sistema operacional terá mais trabalho para gerenciá-la, adicionando-se mais

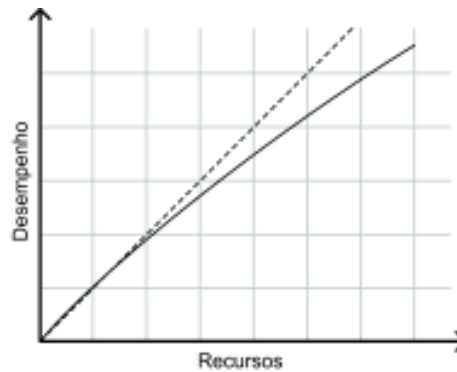


Figura 2.3: Escalabilidade sublinear

nós de processamento o balanceador de carga terá mais trabalho a fazer, se se adiciona mais uma instância do banco de dados ter-se-á sobretaxa de replicação de dados, adicionando-se mais um nó a um *cluster* haverá mais necessidade de comunicação entre os nós para poderem trabalhar juntos, e assim por diante.

Escalabilidade Superlinear

Escalabilidade Superlinear significa que para cada recurso adicionado a um sistema o desempenho aumenta de maneira superior a capacidade do recurso, o fator de escalabilidade é maior que 1,0. Isto parece impossível de ocorrer, mas é possível que se lembra que na adição de um recurso, muitas vezes são adicionados outros recursos [Williams e Smith 2004]. Quando é adicionado mais um computador, por exemplo, a um sistema, está-se adicionando ao mesmo tempo CPU, memória, discos rígidos, conectores de rede, etc. Um modelo formal que descreve a escalabilidade linear é a Lei de Gustafson [Gustafson 1988], [Williams e Smith 2004].

A Figura 2.4 mostra graficamente o que isto significa (figura baseada em [Tharakan 2007]). Na figura a linha pontilhada representa a escalabilidade linear, a linha sólida a escalabilidade superlinear.

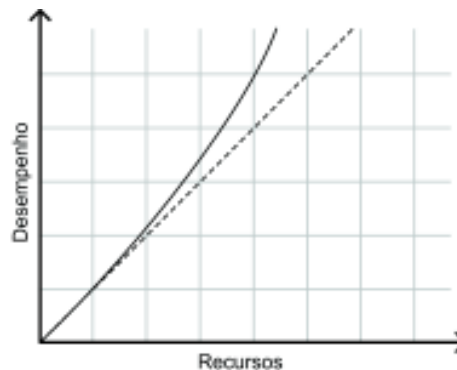


Figura 2.4: Escalabilidade superlinear

2.3 Escalabilidade e Desempenho

Escalabilidade e desempenho têm uma relação muito próxima e forte e em muitos casos escalabilidade é confundida com desempenho, entretanto elas não são a mesma coisa [Williams e Smith 2004]. Desempenho descreve em números o comportamento de um sistema, medindo sua capacidade de realização de trabalho em relação ao uso de recursos e tempo. Escalabilidade é a capacidade de realizar mais trabalho aumentando, ou mantendo, o desempenho. Como foi apresentado em 2.2 a escalabilidade é a relação entre carga de trabalho e desempenho. A partir da relação entre escalabilidade e desempenho consegue-se verificar a influência de um em relação ao outro.

A influência do desempenho na escalabilidade é direta, aumentando o desempenho aumenta-se a escalabilidade. Se um sistema é capaz de realizar mais trabalho utilizando menos recursos e menos tempo então sua escalabilidade aumenta, já que será capaz de processar cargas maiores de trabalho com desempenho satisfatório. Quando se trata de escalabilidade vertical o inverso também é verdadeiro, se se aumenta a escalabilidade adicionando mais recursos, como uma CPU mais rápida, o desempenho aumentará.

Aumentar o desempenho de um sistema é uma boa estratégia para aumentar sua escalabilidade e pode ter um custo baixo, pelo menos inicialmente. A escalabilidade vertical é uma maneira de aumentar a escalabilidade pela melhoria do desempenho através da disponibilização de mais recursos computacionais ao sistema, sem que seja preciso alterá-lo. Muitas das vezes o aumento do desempenho será suficiente para solucionar problemas de escalabilidade. Entretanto, aumentar o desempenho para conseguir escalabilidade funciona apenas até certo ponto, seja através da adição de mais recursos computacionais (como já discutido em 2.2.1) ou através da alteração do sistema.

Eventualmente, técnicas de aumento de desempenho não terão mais efeitos satisfatórios, estarão sendo utilizados os melhores algoritmos e estruturas de dados para o problema que se quer solucionar, todas as partes relevantes do sistema já terão seu desempenho medido e melhorado. Além disso, haverá um momento onde se chegará ao limite dos outros software que são a fundação do sistema (sistema operacional, servidor de aplicações, etc.), e se chegará ao limite da capacidade de processamento do hardware. Quando se chega a este ponto, de inviabilidade técnica ou financeira, a saída é a melhoria da escalabilidade horizontal.

Quando se escala na horizontal aumenta-se seu desempenho, mas não necessariamente todas as métricas de desempenho. Por exemplo, suponha um sistema que execute, em um nó e sob condições normais de carga, uma requisição em 500 ms. Com uma carga maior o processamento de uma requisição passa a levar 1500 ms. Uma nova instância é adicionada, escalando horizontalmente o sistema, dividindo igualmente a carga entre as duas instâncias, uma requisição agora volta a levar 500 ms. para ser processada. Não houve aumento efetivo na velocidade de processamento de uma requisição, apenas retornou-se ao patamar original, entretanto houve aumento real na vazão (pois agora há duas instâncias para processar requisições).

2.4 O Teorema CAP (*Consistency, Availability, Partition Tolerance*)

No ano 2000, Eric Brewer fez uma conjectura [Brewer 2000] que mais tarde foi provada como verdadeira [Gilbert e Lynch 2002]. A prova desta conjectura é conhecida como o Teorema CAP, que diz o seguinte:

Teorema CAP: *Dadas as propriedades de Consistência, Disponibilidade e Tolerância a Particionamento da rede, um serviço web pode ter no máximo duas destas propriedades.*

O teorema não se aplica apenas a serviços *web*, mas a qualquer sistema de dados compartilhados (*shared-data* ou *shared memory*). É importante deixar claro o que cada propriedade significa no contexto do teorema.

Consistência: um serviço consistente pode ser mais facilmente compreendido como um objeto de dados atômico [Gilbert e Lynch 2002]. Um serviço é dito consistente se todos os clientes que acessam o objeto de dados vêem o mesmo dado, mesmo com a ocorrência de escritas concorrentes no objeto de dados. Consistência aqui se refere a serviços atômicos [Lamport 1986a, Lamport 1986b], ou linearizáveis [Herlihy e Wing 1990].

Levando-se em conta esta garantia de consistência, deve existir uma ordem total de todas as operações, de tal forma que cada operação aparente ter sido completada em apenas um instante. Isto é equivalente a exigir que requisições para um sistema distribuído de memória compartilhada se comportem como se estivessem executando em um único nó, respondendo às operações uma de cada vez.

Para um sistema de memória compartilhada atômico, que permita escrita e leitura, como os discutidos neste trabalho, uma importante propriedade é que para qualquer requisição de leitura que se inicie após uma operação de escrita ser completada, retorne o valor escrito por esta última, ou o valor escrito por alguma outra operação de escrita posterior a esta última. Consistência atômica é diferente da consistência de transações ACID (Atômica, Consistente, Isolada, Durável) em termos da granularidade. A consistência ACID se refere a transações, enquanto a consistência atômica se refere apenas à propriedade de uma única sequência de requisição e resposta.

Disponibilidade: para que um sistema distribuído esteja continuamente disponível, toda requisição recebida por um nó do sistema, que não apresente falhas, deve ter uma resposta. Isto quer dizer que qualquer algoritmo utilizado pelo serviço para processar a requisição deve eventualmente terminar. Em um contexto de objeto de dados isto quer dizer que todos os clientes sempre podem acessar alguma versão do objeto de dados. Quando se tem em conjunto a propriedade de Disponibilidade e a propriedade de Tolerância a Partições, tem-se uma definição forte de disponibilidade, pois mesmo com falhas severas de rede toda entidade usuária requisitante deve ter uma resposta. Em um contexto de acesso a dados, disponibilidade quer dizer que

sempre será possível acessar o dado, mesmo que seja uma imagem (réplica).

Tolerância a Partições: as definições anteriores de Consistência e Disponibilidade são qualificadas com a necessidade de Tolerância a Partições de rede. Para modelar a Tolerância a Particionamento, permite-se à rede perder qualquer quantidade de mensagens enviadas de um nó para outro. Quando a rede é particionada, todas as mensagens enviadas pelos nós de uma das partições, aos nós em outra partição, são perdidas.

A propriedade de Consistência definida anteriormente implica, neste caso, que toda resposta será atômica, mesmo que uma quantidade arbitrária de mensagens enviadas como parte do algoritmo não tenham sido entregues.

A propriedade de Disponibilidade implica que todo nó que recebe uma requisição (conceitualmente uma indicação de serviço) deve retornar uma resposta, mesmo que mensagens tenham sido enviadas e perdidas. Neste cenário a única falha que faria com que o sistema respondesse incorretamente seria uma falha total da rede.

O fato de um sistema poder ter apenas duas das propriedades tem uma influência importante, pois a escolha da propriedade que será descartada define a natureza do sistema, e conseqüentemente possui impacto em sua arquitetura. Para ilustrar este ponto, serão apresentados alguns exemplos de sistemas que resultam das escolhas de duas das propriedades.

A escolha das propriedades de Consistência e Disponibilidade resulta em sistemas onde é preciso que todos os nós devem comunicar-se para manter a consistência dos dados. Exemplos de sistemas deste tipo são: Bancos de Dados hospedados em um único local; e Bancos de Dados em cluster (ou qualquer outro tipo de arranjo). Algumas características marcantes destes sistemas são o uso de protocolo 2PC (*two phase commit*) e protocolos de invalidação de cache. Estes são sistemas distribuídos clássicos.

A escolha das propriedades de Consistência e Tolerância a Partições resulta em sistemas onde deve-se tolerar o fato de o sistema parar de responder quando ocorre um particionamento de rede, já que é preciso comunicação entre todos os nós para manter a consistência. Exemplos de sistemas deste tipo são Bancos de Dados distribuídos, Sistemas de Bloqueio Distribuídos de dados. Algumas características marcantes destes sistemas são algoritmos de bloqueios pessimistas, uso de protocolos de quorum, e o fato de que partições pequenas tornam o sistema indisponível.

A escolha das propriedades de Disponibilidade e Tolerância a Partições resulta em sistemas onde nem sempre se trabalha com dados atualizados. Exemplos de sistemas deste tipo são DNS (*Domain Name System*), caches *web*, Coda, Bayou [Demers et al. 1994]. Suas características mais marcantes são uso de consistência temporal (TTL - *time to live*), uso de *leases* [Gray e Cheriton 1989], algoritmos de travas otimistas e atualização otimista dos dados com possível resolução de conflitos.

Esta combinação de propriedades é particularmente interessante, pois, por um lado, o fato de não se ter a propriedade de consistência pode parecer inviável, inaceitável e até impossível, mas que entretanto é possível de se conviver. Um outro lado é a conseqüência gerada no sistema pela

falta da consistência. O sistema, em muitos pontos, torna-se mais fácil de ser implementado e há ganhos muito grandes de desempenho e escalabilidade já que não é mais preciso se preocupar com o controle de concorrência sobre os dados. Estes fatores serão oportunamente discutidos na apresentação de alguns padrões arquiteturais para escalabilidade.

A importância do Teorema CAP no contexto de escalabilidade discutido neste trabalho se deve ao fato de que para escalar horizontalmente deve-se distribuir um sistema. Um sistema então é composto de unidades lógicas de processamento que se comunicam para cumprir as suas responsabilidades, oferecendo e consumindo serviços uma das outras.

O Teorema CAP previne que se tente construir sistemas que tenham Consistência, Disponibilidade e Tolerância a Particionamento, que são propriedades extremamente desejáveis de se ter (ainda mais ao mesmo tempo) e são itens certos em qualquer lista de requisitos de software, mas que provou-se ser impossível de se ter simultaneamente (ver prova formal em [Gilbert e Lynch 2002]).

Além disso, ele ajuda a tomar decisões arquiteturais sensatas e fundamentadas, pois sabe-se agora que deve-se escolher apenas duas das propriedades, contribuindo para que o arquiteto do sistema foque seus esforços em prover as duas propriedades escolhidas e encontrar soluções para que seja possível conviver com a ausência da terceira propriedade que não será possível prover.

Por exemplo, na construção de um sistema que deve ser escalável e disponível, escolhe-se que o sistema terá as propriedades de Disponibilidade e Tolerância a Partições e que o sistema não contemplará a propriedade de Consistência. Ter consciência de que não se pode ter consistência força o arquiteto a encontrar soluções para lidar com a situação, como por exemplo, um modelo de consistência de dados relaxado (ver 3.4).

Uma outra situação onde ganha-se vantagem com o Teorema CAP se dá com *shards* de dados (ver 3.3), onde o teorema auxilia a identificar quais dados podem ser particionados e quais não podem.

Capítulo 3

Padrões Arquiteturais para Escalabilidade

Problemas relativos à paralelização de sistemas, como, por exemplo, particionar as funcionalidades para serem executadas em paralelo, ensejam problemas clássicos de paralelismo que têm suas soluções conhecidas e bem documentadas. Por este motivo, os padrões apresentados neste trabalho, em sua maioria, tratam de problemas relacionados a armazenamento de dados e uso de transações. A atenção especial de padrões de projeto nestes problemas se deve ao fato de que são requisitos corriqueiros, presentes em quase todos os tipos de sistemas, e mais difíceis de serem solucionados ou não são maduros o suficientes para atingirem o estado dos problemas de paralelização.

Este capítulo pretende ser uma fonte de referência na qual leitores encontrarão uma discussão detalhada dos padrões mais utilizados correntemente. É um dos poucos trabalhos no mundo e único em língua Portuguesa, pelo menos isto é o que as pesquisas puderam mostrar, que reúne em um único documento uma coletânea dos principais padrões utilizados na atualidade.

3.1 Padrões: Definição e Aspectos Relevantes

Elaborar arquiteturas de software não é uma tarefa fácil. Arquitetos e projetistas experientes quando estão trabalhando em um problema quase nunca criam uma nova solução, utilizam sua experiência e conhecimento de outros problemas e suas respectivas soluções, para auxiliar na resolução do problema atual. Problemas iguais têm soluções iguais, basta adaptar a solução ao contexto atual; problemas similares têm soluções similares, basta adaptar a solução ao contexto atual. O fato importante para o desenvolvimento de software é que problemas de projeto se repetem, mas em contextos diferentes.

Por exemplo, qualquer sistema é sempre composto de subsistemas. Estes subsistemas disponibilizam uma interface para que possam ser utilizados por outros subsistemas. Eventualmente, subsistemas se tornam grandes e complexos e como consequência, quase que natural, expõem uma interface para sua utilização grande e complexa. Uma solução para este problema

é criar uma fachada de acesso ao subsistema, de tal modo que essa fachada unifique e facilite o uso do subsistema, como representado na Figura 3.1.

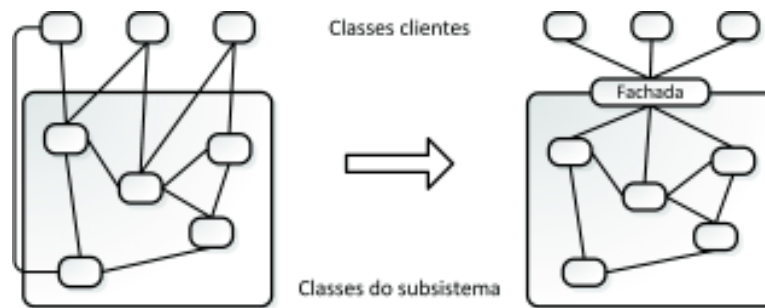


Figura 3.1: Padrão fachada

Como pode ser visto na Figura 3.1, na ilustração à direita, a fachada é uma interface de acesso de nível mais alto, disponibilizada por um subsistema para auxiliar na sua utilização por outros subsistemas. O problema de ter subsistemas grandes e complexos de serem utilizados é um problema recorrente, que ocorre em contextos diferentes; a solução para o problema, independentemente do contexto, é a mesma.

Padrões de projeto originaram-se no campo da arquitetura tradicional (a arquitetura que lida com casas, prédios, cidades, ...), em um trabalho de 1979 de Christopher Alexander [Alexander 1979]. A partir de 1987 alguns pesquisadores da área de computação apresentaram os primeiros trabalhos sobre padrões de projeto aplicados a sistemas orientados a objetos, [Smith 1987], [Beck e Cunningham 1987]. Após todos estes anos, os padrões consolidaram-se e provaram-se uma ferramenta eficaz para construção de sistemas.

Uma definição ampla do que é um padrão de projeto é a seguinte:

Padrão: Um padrão endereça um problema de projeto recorrente que surge em situações de projeto específicas, e apresenta uma solução [Buschmann et al. 2007b].

O uso de padrões tem várias vantagens [Buschmann et al. 2007b]:

- *Padrões documentam a experiência em projetos de software existente:* Isto permite o uso sistemático de experiência adquirida em muitos anos de trabalho e auxilia a arquitetos e projetistas menos experientes na criação de sistemas de melhor qualidade;
- *Padrões identificam e especificam abstrações que estão acima do nível de classes individuais e instâncias de objetos e de componentes:* Geralmente um padrão descreve vários componentes, classes e objetos, e detalha suas responsabilidades e relacionamentos, sendo que estes componentes, juntos e em acordo com o padrão, solucionam o problema colocado;
- *Padrões estabelecem um vocabulário e entendimento comuns dos princípios de projeto:* Os nomes dos padrões se tornam parte de uma linguagem de padrões que é compartilhada

por todos e esta linguagem compartilhada facilita a comunicação, o entendimento do funcionamento do sistema e a discussão de problemas de projeto, sendo que o nome do padrão pode abstrair sua complexidade, e apenas com o nome é possível comunicar toda a solução de um problema;

- *Padrões são uma forma de documentar a arquitetura de sistemas:* Se a arquitetura de um sistema for baseada em padrões este fato ajuda a documentar a arquitetura e a comunicar como o sistema funciona e evita modificações inadequadas ao sistema quando isto for necessário, estabelecendo um conjunto de regras que devem ser seguidas;
- *Padrões auxiliam na construção de sistemas com propriedades definidas:* Padrões fornecem um esqueleto do funcionamento do sistema e com isso ajuda na sua implementação e, além disso, padrões tratam de requisitos não funcionais, tais como escalabilidade, confiabilidade, reusabilidade, disponibilidade, etc;
- *Padrões auxiliam na construção de arquiteturas complexas e heterogêneas:* Todo padrão especifica um conjunto de componentes, papéis e relacionamentos entre eles, sendo que padrões atuam como blocos que podem ser utilizados para construir projetos mais complexos e de maior qualidade, diminuindo o tempo e esforço necessários para construir um sistema;
- *Padrões auxiliam a gerenciar a complexidade do sistema:* Por especificarem a solução para determinado problema, deixando claro como ela funciona, quais são os detalhes que devem ser ocultados, quais abstrações devem ser visíveis, e como tudo funciona conjuntamente, os padrões auxiliam a gerenciar a complexidade do sistema - isto é, fica mais fácil, conhecendo o padrão, entender como tudo funciona, qual a responsabilidade de cada componente, e pode-se confiar que é uma solução que funciona - sendo que, gerenciar a complexidade de um sistema é uma imperativa técnica primordial [McConnell 2004].

Para o escopo deste trabalho é utilizada uma definição mais específica de padrão:

Um *padrão para arquitetura de software* descreve um problema particular e recorrente de projeto que surge em contextos de projeto específicos, e apresenta um esquema genérico e comprovado para sua solução. O esquema da solução é especificado descrevendo os componentes que a constituem, suas responsabilidades, relacionamentos, e as maneiras pelas quais colaboram [Buschmann et al. 2007b].

Existem várias categorias de padrões, já que a área de cobertura dos padrões varia bastante. Têm-se padrões que auxiliam em como dividir um sistema em subsistemas, padrões que dizem como separar subsistemas em componentes, padrões que mostram como implementar padrões em determinadas linguagens de programação, padrões que são específicos de um domínio de problema, padrões que são independentes do domínio do problema. A partir deste fato duas

definições são feitas, a primeira com o objetivo de definir o que é um padrão arquitetural, e a segunda com o intuito de contrastá-lo com a primeira para auxiliar na categorização de padrões.

Padrão arquitetural: um padrão arquitetural expressa o esquema organizacional estrutural fundamental de um sistema. Ele provê um conjunto predefinido de subsistemas, especifica suas responsabilidades, e inclui regras e diretrizes para organizar seus relacionamentos. [Buschmann et al. 2007b]

Padrões arquiteturais são um modelo para arquiteturas de sistemas. Eles especificam a macro estrutura de um sistema, seus subsistemas e suas propriedades.

Padrão de projeto: um padrão de projeto é um esquema para refinar os componentes ou subsistemas de um sistema, ou o relacionamento entre eles. Ele descreve uma estrutura comumente recorrente de componentes que se comunicam para solucionar um problema genérico de projeto em um contexto particular [Buschmann et al. 2007b], [Gamma et al. 1995].

Padrões de projeto trabalham em uma escala menor do que padrões arquiteturais, eles não definem em sua solução, o funcionamento geral de todo o sistema, mas sim o funcionamento das partes menores do sistema, e portanto seu uso não tem influência na estrutura fundamental do sistema.

3.1.1 Estrutura e Descrição de Padrões

Para a descrição de padrões arquiteturais é utilizado o mesmo formato de [Buschmann et al. 2007b], pois é um formato claro, funcional, estruturado, prático e engloba todos os aspectos relevantes de um padrão. A seguir, é discutida brevemente a estrutura dos padrões e como serão feitas as suas descrições neste trabalho, toda a discussão é baseada em [Buschmann et al. 2007b].

A descrição de um padrão é essencialmente composta de 3 partes: Contexto; Problema e Solução.

O **Contexto** descreve situações nas quais o problema ocorre [Buschmann et al. 2007b]. Este Contexto pode ser bastante genérico, como por exemplo, “desenvolver um sistema que escala horizontalmente”, ou bastante específico como “uso de transações distribuídas”. Encontrar o contexto certo de um padrão não é tarefa fácil, determinar todas as situações nas quais o padrão pode ser aplicado muitas vezes é impossível. Aqui, como em [Buschmann et al. 2007b], tenta-se ser pragmático e lista-se a maior quantidade possível de situações em que o padrão pode ser aplicado, o que não garante que todas as situações serão listadas, mas pelo menos dará ao leitor um bom direcionamento.

O **Problema** descreve o problema que surge repetidamente no Contexto [Buschmann et al. 2007b]. A descrição do problema possui uma primeira parte que apresenta a especificação do

problema, por exemplo, “uso de transações distribuídas tem impacto negativo no desempenho”. Em seguida, a especificação do problema é complementada pela listagem e discussão das forças.

As **Forças** denotam qualquer aspecto do problema que deve ser levado em consideração durante a sua solução, como requisitos que a solução deve atender, restrições que devem ser consideradas e propriedades que a solução deve possuir [Buschmann et al. 2007b]. As forças podem ser complementares ou podem ser opostas. Por serem de grande importância, as forças são peças chave para a solução do problema. Quanto mais balanceadas as forças forem pela solução, quanto melhor será a solução, e quando o balanceamento não for possível deve-se deixar claro no padrão qual o impacto de cada força na solução.

A **Solução** descreve como solucionar o problema recorrente, ou, melhor ainda, como balancear as forças associadas ao problema [Buschmann et al. 2007b]. A solução do problema descreve aspectos **Estáticos** e **Dinâmicos**. Em relação aos aspectos Estáticos, o padrão especifica uma determinada estrutura, uma configuração específica de seus elementos, que consiste de seus componentes, seus relacionamentos e suas responsabilidades (também chamados de participantes da solução). Quanto aos aspectos Dinâmicos, a solução descreve o comportamento dos participantes da solução, como se comunicam, quando se comunicam, a maneira como colaboram entre si para resolver o problema.

Dois pontos são importantes em relação ao uso de padrões: (1) a solução de um padrão não necessariamente resolve todas as forças associadas com o problema, a solução pode focar em uma ou outra força e deixar outras sem resolução ou parcialmente resolvidas, especialmente quando as forças forem opostas; (2) um padrão apresenta um esquema de solução para um problema e não uma especificação técnica detalhada de como implementar a solução. A maneira como a solução será implementada deve ser decidida pelo arquiteto do sistema, adequando-a ao seu contexto e problemas particulares. Além da adaptação da implementação muitas vezes é preciso adaptar a própria solução, estrutura e dinâmica da solução para o problema se esta lidando.

3.1.2 Formato da Descrição dos Padrões

Neste trabalho, para se estabelecer uma base de comparação entre os cenários de aplicação e abrangências, os padrões abordados nas seções subsequentes serão descritos de acordo com o seguinte formato:

Nome: o nome do padrão e uma descrição resumida;

Resumo: um breve resumo do padrão;

Exemplo: um exemplo demonstrando a existência do problema e necessidade de um padrão - no restante da descrição do padrão o exemplo é eventualmente referenciado para ilustrar aspectos da solução;

Contexto: situações nas quais o padrão se aplica;

Problema: o problema que o padrão endereça, incluindo as forças associadas ao problema;

Solução: princípios fundamentais da solução;

Estrutura: especificação dos aspectos estruturais do padrão, com a descrição do participantes
- componentes, seus relacionamentos e suas responsabilidades;

Dinâmica: especifica o comportamento da solução, isto é, como os participantes da solução interagem para solucionar o problema;

Implementação: diretrizes para implementação da solução e quando apropriado é apresentado o código fonte para a solução;

Variantes: breve descrição de variações ou especializações da solução, quando houver;

Usos conhecidos: exemplo de uso do padrão em sistemas reais;

Consequências: prós e contras do uso do padrão;

Veja também: referências a outros padrões que solucionam problemas parecidos ou que auxiliam no refinamento da solução.

3.2 Padrão: Arquitetura *Shared Nothing*

Resumo

O padrão Arquitetura *Shared Nothing* (SNA - *Shared Nothing Architecture*) auxilia na construção de sistemas facilmente escaláveis na horizontal, a partir da estruturação do sistema em partes independentes, sem compartilhamento de estado (*Shared Nothing*), possibilitando aproximar-se da escalabilidade linear.

Exemplo

Suponha uma aplicação *web* desenvolvida para oferecer um determinado serviço pela Internet. Usuários acessam o site, realizam um cadastro onde informam vários dados sobre si e após isto podem utilizar a aplicação e usufruir de seus serviços. Este sistema foi desenvolvido utilizando-se uma arquitetura em 3 camadas tradicional, sendo elas apresentação, negócios e acesso a dados [Eckerson 1995].

Na tentativa de aumentar o desempenho e escalabilidade do sistema as camadas são fisicamente separadas, cada camada é composta de um cluster de computadores. Ainda no intuito de melhorar o desempenho e escalabilidade da solução, as camadas de apresentação e de negócio armazenam em memória dados relativos ao estado do uso do sistema realizado pelos usuários, como dados que são usados por várias requisições.

As instâncias de cada camada comunicam-se para sincronizar seu trabalho e para aumentar a disponibilidade do sistema, e há comunicação entre as camadas para sincronização de estado. À medida que o uso do sistema aumenta adiciona-se novas instâncias para atender à demanda, escalando horizontalmente. Entretanto, a cada instância adicionada é verificado que sua capacidade de trabalho não é utilizada em sua totalidade, uma instância trabalhando sozinha tem desempenho maior do que quando está trabalhando em conjunto com outras instâncias, caracterizando um fator de escalabilidade menor do que 1,0.

Além disso, não é possível prever e planejar o aumento do sistema, pois o percentual de uso, o fator de escalabilidade, de cada instância adicionada muda a cada adição. A monitoração dos computadores indica que eles sempre estão sobrecarregados. Para escalar o sistema é preciso recorrer a alterações no sistema e expedientes que tornam o sistema cada vez mais complexo de administrar.

Contexto

- Sistemas que devem escalar horizontalmente com facilidade e previsibilidade.

Problema

Na construção de sistemas distribuídos que devam escalar horizontalmente, a existência de dependência de comunicação para sincronização, armazenamento e compartilhamento de estado em memória entre as instâncias do sistema acaba por prejudicar a escalabilidade e desempenho do sistema.

Com a adição de instâncias, no intuito de escalar horizontalmente o sistema, tem-se cada vez mais interdependência entre elas e cada vez mais comunicação para que possam trabalhar juntas. Assim, são gastos cada vez mais processamento e recursos com tarefas necessárias apenas para o funcionamento do próprio sistema, criando gargalos, já que uma parte considerável dos recursos (CPU, memória, rede, etc.) estão comprometidos com estas tarefas. O sistema desperdiça capacidade de processamento que deveria estar sendo utilizado para atender a seus usuários.

Além da sobretaxa com sincronização e compartilhamento de estado, a administração do sistema se torna complexa devido a interdependência entre as instâncias como ilustrado na Figura 3.2.

Como se sabe que uma parte da capacidade de processamento disponível será usada para tarefas não relacionadas à função principal do sistema, a Lei de Amdahl se aplica [Amdahl 1967] e diz o limite teórico de aumento de desempenho quando são adicionados mais processadores (neste caso mais instâncias) a um sistema onde parte do processamento de todos os processadores não está relacionada à função principal do sistema (aqui este processamento seria a sincronização e compartilhamento de estado ou qualquer outro processamento considerado sobretaxa).

Pela Lei de Amdahl se gasta-se 10% do processamento com sobretaxa, então a adição de 100 processadores resultaria em um aumento de desempenho de 9.17 vezes ao invés das esperadas 10 vezes. Como visto em 2.2.3 a Lei de Amdahl é uma das justificativas formais para a escalabilidade sublinear.

As seguintes forças têm influência:

- A escalabilidade do sistema deve ser a mais previsível possível;
- Escalar horizontalmente o sistema não deve dificultar demasiadamente sua manutenção e administração;
- As sobretaxas de comunicação e sincronização devem ser a menor possível ou não devem existir;
- Cada recurso adicionado ao sistema deve ser utilizado em sua capacidade total ou muito próximo de sua capacidade total;
- O sistema será construído por uma equipe de desenvolvedores e deve-se continuar a ser possível dividir o trabalho em tarefas bem delimitadas.

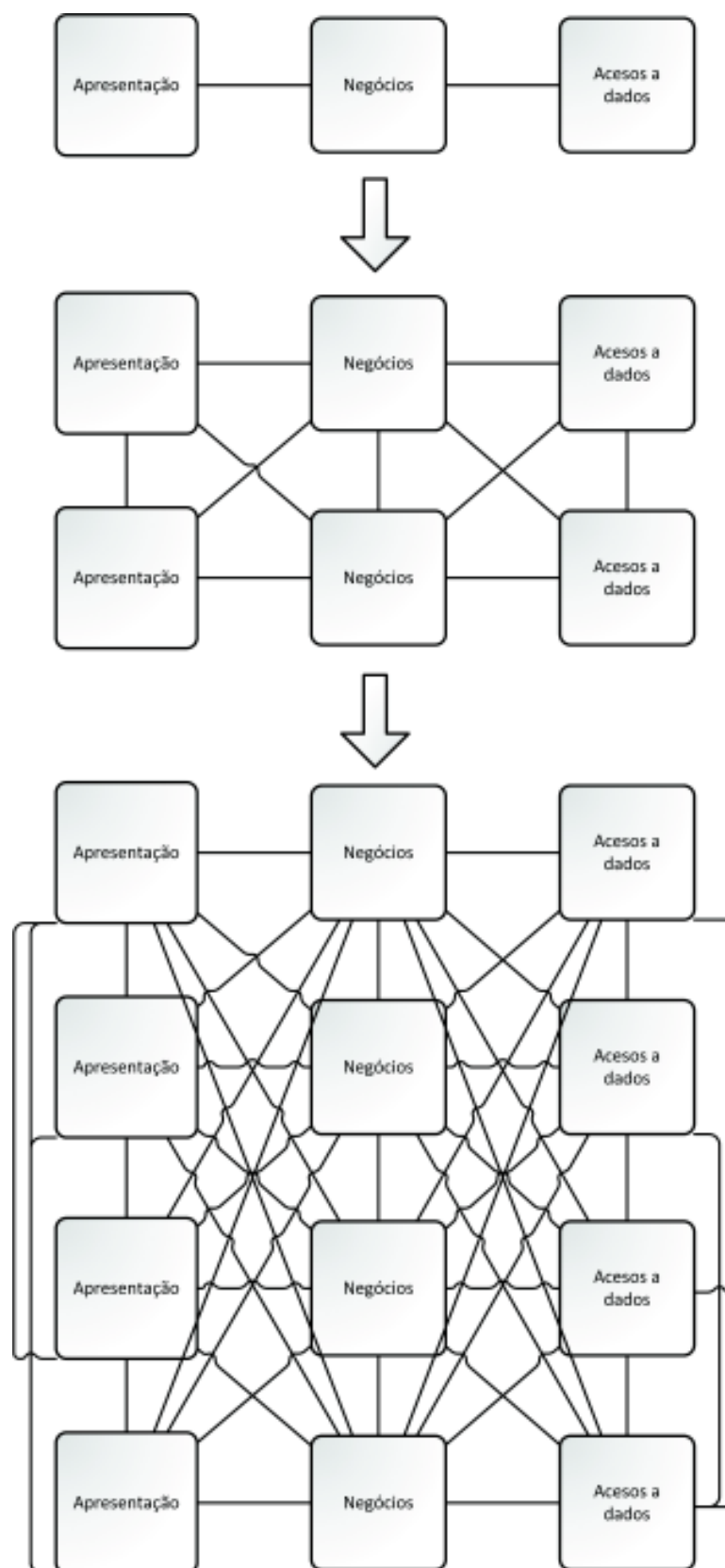


Figura 3.2: Aumento da complexidade do sistema com adição de novas instâncias

Solução

A solução é construir o sistema de maneira que cada instância seja independente, auto-suficiente, e não compartilhe nada com outras instâncias através da rede. Em uma Arquitetura *Shared Nothing* [Stonebraker 1986] cada instância mantém sua própria cópia dos dados da aplicação e utiliza algum protocolo de coordenação para interagir com outras instâncias quando necessário. Em um sistema totalmente *shared nothing*, os nós não compartilham, através de replicação, qualquer estado e nem mesmo há um banco de dados utilizado por todas as instâncias: os dados são particionados entre todas elas. A idéia geral é remover toda a dependência entre instâncias.

A solução não impede que se armazene algum estado referente ao uso do sistema pelo seus usuários, como dados referentes a determinada seqüência de requisições. O que a solução dita é que este estado não seja compartilhado entre todas as instâncias (mais detalhes de alternativas na seção de Implementação).

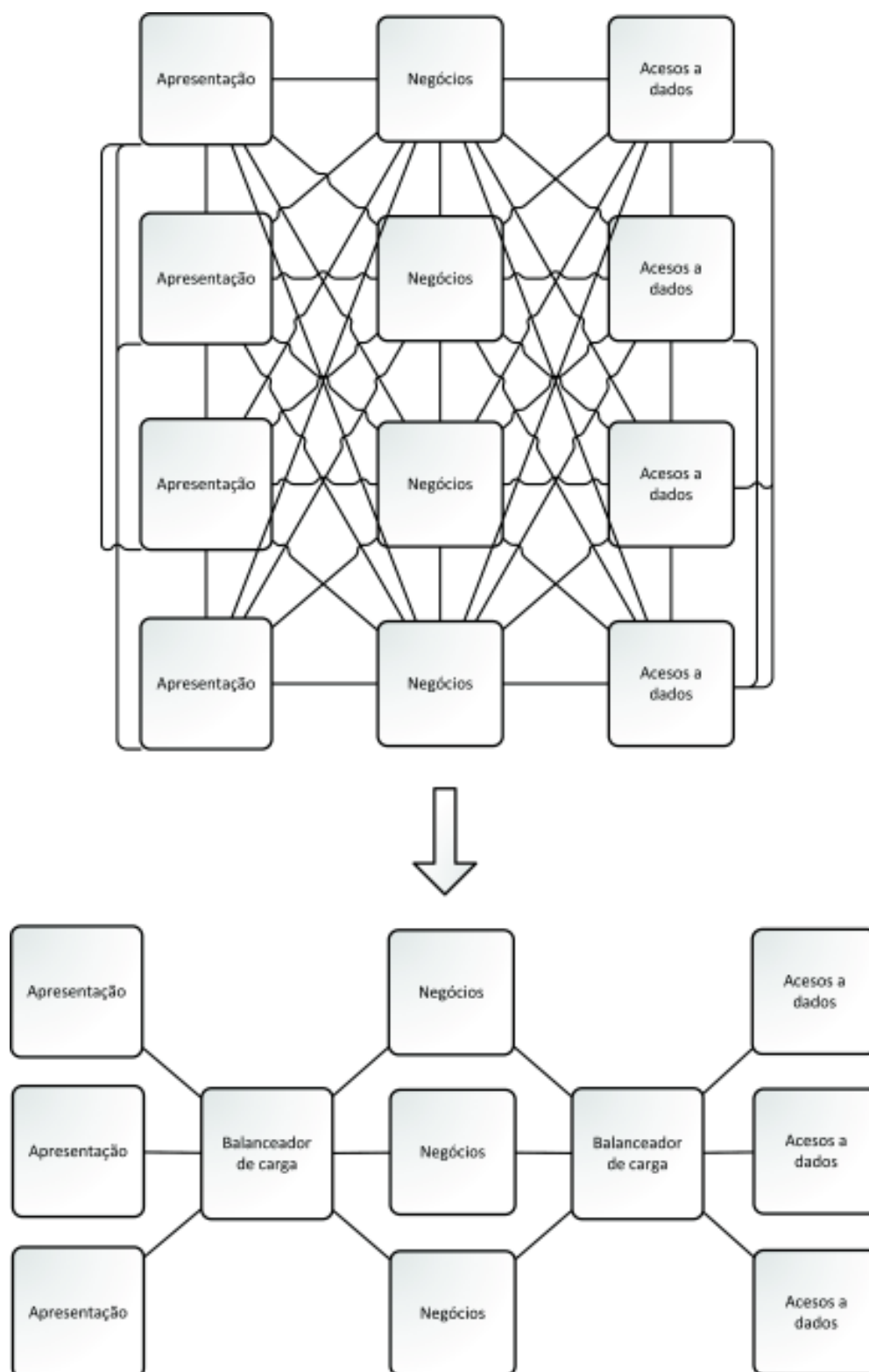
Com nós independentes e auto-suficientes não existem pontos de contenção no sistema. Como agora não há pontos de contenção e os nós não têm sobretaxa de comunicação entre si, toda a capacidade de processamento dos nós é dedicada ao processamento de requisições. O objetivo é que se um nó é capaz de processar n requisições/segundo quando adicionado a um sistema *shared nothing* ele continuará sendo capaz de processar esta quantidade de requisições, e pode-se adicionar quantos nós forem precisos que esta característica será mantida; uma SNA proporciona a possibilidade de um sistema muito próximo de ter escalabilidade linear (2.2.3).

Estrutura

A estrutura de uma SNA é simples, basta que as instâncias do sistema sejam independentes. O resultado final da aplicação da SNA depende muito do problema, apesar do resultado ser sempre instâncias independentes, as instâncias resultantes e suas responsabilidades serão diferentes para cada sistema. Continuando o exemplo do padrão a Figura 3.3 ilustra uma possível solução.

Como pode ser visto na figura, a comunicação entre as instâncias de uma mesma camada não existe mais, agora se tem apenas comunicação entre as camadas. Nesta solução ainda foram adicionados balanceadores de cargas e como não há mais estado compartilhado qualquer instância pode atender a determinada requisição. Para escalar horizontalmente esta solução adiciona-se mais instâncias a cada camada. Não necessariamente cada camada precisa ter a mesma quantidade de instâncias como ilustrado na figura. Aqui se tem um problema inserido pela solução, os balanceadores de carga tornam-se possíveis gargalos e pontos únicos de falhas. Um solução factível é utilizar mais de um balanceador de carga e várias rotas de rede.

A solução proposta para o exemplo pode ser melhorada, diminuindo-se ainda mais a necessidade de comunicação entre as instâncias, a Figura 3.4 ilustra esta nova solução.

Figura 3.3: Possível solução *shared nothing*

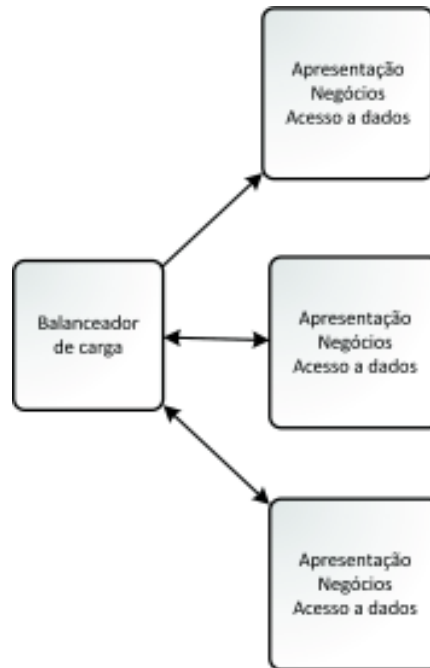


Figura 3.4: Melhoria na solução *shared nothing*

Como mostra a figura, as 3 camadas foram unidas em um único módulo de processamento e hospedadas todas em um único computador. O que foi feito aqui foi fazer com que cada instância fosse o mais auto-suficiente possível. Com todas as camadas em uma única instância não é preciso comunicação com outros nós para atender a uma requisição. Logicamente, as camadas ainda são separadas, possibilitando fácil divisão de trabalho para sua implementação.

No uso de uma SNA também se deve aplicar um particionamento funcional (ver 4.1 ao sistema). No exemplo, suponha que agora o sistema possua suporte a funcionalidades de buscas em seu conteúdo e que e-mails sejam enviados para os usuários em algumas situações específicas. Estas duas funcionalidades podem ser separadas em subsistemas como ilustrado na Figura 3.5. Com esta estrutura é possível escalar horizontalmente, e de maneira independente, as funcionalidades que o sistema agora possui.

Para a construção de um SNA, a parte mais difícil é o particionamento dos dados. Como foi dito antes, em uma solução totalmente SNA, os dados são particionados e distribuídos entre as instâncias, mas é possível utilizar outras soluções que são mais fáceis de implementar. A mais comum é não particionar os dados e usar um banco de dados compartilhado (Figura 3.6).

Um solução como a da Figura 3.6 impede a escalabilidade linear do sistema como um todo, pois o banco de dados se torna um gargalo, mas, contudo, é uma solução fácil de ser implementada. Para continuar a ter escalabilidade linear, evitando o particionamento de dados entre as instâncias e uso de um protocolo de coordenação, pode-se utilizar *sharding* na camada de dados como mostrado na Figura 3.7. Sharding é discutido em 3.3.

Conceitualmente SNAs são fáceis de compreender, mas nem sempre é fácil (ou até mesmo possível) decompor um problema em subdomínios totalmente independentes. Um resultado

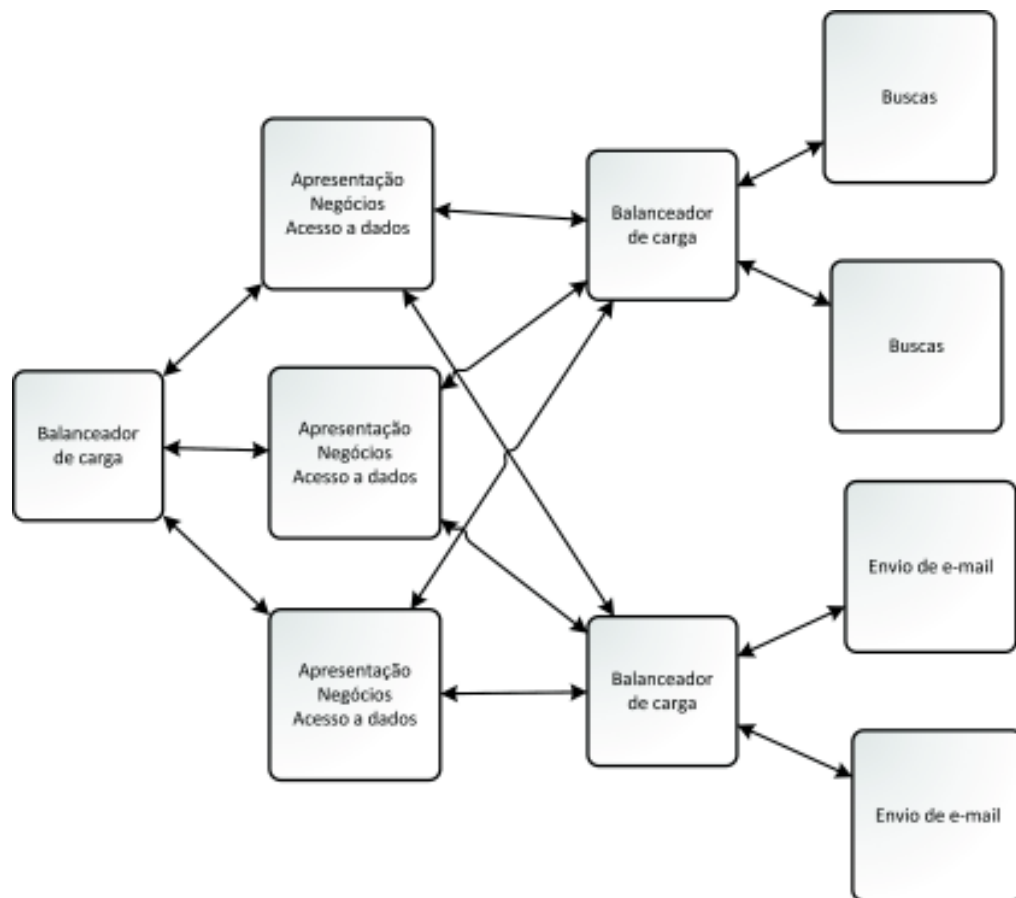


Figura 3.5: Particionamento funcional de um SNA

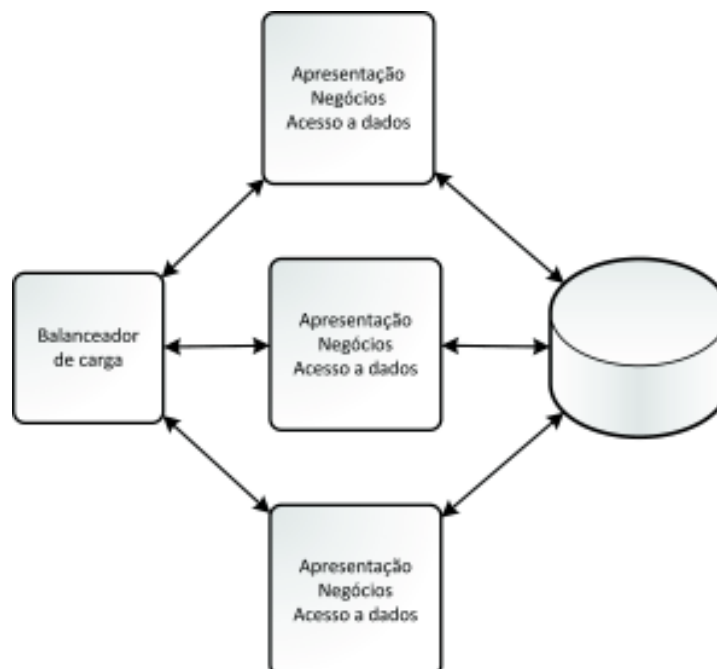


Figura 3.6: SNA com banco de dados único

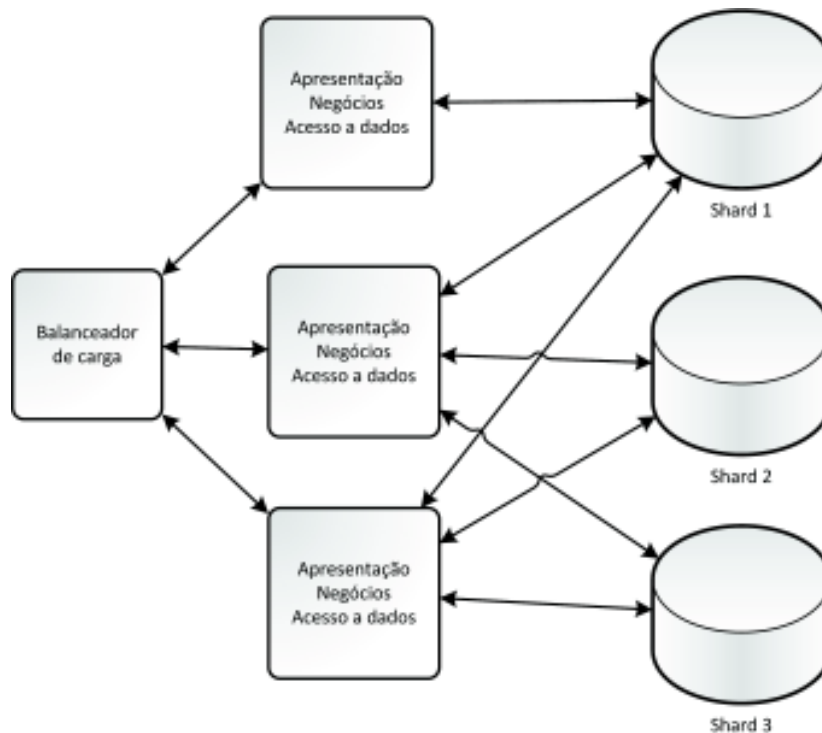


Figura 3.7: SNA com *sharding*

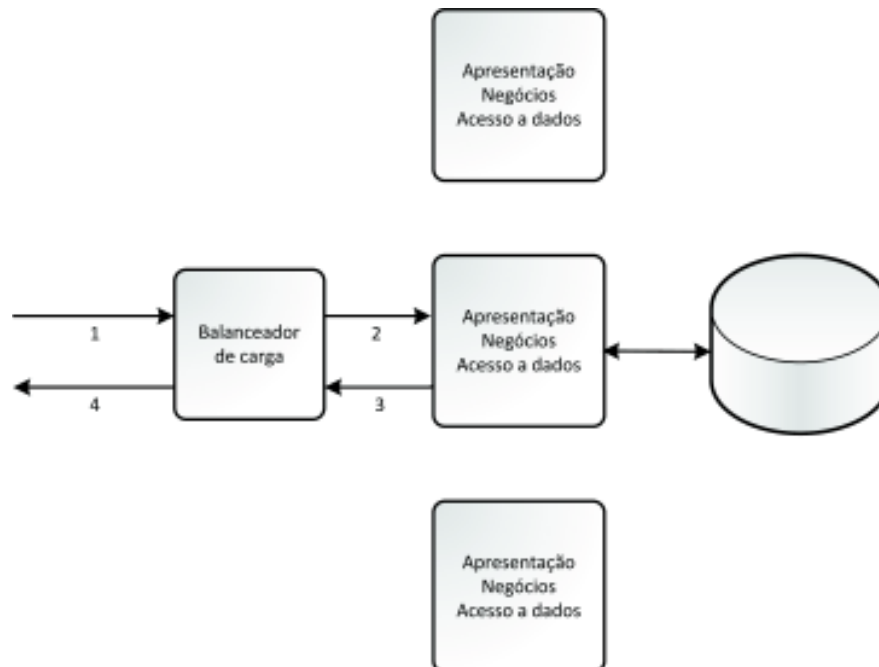
comum da aplicação do padrão são subdomínios de granularidade alta que acabam por não explorar todo o potencial de paralelismo ou subdomínios de granularidade muito baixa onde acaba-se com os mesmos problemas de sobretaxa que se queria evitar.

Devido ao fato de criar instâncias independentes do sistema, as SNA acabam resultando em sistemas compostos de *pools* de instâncias onde qualquer instância pode atender a uma requisição. A construção de *pools* pode ser vista na Figura 3.4. Nesta figura há 3 instâncias, independentes, do sistema e qualquer uma pode atender às requisições. O mesmo ocorre quando é feito um particionamento funcional como apresentado na Figura 3.5. Através destes dois exemplos percebe-se que um sistema escalável é construído a partir de subsistemas, ou sistemas, que por sua vez são escaláveis. A presença de um subsistema não escalável ou que tenha uma escalabilidade menor que outros subsistemas deve ser tratado de maneira especial, como descrito na diretrizes para escalabilidade em 4.1.

Dinâmica

Como cada instância de um sistema *shared nothing* é independente dinâmica da solução é simples e direta como na Figura 3.8.

Na figura, uma requisição é enviada e chega (indicação) ao balanceador de carga (1) que escolhe uma instância (2) e repassa a indicação. A instância processa a indicação, acessa o banco de dados (se necessário) e retorna uma resposta (3) para o balanceador que então repassa

Figura 3.8: Solução *shared nothing*

a resposta (confirmação) para quem fez a requisição (4). Esta mesma dinâmica se aplica a qualquer solução obtida pela aplicação do padrão.

Implementação

A implementação de uma SNA não exige a implementação ou utilização de algum software ou técnica específicos. São sugeridas as seguintes diretrizes para aplicação do padrão:

- *Particione o problema em subdomínios independentes de maneira a conseguir maximizar o paralelismo:* A granularidade dos subdomínios deve ser escolhida para evitar uma granularidade muito baixa, pois neste caso enfrenta-se os mesmos problemas de sobretaxa que se buscava evitar;
- *Construa unidades lógicas de processamento que possam atender à grande maioria das requisições sem ajuda:* Como foi exemplificado na seção Estrutura, na qual as 3 camadas do exemplo foram agrupadas em um único nó, isto tem a grande vantagem de aumentar o desempenho, pois tem-se menos comunicação entre os (sub)sistemas;
- *Use uma SNA apenas onde há necessidade de escalabilidade:* Alguns problemas podem não ser de fácil particionamento em subdomínios independentes, então, nestes casos, identifique as partes do problema que sejam mais suscetíveis de refinamento em subdomínios independentes;
- *Se for realmente necessário armazenar algum estado relativo ao uso do sistema por seus usuários não armazene o estado em memória:* Outras opções de armazenamento, como

sistema de arquivos ou banco de dados devem ser considerados, sendo que outra opção é fazer com que o usuário, a cada requisição, envie o estado, assim a responsabilidade de armazenamento fica com o cliente;

- *Faça balanceamento de carga:* Com vários nós independentes é importante que a carga seja dividida igualmente entre eles para evitar o aparecimento de gargalos.

Variantes

Sistemas que utilizam um banco de dados compartilhado, como o descrito no exemplo da implementação, podem ser considerados uma variante, quando se tem uma visão estreita do padrão, e considera-se uma SNA estrita, isto é, um sistema que não compartilha estado. Sistemas onde apenas algumas partes ou funcionalidades são implementadas como um subsistema *shared nothing* também podem ser consideradas variantes.

Usos conhecidos

PHP é uma linguagem de scripts muito utilizada para desenvolvimento de sites (<http://www.php.net/>). A implementação da máquina virtual do PHP segue o princípio *shared nothing* possibilitando facilidade para escalar.

memcached é um cache distribuído bastante utilizado [Danga]. As instâncias do memcached, apesar de juntas constituírem um só cache, são completamente independentes, seguindo o princípio de *shared nothing*.

Consequências

As seguintes **vantagens** são obtidas:

Escalabilidade horizontal próxima da escalabilidade linear: Com ausência de gargalos ou sobretaxa é possível ter escalabilidade próxima da linear e a adição de novas instâncias utiliza quase que totalmente a capacidade de processamento dos computadores;

Facilidade de gerenciamento: Instâncias independentes são mais fáceis de gerenciar, podendo-se adicionar e remover instâncias sem a preocupação de efeitos colaterais em outras instâncias já que estas não formam um *cluster*;

Aumento da disponibilidade: Para aumentar a disponibilidade adiciona-se mais instâncias do sistema, sendo que não é preciso se preocupar com os efeitos desta adição devido a independência das instâncias umas das outras;

Facilita a implementação do sistema em muitos dos casos: A implementação de um sistema *shared nothing* muitas vezes é mais simples, pois o arquiteto do sistema não precisa mais se preocupar com a sincronização entre as instâncias.

Como consequência tem-se as seguintes **desvantagens**:

Manter estado entre as requisições se torna mais difícil: Como não há compartilhamento de estado entre instâncias, se for mantido estado em memória, para que a arquitetura continue a ser *shared nothing*, este estado não poderá ser compartilhado com outras instâncias, forçando a mesma instância a sempre atender o mesmo cliente, sendo que para evitar esta situação se armazena o estado em outro lugar como banco de dados, cookies, sistemas de arquivos, etc.;

É difícil de ser aplicado aos dados: Distribuir os dados entre instâncias totalmente independentes é difícil e para estes casos uma solução alternativa para obter escalabilidade é utilizar *sharding* (ver 3.3).

Veja também

O padrão *sharding* (3.3) possibilita um particionamento de dados que mantém o sistema linearmente escalável. O padrão cache distribuído (3.6) possibilita aumento do desempenho para manter o sistema escalável.

3.3 Padrão: *Sharding*

Resumo

O padrão *Sharding* proporciona escalabilidade horizontal, através do particionamento dos dados em vários bancos de dados, possibilitando uma alternativa de custo mais baixo e sem as limitações da escalabilidade vertical que normalmente é aplicada a bancos de dados.

Exemplo

Suponha uma aplicação *web* que ofereça um serviço qualquer pela Internet. Usuários acessam a aplicação, realizam um cadastro onde informam vários dados sobre si e depois podem utilizar a aplicação e usufruir de seus serviços. Todas as informações dos usuários são armazenadas em um banco de dados relacional. Devido à quantidade de usuários cadastrados no sistema e o seu uso intenso, o banco de dados começa a apresentar um desempenho que não é satisfatório e torna-se indisponível em muitos momentos.

Escalar verticalmente o banco de dados não é mais uma opção, pois está se tornando cada vez mais caro e a escala que se consegue é limitada. Além disso, há no planejamento do sistema novas funcionalidades, como oferecer serviços *web* para que outros sistemas possam integrar-se a ele, e assinaturas com pagamentos recorrentes onde os usuários assinantes terão funcionalidades adicionais. Bom desempenho é um dos requisitos mais pedidos pelos usuários.

Contexto

- Sistemas com grande volume de informações, que excedem a capacidade de gerenciamento do banco de dados com desempenho satisfatório;
- Sistemas que devem apresentar alto desempenho e ter suporte a grande quantidade de acessos simultâneos a dados armazenados em banco de dados.

Problema

Em um cenário onde o volume de dados, processados e armazenados pelo sistema, é muito grande e está sempre crescendo, há requisitos de alto desempenho, sendo que o volume de acessos simultâneos é grande tanto para leitura quanto para escrita, o que invariavelmente levará ao limite o banco de dados. Outro fato relevante é que quando há a necessidade de escalar um

banco de dados, ou a camada de dados de uma aplicação, o que geralmente se faz é adquirir computadores com maior capacidade de processamento, escalando verticalmente, e como se sabe esta abordagem é cara e limitada (ver 2.2.1.)

As seguintes Forças devem ser consideradas:

- A solução deve ser simples o suficiente para que possa ser implementada em sistemas existentes sem a necessidade de grandes alterações em seu código fonte;
- À medida que se distribui os dados isto não deve ter impacto perceptível na complexidade da implementação da solução e em seu desempenho;
- A solução deve suportar mais de uma estratégia para distribuir os dados do sistema;
- A solução não pode tornar a manutenção do sistema e de seus dados excessivamente complexa;
- A reorganização dos dados, quando necessária, deve ser possível e viável tecnicamente;
- A solução deve englobar tanto o problema das leituras quanto das escritas realizadas nos dados do sistema;
- A solução deve ser previsível para possibilitar o planejamento do crescimento do sistema;
- Deve ser possível realizar a agregação de dados espalhados em vários *shards* com desempenho satisfatório.

Solução

A solução é a distribuição dos dados da aplicação em vários bancos de dados. Deve ser feito um particionamento lógico dos dados de tal forma que os dados sejam atribuídos a partições diferentes em bancos de dados diferentes. Cada partição é chamada de *shard* e a técnica é chamada de *Sharding*. Realizar *Sharding* de um banco de dados significa dividi-lo em instâncias menores, particionando e distribuindo os dados em um número de bancos de dados. *Sharding* é um método de particionamento horizontal de dados que tem como objetivo a escalabilidade horizontal e o desempenho a um preço acessível.

Com o uso de *Sharding*, a aplicação é responsável por implementar o particionamento dos dados através de alguma estratégia pré-definida e por agregar e garantir, quando necessário, a consistência dos dados, já que estes agora estão espalhados em vários bancos de dados.

Sharding de dados não é uma maneira de replicar, realizar backup ou construir *clusters* de banco de dados, é uma técnica para dividir e espalhar dados em várias instâncias de bancos de dados.

Estrutura

O uso de *Sharding* não tem uma grande influência na elaboração da arquitetura de um sistema quando feito de maneira correta, não sendo uma solução que permeia todo o sistema, mas que tem grande impacto no acesso e distribuição dos dados nos banco de dados e conseqüentemente na camada de acesso a dados de um sistema.

A Figura 3.9 ilustra o uso de *Sharding*, na qual quer-se transformar o banco de dados em vários bancos.

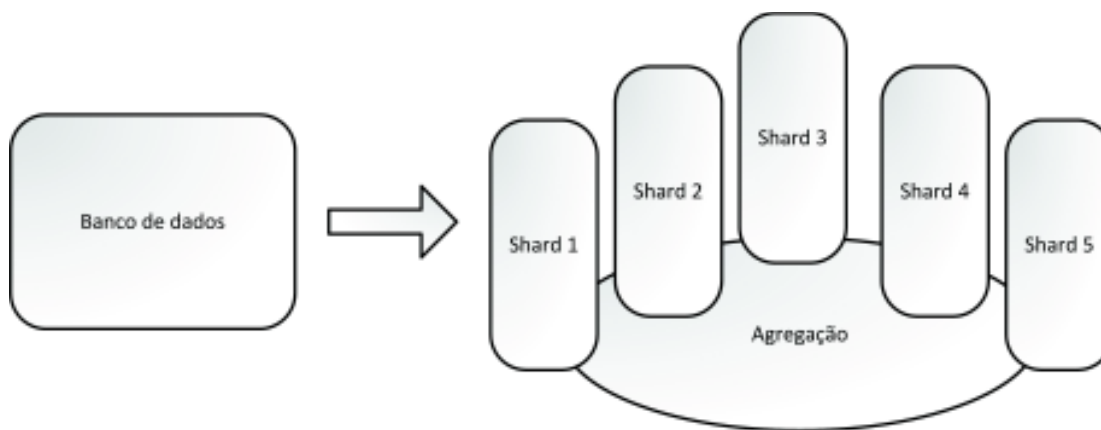


Figura 3.9: Objetivo do *sharding*

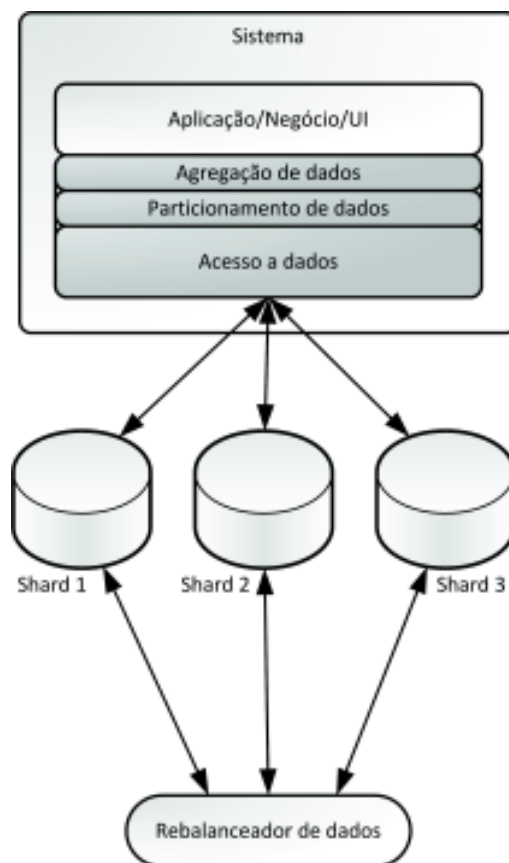
Os principais componentes a serem considerados no uso de *Sharding* são: os *Shards*; o esquema de particionamento; o agregador de dados; e o rebalanceador de dados. A Figura 3.10 detalha esta estrutura, utilizando uma arquitetura em camadas.

Os *shards* são os bancos de dados. A camada de acesso a dados é a parte do sistema responsável por acessar as informações armazenadas nos bancos de dados. A camada de esquema de particionamento de dados localiza-se acima da camada de acesso aos dados da aplicação e tem como responsabilidade determinar em qual *shard* um dado será, ou está, armazenado. O esquema de particionamento de dados é tratado aqui como uma camada separada, de maneira abstrata, para suportar qualquer tipo de implementação, desde as mais simples até as mais complexas.

Uma vez determinado em qual *shard* está um dado, a responsabilidade de acessar o *shard* e atuar sobre o dado é da camada de acesso a dados. A camada de agregação de dados tem como responsabilidade agregar e consolidar dados que estão armazenados em *shards* diferentes, sendo que esta camada nem sempre está presente.

O rebalanceador de dados é um componente à parte que tem como responsabilidade realizar o rebalanceamento dos dados entre os *shards* quando necessário. Sua implementação depende totalmente dos padrões de uso do sistema e do esquema de particionamento.

O esquema de particionamento de dados é um ponto de grande importância na arquitetura, pois todos os acessos a dados, para escrita ou leitura, são determinados pelo esquema de particionamento. A partir do esquema de particionamento é que se sabe quais dados estarão agrupados e quantos *shards* o sistema terá. Não há uma maneira única de particionar os dados,

Figura 3.10: Arquitetura de um sistema com *sharding*

é preciso escolher uma entre algumas opções e adaptá-la ao sistema.

Um esquema simples é o *particionamento por faixas de valores*. Neste esquema escolhe-se alguma informação particular dos registros do banco de dados e, baseado nesta informação, atribui-se o *shard*. No exemplo apresentado anteriormente poder-se-ia utilizar a primeira letra do nome do usuário para escolher o *shard*. Haveriam então 26 *shards*, um para cada letra do alfabeto, e baseado no nome do usuário seria possível determinar o seu *shard*. Apesar deste esquema ser simples, ele apresenta um ponto fraco grave que é o balanceamento dos dados nos *shards*. Pelo exemplo das letras do alfabeto, caso existam muito mais pessoas com nomes que iniciam com a letra “A” do que com a letra “W”, haverá um desbalanceamento dos dados. Um desbalanceamento grande dos dados prejudica a escalabilidade e o desempenho do sistema, pois a carga de trabalho não será distribuída igualmente entre os bancos de dados.

Outro esquema é *particionamento por chave ou hash*. Aqui escolhe-se alguma informação particular, chamada de chave, dos registros do banco de dados e usa-se este valor como entrada de uma função que determina o *shard*. Continuando o exemplo, assuma que seja atribuído um identificador único numérico a cada usuário ao se cadastrar no sistema, para identificá-lo internamente. A seguinte função matemática é usada para determinar o *shard*: $id \bmod N$. Onde id é o identificador único numérico do usuário, $\bmod$ é a operação aritmética de módulo e N é a quantidade de *shards*.

Por exemplo, para um usuário com identificador 103 e um particionamento em 5 *shards*, os dados deste usuário estariam armazenados no *shard* 3 ($103 \bmod 5 = 3$). A vantagem deste esquema é a facilidade de implementação e o bom balanceamento de dados entre os *shards*, se a chave e a função forem escolhidas corretamente (a chave a ser escolhida depende do uso do sistema e deve ser escolhida para cada sistema individualmente).

A principal desvantagem deste esquema ocorre quando se aumenta (ou diminui) a quantidade de *shards*, pois a adição de mais um *shard* implica em redistribuir todos os dados de todos os *shards*. No exemplo utilizado se for adicionado mais um *shard* o usuário de identificador 103 deveria ser realocado para o *shard* 1.

O rebalanceamento de todos os *shards* neste caso não é impossível, mas pode ser inviável, demorado e como afeta todos os *shards* pode ser preciso indisponibilizar o sistema até que todos os *shards* sejam rebalanceados. Para minimizar este problema a função que determina o *shard* deve ser bem escolhida. Uma alternativa é utilizar uma função de hash consistente [Karger et al. 1997] para que a adição de *shards* não afete todos os dados.

Uma opção um pouco diferente das anteriores é o *particionamento com tabela de consulta*. Este esquema mantém uma tabela de consulta que associa os dados a seus respectivos *shards*. O uso de uma tabela de consulta possibilita grande liberdade para implementar algoritmos de distribuição dos dados. Usando o exemplo da aplicação *web*, suponha que seja determinado o uso de 3 *shards* baseando-se na previsão de crescimento dos dados. Poder-se-ia então implementar um algoritmo de particionamento de tal forma que, quando um usuário se cadastra no sistema lhe seja atribuído um *shard* utilizando-se um algoritmo de *round robin* para balancear os usuários igualmente entre os *shards*; se um usuário se cadastra no sistema e é atribuído ao *shard* 2 então o próximo usuário será atribuído ao *shard* 3, o próximo ao *shard* 1, o próximo ao *shard* 2, e assim por diante.

A vantagem deste esquema é uma maior facilidade na redistribuição dos dados nos *shards* e a possibilidade de alterar mais facilmente o esquema de particionamento. Continuando o exemplo acima, se for adicionado mais um *shard* é possível alterar o algoritmo de distribuição para que todos os novos usuários sejam atribuídos ao novo *shard* até que este atinja determinada quantidade de dados e então volta-se ao algoritmo de *round robin*.

As desvantagens deste esquema são a possibilidade da tabela de consulta de tornar um gargalo e um desempenho inferior aos outros esquemas de particionamento, já que é preciso primeiro acessar a tabela de consulta para determinar o *shard* e depois acessar o *shard* propriamente dito.

Os esquemas descritos até aqui lidam com o particionamento e a distribuição de dados pertencentes a um mesmo domínio funcional como, por exemplo, o particionamento de usuários entre vários *shards*. Com estes esquemas todos os *shards* armazenam o mesmo tipo de dados, todos os *shards* possuem as mesmas tabelas mas com dados diferentes. Este tipo de particionamento, do mesmo tipo de dado, é chamado de *Horizontal Sharding* (ou simplesmente *Sharding*).

Uma outra maneira de particionar os dados é o *Vertical Sharding* (ou *Sharding Funcional*).

Este esquema agrupa em um mesmo *shard* dados relacionados a uma única e específica funcionalidade do sistema. Estendendo o exemplo da aplicação *web*, suponha que agora é possível que os usuários postem mensagens que são visíveis a todos os outros usuários e que eles possam colocar suas fotos no sistema.

Neste caso é possível particionar os dados em três grupos: informações sobre os usuários (nome, telefone, endereço, etc.); as mensagens postadas pelos usuários; e as fotos. Os dados referentes aos usuários seriam armazenados em um *shard*, as mensagens postadas seriam armazenadas em outro e as fotos em outro, como na Figura 3.11.

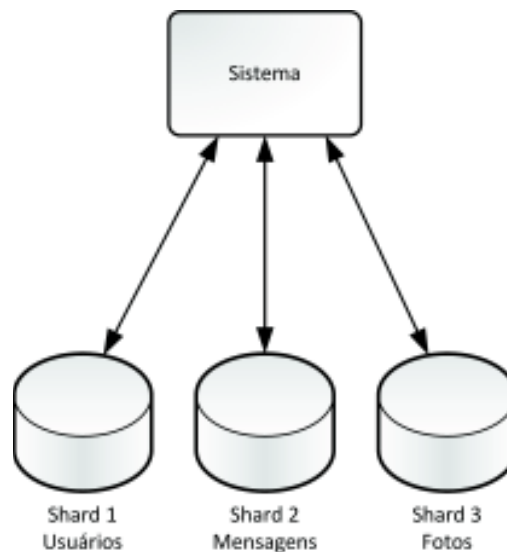


Figura 3.11: *Sharding* vertical

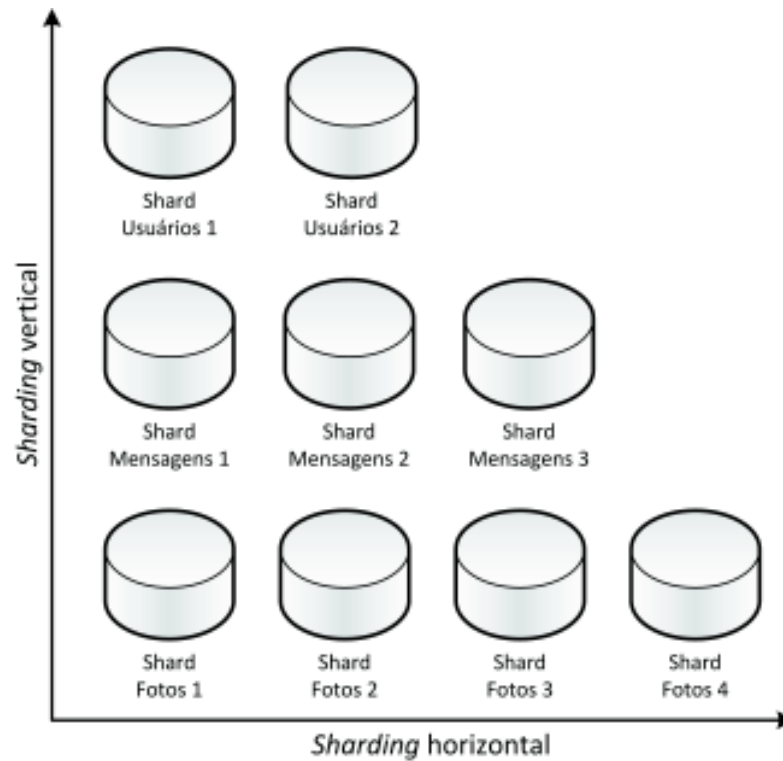
A vantagem deste esquema é a facilidade de determinar como serão particionados os dados e, principalmente, a possibilidade de combiná-lo com outros esquemas, como particionamento baseado em chaves. Outra vantagem é a liberdade de escalar horizontalmente cada partição funcional, já que são independentes. Esta possibilidade dá origem a um novo tipo de particionamento, chamado *Sharding* Diagonal. A Figura 3.12 ilustra o *Sharding* Diagonal.

Na Figura 3.12 verifica-se a aplicação de *sharding* vertical através da separação em *shards* diferentes para usuários, mensagens e fotos e *sharding* horizontal através do uso de vários *shards* para cada domínio funcional.

Dinâmica

O seguinte diagrama de sequência, Figura 3.13, ilustra o comportamento de um sistema que utiliza *sharding*.

Na figura, um usuário faz uma requisição qualquer para acessar algum dado. Esta requisição é recebida pela camada de negócios e então repassada para o agregador de dados, que por sua vez repassa a requisição para a camada de particionamento. A camada de particionamento

Figura 3.12: *Sharding Diagonal*

determina qual é o *shard* no qual o dado está armazenado e requisita à camada de acesso a dados que o dado seja acessado.

Implementação

A implementação de *sharding* não é complexa, mas deve ser bem planejada. A camada que implementa o esquema de particionamento dos dados pode ser tão simples ou tão complexa quanto se queira.

Continuando o exemplo da aplicação *web*, suponha o uso do esquema de particionamento baseado em chave ou *hash*, e que como entrada para a função de *hash* seja utilizado o identificador numérico único que identifica os usuários. A seguinte listagem, 3.3, mostra como poderia ser implementado este esquema.

```
1 Connection con = getConexaoDoShard(idUsuario);
2 Statement stmt = con.createStatement();
3 ResultSet rs = stmt.executeQuery("SELECT * FROM T_USUARIOS");
```

Primeiro, utilizando o identificador do usuário, `idUsuario`, determina-se em qual *shard* estão armazenados os dados do usuário, através do método `getConexaoDoShard`, com a conexão do *shard* correto em mãos é consultada a tabela `T_USUARIOS` para buscar as informações do usuário. A implementação do método que determina o *shard* de um usuário é apresentada em seguida na listagem 3.3:

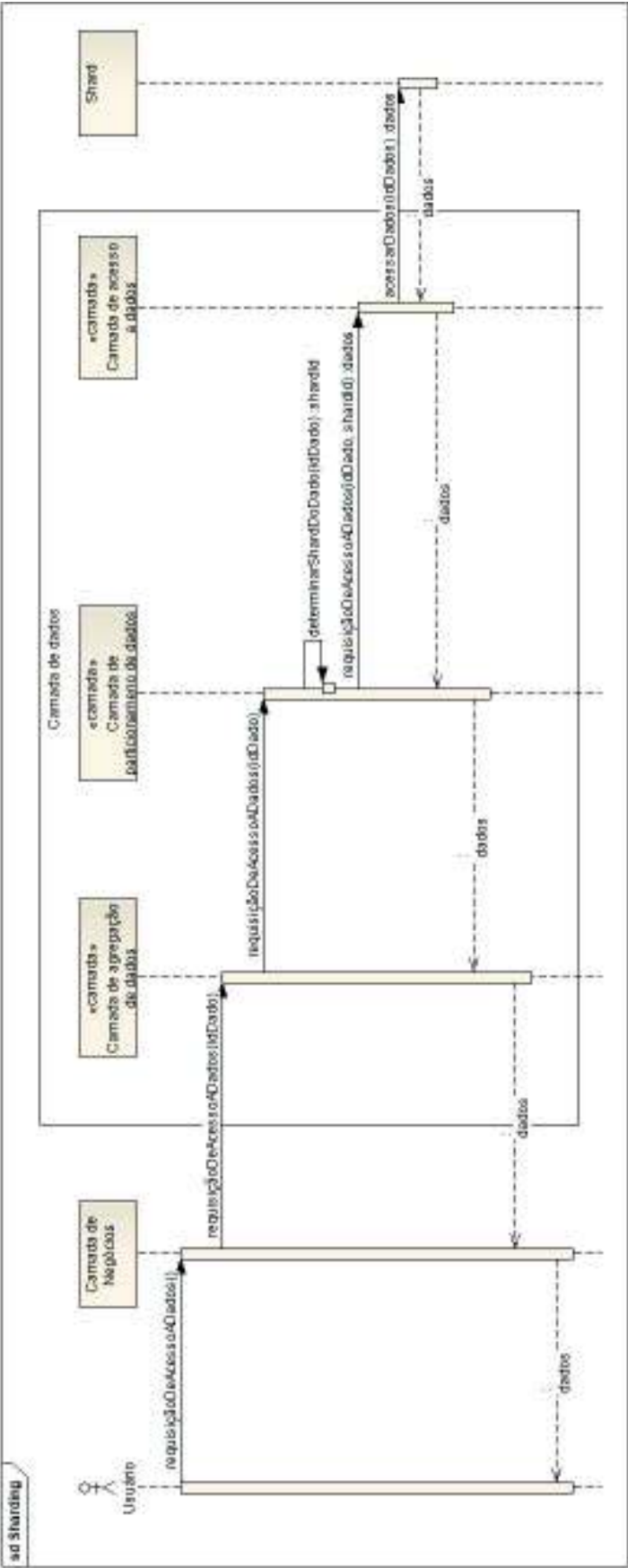


Figura 3.13: Dinâmica do *sharding*

```

1 public Connection getConexaoDoShard(long idUsuario) {
2     int idShard = idUsuario % 3;
3     return getConexaoComBancoDeDados(idShard);
4 }

```

Neste exemplo, o sistema possui 3 *shards* e uma vez identificado o *shard* na linha 2 através da função de *hash*, a conexão física com o banco de dados é feita na linha 3. A quantidade de *shards* do sistema e as configurações podem ser armazenadas de qualquer maneira pela aplicação, como em um arquivo de configuração, por exemplo. O exemplo apresentado é bastante simples de implementar, inclusive em um sistema já existente, com alterações pequenas, sendo que o mais difícil em um caso destes seria migrar os dados já existentes para seus *shards*.

Para um esquema de particionamento mais complexo, suponha o uso de particionamento com tabela de consulta, particionando os dados em *shards* baseando-se no identificador numérico do usuário. A tabela de consulta seria como outra qualquer do banco de dados e armazenaria o mapeamento entre os usuários e seus *shards*. A Figura 3.14 ilustra como seria uma tabela de consulta.

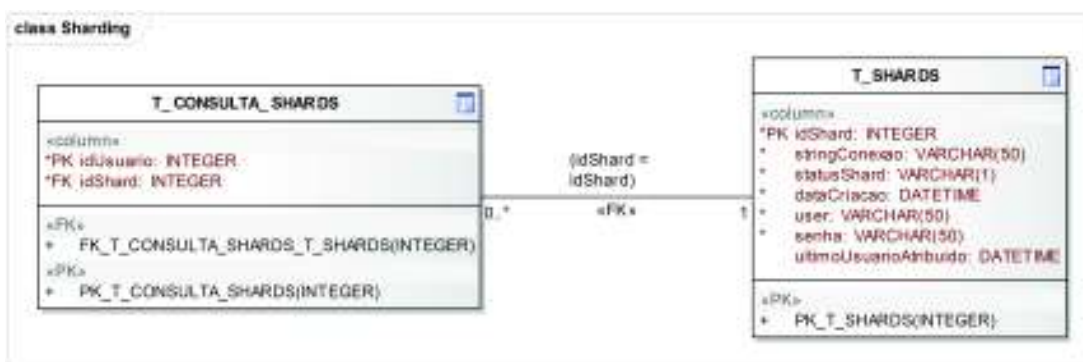


Figura 3.14: Modelo de dados da tabela de consulta

Neste caso foram utilizadas duas tabelas. A tabela T_CONSULTA_SHARDS é a tabela de consulta, a coluna idUsuario armazena o identificador do usuário, a coluna idShard é uma chave estrangeira para a tabela T_SHARDS. Esta tabela contém as informações referentes a todos os *shards*, a coluna idShard é o identificador do *shard*, a coluna stringConexao é uma string que detalha como se conectar ao banco de dados (como por exemplo “jdbc:mysql://shard1/dbname”), dataCriacao é a data em que o *shard* foi adicionado ao sistema, user e senha são respectivamente o usuário e senha para acesso ao banco de dados.

A coluna statusShard indica o status do *shard*, esta é uma coluna para auxiliar a gerenciar os *shards*. Por exemplo, caso algum *shard* não deva ser utilizado para armazenar dados de novos usuários, este fato pode ser indicado pelo status na coluna statusShard e o algoritmo de esquema de particionamento saberia que este *shard* não deve ser considerado para armazenar novos usuários.

Também poderia ser utilizada para ajudar no balanceamento dos *shards*, sendo que quando um novo *shard* for criado, pode-se alterar o status de todos os outros *shards* para indicar que não se deve armazenar novos usuários nestes *shards*. Apenas o novo *shard* teria um status que permitiria armazenamento de novos usuários e quando o novo *shard* estiver balanceado com os demais *shards* o status volta ao normal para todos. Na tabela de consulta pode-se colocar qualquer informação que auxilie na implementação do esquema de particionamento de dados. Poder-se-ia adicionar uma nova coluna para indicar a quantidade máxima de usuários que devem ser armazenados no *shard*.

A implementação de um esquema de particionamento com tabela de consulta não seria muito diferente do exemplo da implementação de particionamento baseado em chave ou *hash*. A listagem 3.3 mostra como pode ser feito.

```
1 public Connection getConexaoDoShard(long idUsuario) {
2     // cria conexão com o banco de dados que armazena a tabela de consulta
3     Connection con = ...
4     ResultSet rs = con.createStatement().executeQuery(
5         "SELECT s.* FROM T_SHARD s, T_CONSULTA_SHARDS c" +
6         "WHERE c.idUsuario = " + idUsuario + " AND c.idShard = s.idShard");
7
8     Connection shardCon = DriverManager.getConnection(
9         rs.getString("stringConexao"),
10        rs.getString("user"),
11        rs.getString("senha"));
12
13    return shardCon;
14 }
15
16 Connection con = getConexaoDoShard (idUsuario);
17 Statement stmt = con.createStatement();
18 ResultSet rs = stmt.executeQuery("SELECT * FROM T_USUARIOS");
```

Nesta listagem, o método `getConexaoDoShard` primeiro realiza uma conexão ao banco de dados (ou *shard* específico) que armazena a tabela de consulta e realiza uma consulta para determinar em qual *shard* estão os dados do usuário (linhas 1 a 6). Em seguida é estabelecida uma conexão com o *shard*, linhas 8 a 11, e a conexão é retornada na linha 13. Nas linhas 16 a 18 a conexão é utilizada como anteriormente.

Este é um exemplo simples. Em uma implementação real seria utilizado um *pool* de conexões com os bancos de dados, tanto o banco de dados que contém a tabela de consulta quanto os *shards*. Outro ponto importante é não tornar a tabela de consulta um gargalo. Para evitar este risco, armazena-se em cache a maior quantidade possível de registros da tabela de consulta.

O método `getConexaoDoShard` encontra o *shard* de um determinado usuário, mas não atribui a um novo usuário o *shard* onde suas informações serão armazenadas. Esta responsabil-

idade seria de outra parte da implementação, mostrada na listagem 3.3.

```

1 public Connection atribuirShard(long idUsuario) {
2     // cria conexão com o banco de dados que armazena a tabela de consulta
3     Connection con = ...
4     ResultSet rs = con.createStatement().executeQuery(
5         "SELECT * FROM T_SHARD WHERE idShard = (SELECT idShard FROM
6         T_SHARD ORDER BY ultimoUsuarioAtribuido ASC LIMIT 1)");
7
8     // determina através de algoritmo específico qual
9     // o \emph{shard} do novo usuário
10    int idShard = rs.getString("idShard");
11    String stringConexao = rs.getString("stringConexao");
12    String user = rs.getString("user");
13    String senha = rs.getString("senha");
14
15    // registra o \emph{shard} do usuário
16    con.createStatement().executeUpdate(
17        "INSERT INTO T_CONSULTA_SHARDS VALUES ("
18        + idUsuario + "," + idShard + ")");
19
20    // registra o \emph{shard} como o último a ter um usuário atribuído
21    con.createStatement().executeUpdate(
22        "UPDATE T_SHARDS SET ultimoUsuarioAtribuido = CURR_DATE"
23        + "where idShard = " + idShard);
24
25    Connection shardCon =
26        DriverManager.getConnection(stringConexao, user, senha);
27
28    return shardCon;
29 }
30
31 Connection con = getShardConnection(idUsuario);
32 Statement stmt = con.createStatement();
33 ResultSet rs = stmt.executeQuery("SELECT * FROM T_USUARIOS");

```

Neste exemplo de esquema, foi adicionada a tabela `T_SHARDS` e a coluna `ultimoUsuarioAtribuido` que armazena a data e hora do último usuário que foi atribuído ao *shard* (ver Figura 3.14). Nas linhas 3 e 4 é encontrado entre todos os *shards* aquele que a mais tempo não tem um usuário atribuído. Nas linhas 16 a 18 registra-se que o usuário de identificador `idUsuario` agora está atribuído ao *shard*. A coluna `ultimoUsuarioAtribuido` é atualizada nas linhas 21 a 23. Nas linhas 25 a 28 é criada uma conexão com o *shard* e ela é retornada.

Deve-se deixar claro que, apesar de ter sido utilizado nos exemplos de esquema de particionamento apenas um item de dado (identificador de um usuário), é possível, e recomendado, o uso de outros itens de dados, possibilitando mais opções de particionamento.

Como foi possível verificar, a implementação do esquema de particionamento não é complexa e não resulta em grandes alterações de código. O mais importante da implementação do *sharding* é determinar a melhor maneira de particionar os dados e como serão gerenciados os *shards*. Sobre este ponto são sugeridas as seguintes diretrizes:

- *Utilize, primeiramente, Sharding Vertical:* *Sharding Vertical* é o mais fácil de ser concebido e pode ser a solução para muitos problemas sem a necessidade de outro tipo de particionamento. Com *shards* para cada domínio funcional aqueles que possuem grande volume de leituras e escritas não tem influência nos outros *shards*;
- *Utilize Sharding Diagonal:* Em casos mais complexos, um melhor resultado será obtido com a combinação de *Sharding Vertical* e Horizontal;
- *Faça Sharding apenas daquilo que se mostrar um gargalo:* Nem todos os dados de um sistema estarão sujeitos a altas taxas de acesso, portanto, identifique os dados que possuem alta frequência de acesso, tanto escrita quanto leitura, e aplique *sharding* nestes dados;
- *Os requisitos funcionais devem ser considerados na escolha do esquema de particionamento:* Os requisitos funcionais podem ter influência em como os dados serão particionados e, então, geralmente tenta-se escolher um esquema que realize um balanceamento uniforme dos dados, sendo que os requisitos podem apontar outros caminhos além deste - por exemplo, em um sistema onde os usuários são divididos em grupos talvez seja melhor realizar o particionamento atribuindo o mesmo *shard* a usuários de um mesmo grupo [Pritchett 2008b];
- *Diferencie shards lógicos de computadores:* Não necessariamente, quando se diz *shard*, se está referindo a um computador, afinal, é possível que um computador hospede vários bancos de dados ao mesmo tempo. A diferenciação entre *shards* lógicos e computadores é importante para o planejamento da distribuição dos bancos de dados nos computadores;
- *Determine a quantidade de shards inicial e faça um planejamento:* Avalie seus dados e sua taxa de crescimento e determine quantos *shards* serão usados inicialmente. Faça um planejamento para que não seja necessário adicionar mais *shards* por algum tempo; apesar de possibilitar escalabilidade horizontal através da adição de mais *shards*, deve-se pensar que haverá mais um computador para administrar e que deverá ser feito rebalanceamento dos dados. Além disso, é uma boa alternativa ter uma quantidade razoável de *shards* como provisão para o futuro, ao invés de iniciar com 3 *shards*, caso seja viável, inicie com 10, por exemplo. Neste caso utilize inicialmente um computador para hospedar os 10 *shards*, diminuindo o custo da implementação do *sharding* [Pritchett 2008b];
- *Use matemática para planejar o crescimento da quantidade de shards:* Suponha que a quantidade inicial de *shards* seja 10 e eles serão hospedados em 2 computadores, sendo cinco *shards* em cada computador. Quando um novo computador for adicionado, os

shards terão uma distribuição de 3, 3, 4 e um dos computadores terá carga 33% maior pois possui um *shard* a mais que os outros. Caso a quantidade de *shards* inicial escolhida fosse 12, com 2 computadores, ter-se-ia 6 *shards* em cada computador. Quando um computador for adicionado resultará em uma distribuição de 4, 4, 4. Se mais um computador for adicionado resultará em uma distribuição de 3, 3, 3, 3. Cinco computadores não possibilitariam um bom balanceamento, mas seis computadores possibilitariam. Nenhuma combinação de quantidade de *shards* e computadores é perfeita, mas escolha uma quantidade de *shards* e computadores que minimize os problemas de distribuição dos *shards* nos computadores [Pritchett 2008b];

- *Implemente o esquema de particionamento com baixo acoplamento e alto nível de abstração*: A possibilidade de alteração do esquema de particionamento é real, seja por uma mudança da carga no sistema, padrões de uso, requisitos funcionais ou não funcionais ou mesmo erro na escolha do particionamento. Com uma implementação de baixo acoplamento e alto nível de abstração é mais fácil alterar o esquema de particionamento sem impacto para o restante do sistema;
- *Use os padrões de acesso aos dados para escolher a melhor alternativa de particionamento*: Os padrões de acesso aos dados indicarão com segurança quais são os dados mais acessados tanto para escrita e quanto para leitura. Os dados mais acessados devem então ser os dados particionados.

O rebalanceamento de dados nos *shards* pode ser implementado como um aplicativo separado, que monitora os *shards* e faz o rebalanceamento dos dados. Não necessariamente o balanceamento precisa ser feito constantemente, com o monitoramento dos dados dos *shards* é possível realizar o balanceamento apenas eventualmente e aos poucos.

Em algumas ocasiões, o uso de *sharding* pode trazer dificuldades. Suponha que o sistema usado como exemplo possua dois domínios funcionais, usuários e mensagens postadas pelos usuários (como descrito na seção de Estrutura), resultado de *sharding* vertical. Estes dois domínios foram então particionados por *sharding* horizontal e distribuídos em vários bancos de dados.

Se for preciso gerar relatórios com informações sobre os usuários e suas mensagens postadas será preciso consultar dados em vários *shards*. Se para a gerar os relatórios for feita uma consulta em cada um dos *shards*, em sequência, e depois for feita uma agregação dos dados, não se terá um bom desempenho e escalabilidade e à medida que a quantidade de nós aumenta seu desempenho piora cada vez mais.

Uma solução direta é realizar a consulta dos dados nos vários *shards* em paralelo. A mesma consulta é enviada, ao mesmo tempo, para todos os *shards* e os resultados então são agregados. Se os dados agregados ainda não forem suficientes ou não forem a resposta que se precisa então uma nova consulta é enviada para todos os *shards* e os resultados são novamente agregados. Este ciclo de consultas e agregação de dados é repetido até que se obtenha os resultados desejados.

A estrutura para execução de consultas em paralelo é composta de 3 participantes, a consulta em si, o coordenador de consultas, os executores de consultas e o agregador de dados. A Figura 3.15 ilustra o relacionamento dos participantes.

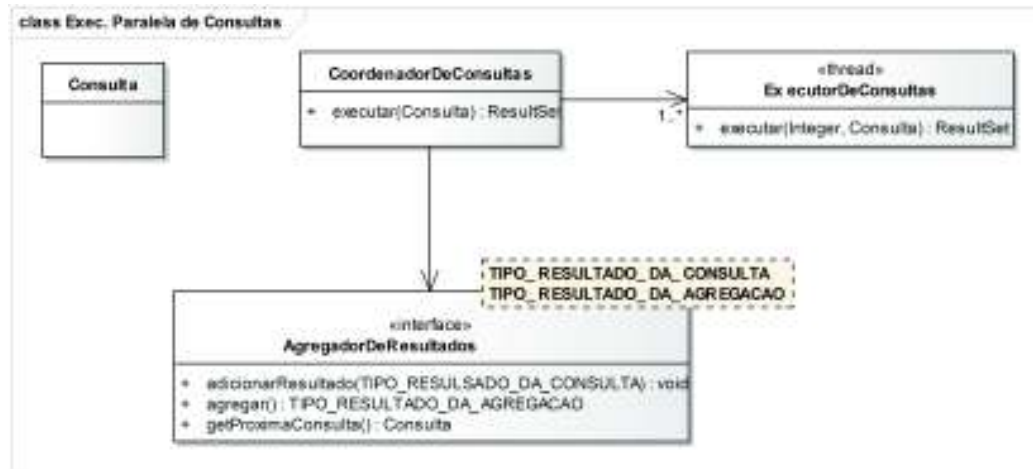


Figura 3.15: Estrutura para consultas paralela em shards

O coordenador de consultas é responsável pelo gerenciamento do processo de execução das consultas. Os executores de consultas são os componentes que realizam as consultas nas fontes de dados. O agregador de resultados é responsável por agregar todos os dados e construir o resultado desejado.

Para a execução das consultas, os participantes colaboram de acordo com a maneira descrita a seguir. O coordenador de consultas recebe um pedido de consulta, ele então determina quais *shards* devem acessados, e delega para cada executor de consulta o acesso a um *shard* e aguarda pelos resultados. Quando os resultados são retornados ao coordenador de consultas, ele os repassa para o agregador de dados, que então pode tomar duas ações, indicar que os dados retornados são os dados desejados ou requerer que uma nova consulta seja realizada. Neste último caso o processo é iniciado novamente. A Figura 3.16 mostra esta dinâmica de interação entre os participantes.

A implementação da geração do relatório do exemplo poderia ser feita da seguinte maneira (listagem 3.3):

```

1 Consulta c = criarNovaConsultaDeRelatorio();
2 CoordenadorDeConsultas coord = new CoordenadorDeConsultas();
3 AgregadorDeResultados<List<ResultSet>, ResultSet> ar =
4     new AgregadorDeResultados<List<ResultSet>, ResultSet>() {
5
6     public List<ResultSet> resultados;
7
8     public adicionarResultado(ResultSet resultado) {
9         resultados.add(resultado);
10    }
11

```

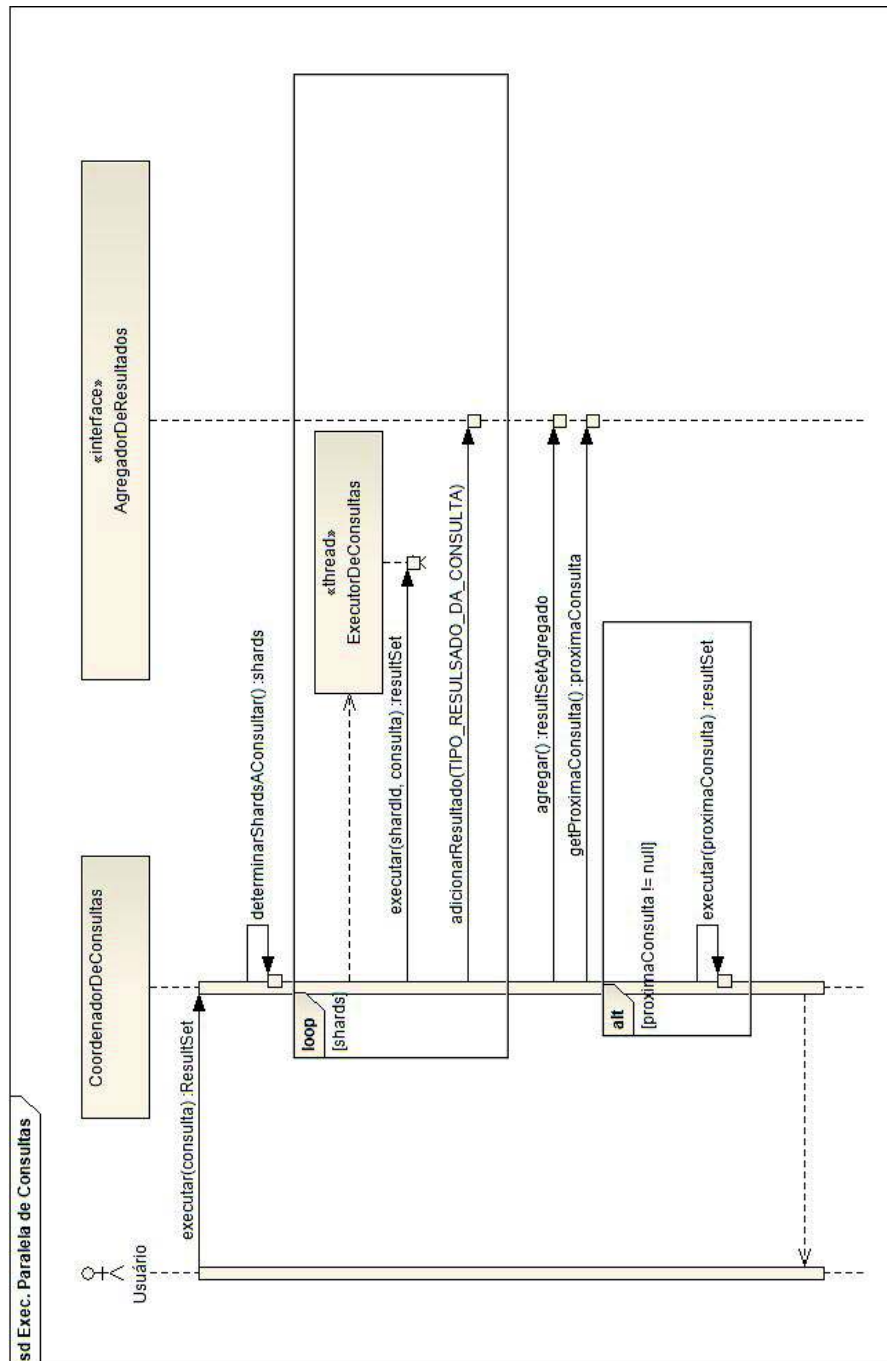


Figura 3.16: Dinâmica das consultas paralelas em shards

```
12     public List<ResultSet> agregar() {
13         return resultados;
14     }
15
16     public Consulta getProximaConsulta() {
17         return null;
18     }
19 };
20
21 coord.executar(c, ar);
```

Na linha 1 é criada a consulta que gera os dados do relatório. Em seguida é criado o coordenador de consultas, linha 2. Depois é criado e definido o agregador de resultados do relatório, linha 3. Neste exemplo é utilizado como resultado das consultas objetos do tipo `ResultSet` que fazem parte da API JDBC (*Java DataBase Connectivity*).

Na linha 6 é declarada uma lista que agrega todos os resultados. O método `adicionarResultado`, linha 8, aceita como parâmetro o resultado da execução da consulta em um dos *shards* e o armazena na lista de resultados. O método `agregar`, linha 12, apenas retorna todos os resultados das consultas, a agregação feita neste exemplo é bem simples.

Na linha 16, o método `getProximaConsulta`, indica se após agregados todos os resultados será preciso realizar uma nova consulta ou não. Aqui é retornado o valor `null`, que indica que não há mais consultas a serem feitas. Caso fosse necessário outra consulta este método construiria a consulta a partir dos resultados obtidos e a retornaria, ela seria então executada pelo coordenador de consultas.

Quando se trabalha com consultas paralelas dois pontos são importantes. Primeiro, não se deve usar transações distribuídas. Se a consulta apenas lê dados, então nem se precisa de transações. Se a consulta atualiza dados, para evitar transações distribuídas, use sagas com execução paralela (ver 3.5).

Segundo, se a consulta apenas lê dados e não são usadas transações distribuídas, deve-se lidar com falhas parciais, onde algum *shard* pode não conseguir responder à consulta. Neste caso o melhor a fazer é implementar uma estratégia de melhor esforço, ao invés de considerar a consulta como uma falha e não retornar nada aos usuários, agregue os dados que foram retornados pelos *shards*, que responderam, e retorne estes dados aos usuários. Esta estratégia de melhor esforço ajuda a aumentar a disponibilidade do sistema, pois em um sistema altamente distribuído falhas são mais frequentes.

O conceito de *sharding* não é aplicável apenas a banco de dados, mas a qualquer sistema que armazene dados. Em um sistema desenvolvido pelos autores, *sharding* foi utilizado com servidores de e-mail (SMTP [Klensin 2008] e IMAP [Crispin 1996]). No sistema desenvolvido, ao se cadastrarem, os usuários obtinham uma conta de e-mail para se comunicar com outros usuários. Os requisitos do sistema especificavam uma expectativa de 1 milhão de usuários cadastrados ao final do período de um ano. Para evitar que os servidores de e-mails se tornassem

um gargalo foi feito *sharding* dos usuários entre vários servidores de e-mail.

O esquema de particionamento utilizado foi um algoritmo de *round robin* associado a uma tabela de consulta. Durante o cadastro de um usuário era atribuído a ele o próximo servidor de e-mail da lista de servidores, pelo algoritmo de *round robin*, e a associação entre o usuário e seu servidor de e-mail era registrada em uma tabela de consulta para uso posterior. A lista de servidores continha os dados necessários para estabelecer uma conexão para envio e recebimento de e-mail e dados adicionais, como a quantidade de usuários do servidor e se o servidor estava disponível para criação de novos usuários.

Variantes

Sharding não é uma técnica de replicação, e nada impede que se use replicação de dados com *sharding*, como um esquema mestre/escravo. Com o uso de replicação é possível o aumento da disponibilidade do *sharding* e do desempenho, para isso as consultas de leituras são direcionadas às réplicas e escritas de dados são direcionadas ao mestre.

Com o uso de replicação de dados deve-se lidar com suas conhecidas consequências, como o tempo de atualização da réplica que fica dessincronizada por algum tempo (inconsistência espacial).

Usos conhecidos

Sharding é uma técnica utilizada por sites muito populares como Flickr [Flickr], Friendster [Friendster] [Pattishall 2006], LiveJournal [LiveJournal] [Fitzpatrick 2007], Facebook [Facebook].

Consequências

As seguintes **vantagens** são obtidas:

Escalabilidade horizontal próxima da escalabilidade linear: Para que o sistema comporte mais dados adiciona-se mais *shards* ao sistema;

Distribuição de carga: Com os dados distribuídos em vários bancos de dados, a carga do sistema é dividida entre eles, minimizando possíveis gargalos;

Melhoria do desempenho nos acessos de leitura e escrita: Com os dados distribuídos em vários computadores é possível executar acessos em paralelo, especialmente as escritas que são mais difíceis de serem otimizadas, consultas de leitura podem ainda ser otimizadas através de replicação do banco de dados com as consultas de leitura sendo

realizadas nas réplicas. Com *sharding* é possível executar várias consultas de escrita em paralelo;

Banco de dados menores: Com bancos de dados menores, os *shards*, eles terão um melhor desempenho, pois com menos dados é possível armazená-los em maior quantidade em memória (cache);

Aumento da disponibilidade: Se um *shard* se tornar indisponível, apenas as funcionalidades do sistema relacionadas aos dados daquele *shard* se tornarão indisponíveis - por exemplo, se os usuários do exemplo do são armazenados em *shards* e um *shard* pára de funcionar apenas uma parte dos usuários será afetado;

Implementação relativamente simples: Como visto nos exemplos de código apresentados, a implementação do esquema de particionamento de dados é relativamente simples de ser feita.

Como consequência tem-se as seguintes **desvantagens**:

Rebalanceamento de dados: A alteração da quantidade de *shards* implica no rebalanceamento de dados. O esquema de particionamento deve ser escolhido para o sistema de tal maneira a minimizar o rebalanceamento necessário. É possível que o esquema de particionamento precise ser alterado, devido a uma escolha errada ou ao chegar ao limite do esquema atual. O rebalanceamento dos dados pode tornar o sistema indisponível;

Manutenção das restrições de integridade se torna uma responsabilidade do sistema:

Como não é possível se ter restrições de integridades, como chaves estrangeiras, entre bancos de dados diferentes, esta se torna uma responsabilidade do sistema. Lidar com dados inconsistentes e a falta de restrições de integridade pode ter um grande impacto no desenvolvimento do sistema, exigindo grande esforço de implementação e manutenção. Um alternativa para minimizar esta desvantagem é utilizar um esquema de dados desnormalizado para evitar acessos a dados em mais de um *shard*;

Consultas que realizam *join* de dados se tornam difíceis: Consultas que realizam *join* de dados não têm como serem executadas em *shards*, assim isto deve ser implementado pelo sistema. Neste caso devem ser consultados os dados em mais de um *shard* e o sistema então faz o *join* “manualmente”, que é uma tarefa do agregador de dados. Uma opção para solucionar este problema é desnormalizar o modelo de dados para que não seja preciso utilizar *join*, entretanto é preciso lidar com as desvantagens da desnormalização dos dados (como dados duplicados e possíveis problemas de integridade);

Transações que acessam mais de um *shard* não possuem garantias ACID: Portanto, podem obrigar o uso de transações distribuídas. Caso a quantidade de transações que

envolvem acessos a mais de um *shard* seja pequeno, utilizar transações distribuídas é uma opção para garantir as propriedades ACID (Atomicidade, Consistência, Isolamento, Durabilidade). Caso não seja viável utilizar transações distribuídas outra saída é utilizar sagas (ver 3.5) ou utilizar rotinas periódicas que verificam e corrigem a integridade dos dados entre os *shards* ou ainda um modelo de consistência de dados relaxado (ver 3.4);

Hardware heterogêneo prejudica o desempenho: Com o uso de hardware diferentes o desempenho do sistema será experimentado de formas diferentes por seus usuários. Nem sempre é possível ter um hardware homogêneo quando a quantidade de máquinas é grande e varia com o tempo;

Manutenção mais complexa dos bancos de dados: Com *sharding* tem-se vários bancos de dados a serem administrados. *Sharding* tem influência em como os backups dos bancos de dados são feitos.

Veja também

O padrão BASE, 3.4, pode ser utilizado conjuntamente com *sharding* para uma melhor escalabilidade.

3.4 Padrão: BASE (*Basically Available, Soft state, Eventual consistency*)

Resumo

O padrão BASE *Basically Available, Soft state, Eventual consistency* possibilita a construção de sistemas nos quais se troca a consistência de dados por escalabilidade e disponibilidade, através da construção de um sistema que é basicamente disponível, lida com dados ligeiramente desatualizados e é eventualmente consistente.

Exemplo

Suponha um site de comércio eletrônico que vende livros. Devido ao grande volume de usuários e de vendas, o sistema é distribuído e escalado na horizontal. Entre as várias regras de negócio e de consistência de dados, as seguintes regras se aplicam para a realização de uma aquisição de livros: (1) “Deve haver livros suficientes em estoque antes de realizar uma remessa para cumprir uma ordem de compra”; (2) “Se o cliente pagar com cartão de crédito, deve-se garantir que o cliente tem crédito para a compra através da aprovação da compra pela administradora de cartão de crédito”; (3) “Quando uma ordem de compra for completada deve-se notificar os sistemas de cobrança e entrega”. Para garantir a consistência dos dados são usadas transações ACID (Atomicidade, Consistência, Isolamento, Durabilidade). À medida que a carga de trabalho do sistema aumenta, as transações de ordem de compra ficam cada vez mais lentas. Escalar o sistema horizontalmente não produz efeitos benéficos e para algumas operações há uma piora do desempenho.

Contexto

- Sistemas onde a escalabilidade horizontal não consegue solucionar problemas de acesso intenso a alguns dados;
- Sistemas onde o uso de replicação de dados, independentemente do objetivo (desempenho, escalabilidade, disponibilidade, etc.), se tornou um problema por ter impacto no desempenho e ter tempo excessivo para que os dados sejam copiados para todas as réplicas;
- Sistemas onde o uso de transações distribuídas têm impacto no desempenho e disponibilidade; e

- Sistemas com dados particionados (*sharding*, 3.3), onde foi usada replicação para melhoria de desempenho, escalabilidade e disponibilidade, e as situações anteriores se aplicam.

Problema

Quando é preciso garantir a consistência e a integridade dos dados é utilizado um modelo de transações ACID. A existência de alguns dados que possuem acesso intenso, com integridade garantida o tempo todo, através de ACID, cria gargalos no sistema.

Devido a estes gargalos a única opção é escalar na vertical. Escalar na horizontal não solucionará o problema, pois é um ponto único com acesso intenso. Além disso, o possível uso de transações distribuídas tem impacto no desempenho e na escalabilidade, pois é feito uso de protocolos de coordenação como 2PC (*two phase commit*).

Possivelmente, pode-se ter replicação de dados, utilizada para melhorar desempenho, escalabilidade e disponibilidade através da separação das escritas dos dados de suas leituras em bancos de dados separados, mas a replicação acaba prejudicando o desempenho por ser feita de maneira síncrona ou caso seja assíncrona gera divergência entre os dados das réplicas devido ao tempo requerido para propagar os dados para as réplicas.

Utilizando o exemplo do site de venda de livros, a restrição de integridade (1) poderia ser expressa da seguinte maneira: *quantidade_de_livros_em_estoque* > *quantidade_total_de_livros_comprados* (aqui é feita uma simplificação para ilustrar a situação, pois deveria haver um contador para cada título disponível na livraria online).

Para garantir esta restrição, o sistema possui um contador, *qtdeDeLivrosEmEstoque*, que armazena no banco de dados a quantidade de livros disponíveis no estoque. A cada ordem de compra feita, o contador é verificado e atualizado para garantir a restrição (1). O contador *qtdeDeLivrosEmEstoque* torna-se um dado muito acessado, afinal deve ser utilizado em toda venda, e em qualquer lugar onde o comprador queira saber se há livros em estoque para comprar, tornando-se um gargalo - em toda transação deve-se bloquear o acesso ao contador, atualizar o valor do contador, salvar o novo valor no banco de dados, possivelmente propagar o novo estado e desbloquear o contador.

Para garantir a restrição de integridade (2) do exemplo, a cada ordem de compra é feito acesso a um sistema externo, da operadora de cartão de crédito, para garantir que o comprador tem crédito para realizar a compra, entretanto o tempo de resposta desta operação é alto. O acesso a este sistema externo prejudica ainda mais o desempenho, pois faz parte da transação de fechamento de ordem de compra, e acaba por aumentar ainda mais o gargalo do contador *qtdeDeLivrosEmEstoque*.

Para garantir a restrição de integridade (3) o sistema utiliza transações distribuídas para garantir que os sistemas de cobrança e entrega foram notificados. O uso de transações distribuídas tem impacto negativo no desempenho e na escalabilidade do sistema.

As seguintes forças devem ser consideradas:

- Todas as forças consideradas no padrão *sharding* (ver 3.3);
- As restrições de integridade, ditadas pelos requisitos do sistema, devem ser obedecidas; e
- Gargalos de acesso a dados devem ser eliminados ou minimizados.

Solução

A solução é utilizar um modelo de consistência de dados mais fraco, relaxado, e abandonar o uso de transações distribuídas para eliminar os gargalos do sistema. Ao invés de utilizar transações ACID utiliza-se BASE (*Basically Available, Soft state, Eventual consistency*) [Pritchett 2008a]. O uso de BASE em um sistema significa que o sistema terá as seguintes propriedades:

- *Basicamente disponível*: O sistema terá tolerância a falhas parciais, mas não a falhas totais do sistema;
- *Soft state*: O sistema trabalhará com dados ligeiramente desatualizados;
- *Consistência eventual*: A propriedade de Consistência, como especificada no Teorema CAP, 2.4, não será respeitada, isto é, uma escrita em determinado dado não será visível a todo o sistema imediatamente, mas eventualmente o novo valor do dado será propagado para todo o sistema e todos os clientes e nós do sistema verão o mesmo dado [Vogels 2008].

Do ponto de vista do Teorema CAP, transações ACID possuem as propriedades de Consistência e Disponibilidade, já BASE possui Disponibilidade e Tolerância a Partições.

O uso do padrão BASE tem como princípio que a consistência dos dados não precisa ser uma questão de tudo ou nada, como nas transações ACID, sendo que a integridade dos dados é definida pelos usuários do sistema e em muitos casos existe a possibilidade de se trabalhar com uma consistência mais fraca. Quando o sistema realiza uma operação qualquer para um usuário nem sempre é necessário que os dados, ao final da operação, estejam totalmente consistentes naquele momento, mas é preciso que estejam consistentes em algum momento no futuro.

BASE é diametralmente oposto a ACID, que é pessimista e força a consistência dos dados ao final de cada operação. O padrão BASE tem uma abordagem otimista e aceita que a consistência dos dados esteja em um estado de fluxo contínuo [Pritchett 2008a].

Deve-se deixar claro que a consistência dos dados não precisa ser relaxada para todos os dados, mas em pontos específicos onde existam gargalos. Como o padrão BASE é otimista, haverá situações onde não será possível garantir a consistência dos dados e estes podem ficar inconsistentes.

Nestes casos, são iniciadas ações compensatórias para que os dados sejam levados a um estado consistente. No exemplo da venda de livros, caso tenham sido vendidos mais livros do que se tem em estoque, este fato seria detectado pelo sistema e uma ação compensatória seria iniciada para comprar mais livros para repor o estoque.

Em resumo, estas são as características de ACID e BASE:

- *ACID*:

- Consistência forte dos dados é a prioridade;
- Disponibilidade não é o mais importante;
- Pessimista;
- Respostas sempre corretas;
- Mecanismos complexos.

- *BASE*:

- Disponibilidade, desempenho e escalabilidade são as prioridades;
- Consistência fraca e eventual;
- Otimista;
- Melhor esforço;
- Simples e rápido.

Para que o sistema seja basicamente disponível utiliza-se *sharding* (ver 3.3) dos dados, com a intenção de aumentar o desempenho, a escalabilidade e a disponibilidade. Para que o sistema tenha estado *soft*, os dados que possuem acesso intenso não têm seu estado propagado imediatamente para todo o sistema, o que resultará em ganhos de desempenho. Para garantir a consistência eventual dos dados, é feita uma reconciliação ou propagação periódicas dos dados.

Por último, como agora há uma reconciliação periódica da consistência dos dados, não são utilizadas transações distribuídas, e se algum problema ocorrer em uma transação que era distribuída e agora não é, será feita uma reconciliação da consistência dos dados.

O uso de estado *soft* implica que o sistema deverá trabalhar com dados ligeiramente desatualizados, que podem não refletir a realidade. Entretanto, muitas das vezes, uma resposta aproximada e rápida é mais útil que uma resposta exata e demorada.

O que se está fazendo é trocar a consistência de dados por mais escalabilidade e ganhos de desempenho. Gargalos de acesso são eliminados em itens de dados muito utilizados, o uso de transações distribuídas é evitado e o gargalo da replicação de dados, que piora à medida que se escala horizontalmente, não mais ocorre.

Estrutura

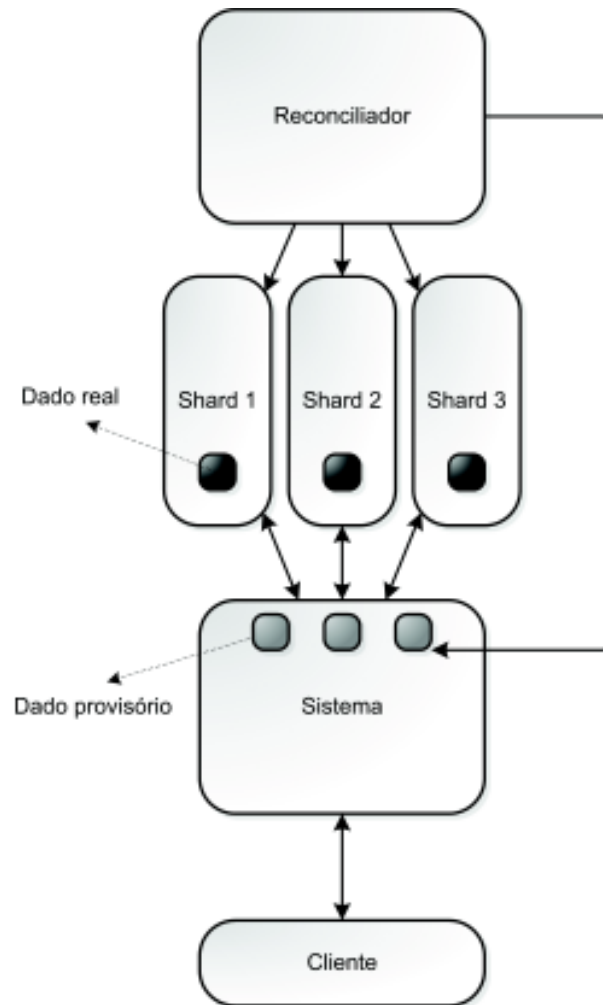


Figura 3.17: Estrutura de uma arquitetura BASE

A Figura 3.17 ilustra a estrutura e seus participantes.

Os seguintes participantes fazem parte de uma arquitetura BASE:

- *Dados particionados*: Dados particionados e distribuídos em bancos de dados (ver a estrutura de *sharding* em 3.3);
- *Dados em estado provisório*: Dados locais, possivelmente não compartilhados entre os nós, para acesso rápido, utilizados pelas transações processadas pelo sistema. Seu estado é atualizado pelas transações, mas nem sempre é propagado para todo o sistema, seu objetivo é eliminar o gargalo existente no acesso intenso a alguns dados;
- *Dados em estado real*: Dados armazenados de maneira persistente e que refletem a situação real do objeto que o dado representa;
- *Reconciliador de consistência dos dados*: Responsável por reconciliar os dados e garantir sua consistência. Periodicamente ele reconcilia os dados em estado provisório e estado real e é responsável por iniciar e/ou executar ações compensatórias para os casos onde não se pode estabelecer a consistência dos dados.

No caso de dados provisórios, uma boa alternativa de implementação é um cache distribuído como descrito em 3.6 e ilustrado na Figura 3.18.

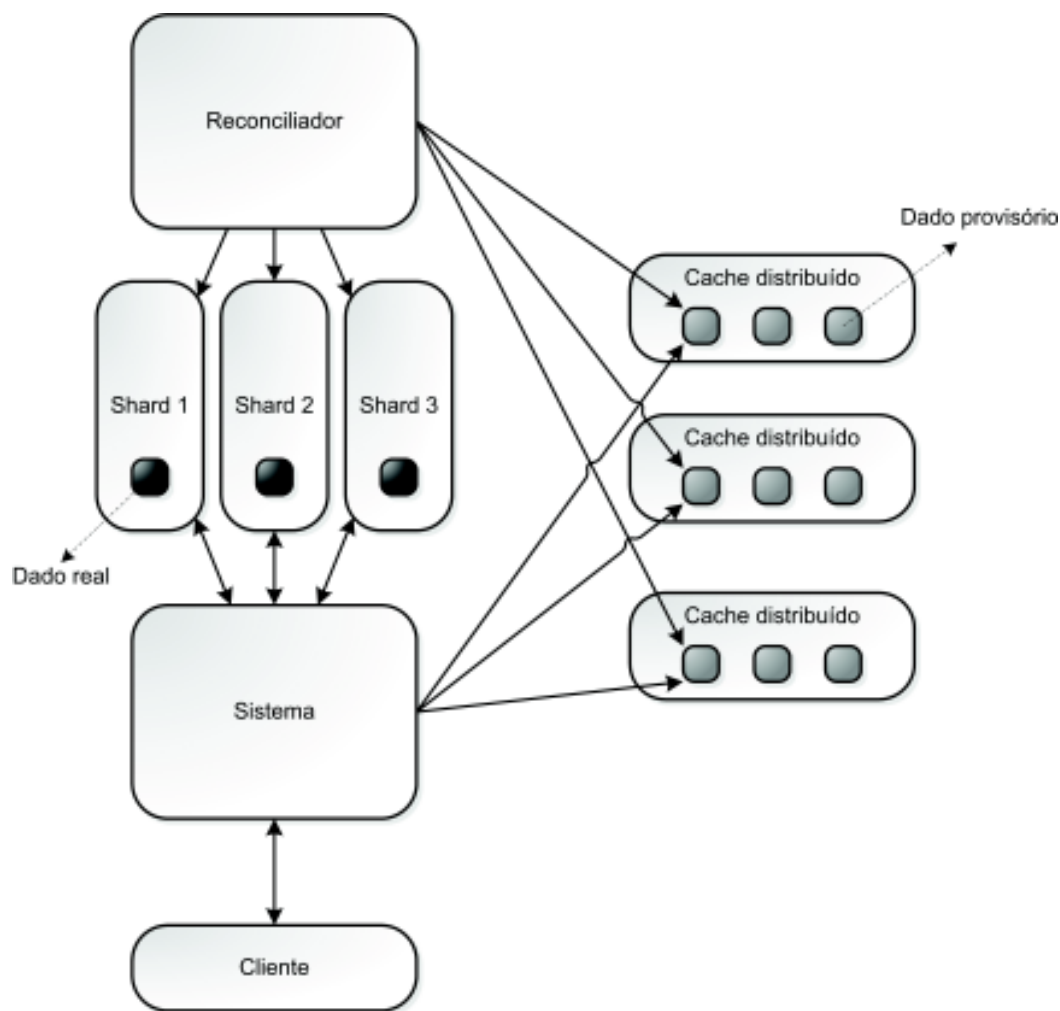


Figura 3.18: Estrutura de uma arquitetura BASE com cache distribuído

Continuando o exemplo da venda de livros online, o contador *qtdeDeLivrosEmEstoque* ficaria armazenado em memória, em estado provisório. As transações de venda atualizariam este estado, ao invés de atualizar o dado real no banco de dados. Neste caso o reconciliador, periodicamente, utilizando as ordens de compras armazenadas no banco de dados, reconciliaria os valores do contador provisório e o contador real.

Dinâmica

A dinâmica de colaboração dos participantes de uma estrutura BASE é ilustrada na Figura 3.19.

Na figura, a dinâmica é ilustrada com o uso de um cache distribuído. Primeiro um cliente faz uma requisição ao sistema (1), que então processa a requisição, armazena dados provisórios no

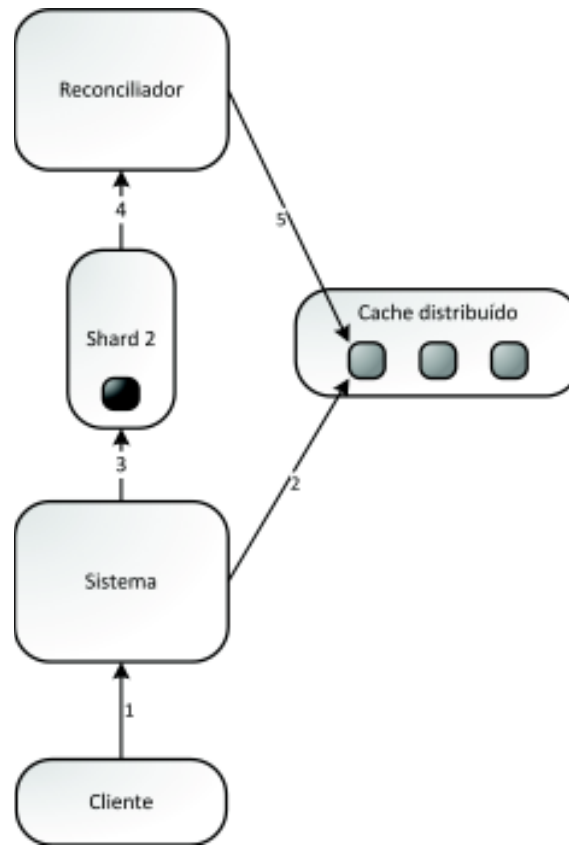


Figura 3.19: Dinâmica de uma arquitetura BASE

cache (2) e armazena dados no banco de dados (3). Não necessariamente os dados armazenados no banco de dados são os mesmos armazenados provisoriamente.

Algum tempo depois o reconciliador lê os dados do banco de dados (4), realiza a reconciliação, atualiza dados provisórios (5) e inicia ou executa ações compensatórias para os dados inconsistentes.

Na dinâmica descrita existe um intervalo de tempo onde os dados estarão desatualizados, entre a finalização do processamento da requisição e a finalização da execução do reconciliador, este período de tempo é chamado de *janela de inconsistência* [Vogels 2008].

Implementação

É difícil detalhar uma implementação de padrão BASE que possa ser usada por qualquer sistema. A implementação depende muito do sistema em si. Mesmo a dinâmica apresentada acima pode mudar de um sistema para outro e deve ser considerada como um exemplo de dinâmica.

Por isso, para ilustrar a implementação de BASE usa-se um exemplo que, espera-se, demonstrará a aplicação do padrão. Será usado um exemplo baseado em [Pritchett 2008a], que será estendido para incluir os itens apresentados na seção de exemplo do padrão. Para tornar a discussão mais focada, será utilizado como exemplo o fechamento de uma ordem de compra, que é a compra de itens no site por algum cliente.

Para o fechamento da ordem de compra devem ser obedecidas as 3 restrições de integridade apresentadas na seção ‘Exemplo’, e que são repetidas aqui:

- (1) “Deve haver livros suficientes em estoque antes de realizar uma remessa para cumprir uma ordem de compra”;
- (2) “Se o cliente pagar com cartão de crédito deve-se garantir que o cliente tem crédito para a compra através da aprovação da compra pela administradora de cartão de crédito”;
- (3) “Quando uma ordem de compra for completada devem ser notificados os sistemas de cobrança e entrega”.

Além das restrições de integridade acima, devem ser armazenadas as transações realizadas e atualizados alguns outros dados.

Para discussão do exemplo serão apresentados alguns trechos de código escritos em uma pseudo-linguagem onde tomou-se algumas liberdades para que a intenção das ações seja clara para o leitor.

Primeiramente, é feito *sharding* dos dados do sistema. A Figura 3.20 ilustra o particionamento dos dados.

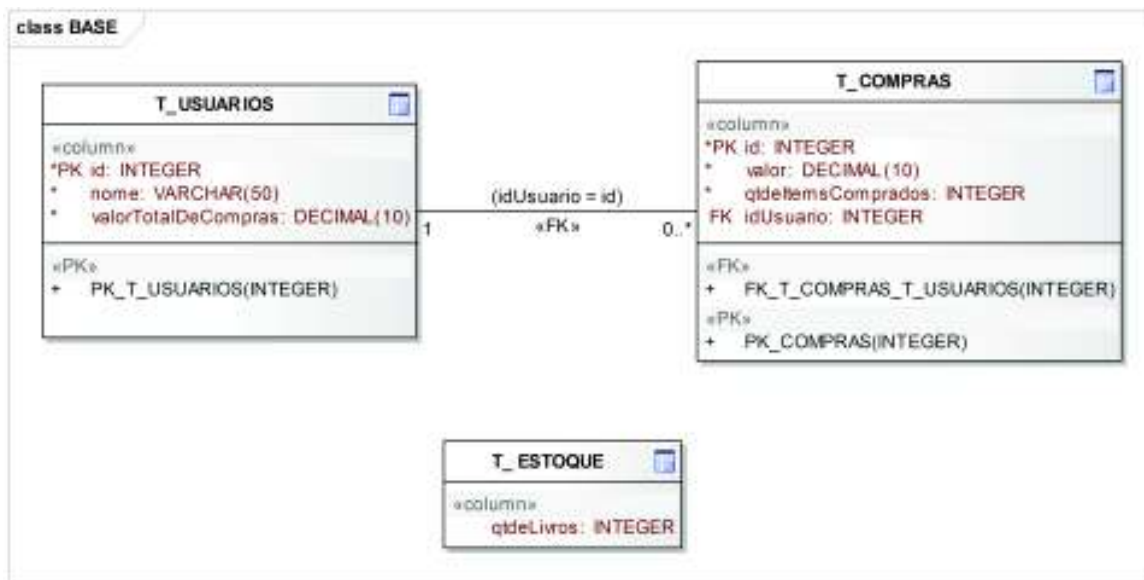


Figura 3.20: *Shards* do exemplo

No exemplo foi realizado um *sharding* diagonal (o esquema de particionamento utilizado no *sharding* horizontal, para esta discussão, é irrelevante). O *sharding* funcional criou *shards* para armazenamento de dados dos usuários, produtos oferecidos no site e transações de compra realizadas. Além destes *shards*, há outros componentes e bancos de dados que fazem parte dos sistemas de cobrança e entregas que não são mostrados na figura.

A Figura 3.21 ilustra um esquema de dados que atende as restrições de integridade.

Na tabela T_USUARIOS armazena-se dados dos clientes, incluindo o valor total das compras do cliente no site. A tabela T_COMPRAS armazena dados das compras, indicando quem foi o

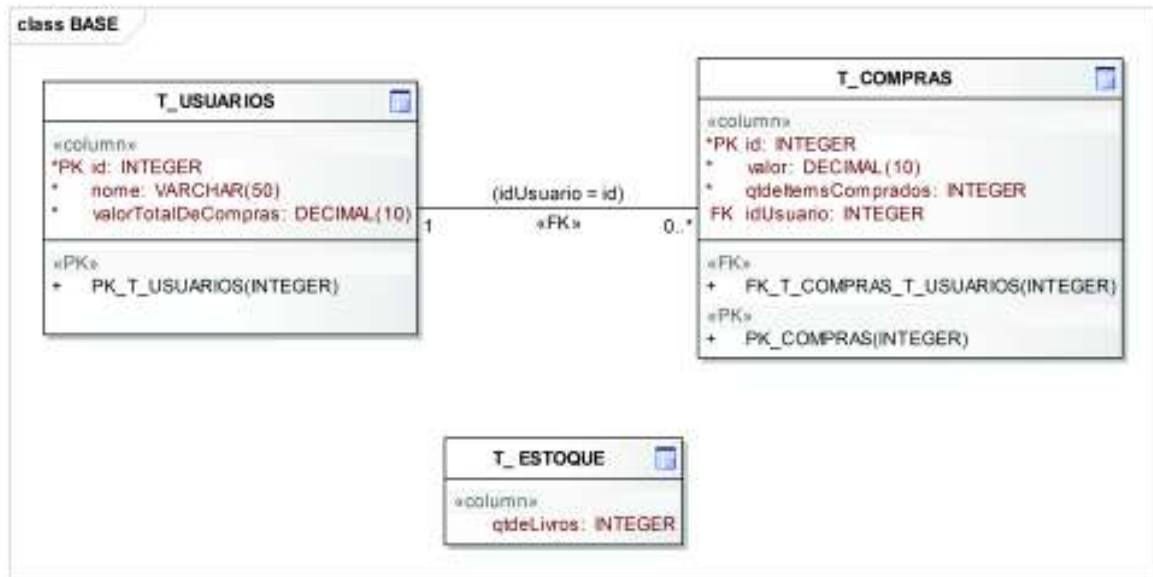


Figura 3.21: Modelo de dados do exemplo

comprador e o valor total da compra. Devido ao *sharding* funcional, as duas tabelas estão armazenadas em banco de dados diferentes. A cada compra realizada um registro é criado na tabela T_COMPRAS e o valor total comprado pelo cliente é atualizado. A tabela T_ESTOQUE armazena o contador de livros disponíveis em estoque.

Utilizando um modelo ACID, uma transação de compra poderia ser implementada da seguinte maneira:

```

1  begin transaction
2      $livrosEmEstoque = SELECT qtdeLivros FROM T_ESTOQUE FOR UPDATE;
3      if ( ($livrosEmEstoque - $qtdeItensComprados) >= 0 ) {
4          aprovarCompraComCartaoDeCredito($idComprador);
5          INSERT INTO T_COMPRAS(
6              $idTransacao, $idComprador, $valor, $qtdeItensComprados);
7          UPDATE T_ESTOQUE SET qtdeLivros = qtdeLivros - $qtdeItensComprados;
8          UPDATE T_USUARIOS SET
9              valorTotalDeCompras = valorTotalDeCompras + $valor
10             WHERE idUsuario = $idUsuario;
11             notificarSistemaDeCobrança($idTransacao);
12             notificarSistemaDeEntrega($idTransacao);
13     }
14  end transaction

```

Listagem 3.1: Transação ACID

Na linha 1 uma transação é iniciada, em seguida nas linhas 2 e 3 é verificado se há livros suficientes em estoque, para que se mantenha a consistência dos dados na linha 2 é colocada uma trava no registro lido.

O que acontece nesta listagem é que para garantir uma consistência forte, imediata e global,

deve ser feita uma serialização global das operações com dados. Com livros em estoque, a compra é aprovada com pagamento via cartão de crédito, através de uma consulta a um sistema externo na linha 4.

Na linha 5, armazena-se na tabela T_COMPRAS o registro da compra, e em seguida atualiza-se a quantidade de livros em estoque. Na linha 8, o valor total de compras realizadas pelo usuário é atualizado. Para finalizar a compra, são notificados os sistemas de cobrança e entrega, nas linhas 9 e 10, respectivamente. Todo este processo é realizado em uma única transação distribuída a qual garante total consistência dos dados.

Inicia-se a transformação da implementação ACID em uma implementação BASE pelo gargalo de acesso a tabela T_ESTOQUE. Para eliminar o gargalo e para atualizar a quantidade de itens em estoque, usa-se a quantidade de livros em estoque como um dado provisório, que pode estar desatualizado e não refletir a realidade. A listagem 3.2 ilustra esta transformação.

```
1 begin transaction
2     $livrosEmEstoque = SELECT qtdeLivros FROM T_ESTOQUE;
3     if ( ($livrosEmEstoque - $qtdeItensComprados) >= 0 ) {
4         aprovarCompraComCartaoDeCredito($idComprador);
5         insert INTO TRANSACOES($idTransacao, $idComprador, $valor, $qtdeItensComprados);
6         UPDATE T_USUARIOS SET valorTotalDeCompras = valorTotalDeCompras + $valor
7             WHERE idUsuario = $idUsuario;
8         notificarSistemaDeCobrança($idTransacao);
9         notificarSistemaDeEntrega($idTransacao);
10    }
11 end transaction
```

Listagem 3.2: Transação BASE

Observe-se que foi retirada a trava colocada no contador de livros em estoque durante a transação na linha 2. Removeu-se a linha onde era feita a atualização do contador, e agora isto é responsabilidade do reconciliador de dados, e o valor do contador de livros em estoque é tratado como um dado provisório, pois seu valor pode estar desatualizado. Como a atualização do contador é feita pelo reconciliador periodicamente, haverá uma janela de inconsistência. Ainda é feita uma validação na linha 3, para verificar se há livros em estoque, mas agora essa validação é probabilística. Há, portanto, a possibilidade de serem vendidos livros que não existem no estoque, mas não se deixará de vender livros que estão em estoque.

No caso particular deste exemplo, é possível ainda fazer mais uma otimização, como mostrado na listagem 3.3.

```
1 begin transaction
2     aprovarCompraComCartaoDeCredito($idComprador);
3     insert INTO TRANSACOES(
4         $idTransacao, $idComprador, $valor, $qtdeItensComprados);
5     UPDATE T_USUARIOS SET valorTotalDeCompras = valorTotalDeCompras + $valor
6         WHERE idUsuario = $idUsuario;
7     notificarSistemaDeCobrança($idTransacao);
```

```

8      notificarSistemaDeEntrega($idTransacao);
9  end transaction

```

Listagem 3.3: Otimização do contador

Aqui foi removida a validação do contador de livros em estoque. Como o reconciliador atualizará o contador e caso sejam vendidos livros que não existam no estoque, ele executará ações compensatórias, como comprar mais livros para repor o estoque, não havendo, portanto, mais a necessidade de verificar o contador.

Neste caso específico, foi possível eliminar a necessidade de um dado provisório. Vale notar que, no caso do contador de livros em estoque, tratá-lo como um dado que tem seu estado sempre em fluxo reflete a realidade, pois a todo momento há livros sendo vendidos e livros sendo repostos no estoque.

Este é um exemplo onde um estudo dos requisitos do sistema, com o objetivo de identificar dados que podem ter sua consistência relaxada, evitaria a implementação e problemas gerados pela listagem 3.1. Com esta última alteração, o problema do gargalo de acesso ao contador de livros em estoque foi eliminado.

Para evitar que a janela de inconsistência do contador fique muito grande, pode-se notificar o reconciliador de dados de que uma compra foi realizada, conforme mostra a listagem 3.4.

```

1  begin transaction
2      aprovarCompraComCartaoDeCredito($idComprador);
3      insert INTO T_COMPRAS(
4          $idTransacao, $idComprador, $valor, $qtdeItensComprados);
5      enviarMensagemParaReconciliador($idTransacao);
6      UPDATE T_USUARIOS
7          SET valorTotalDeCompras = valorTotalDeCompras + $valor
8          WHERE idUsuario = $idUsuario;
9      notificarSistemaDeCobrança($idTransacao);
10     notificarSistemaDeEntrega($idTransacao);
11 end transaction

```

Listagem 3.4: Diminuindo a janela de inconsistência

Na linha 5, uma mensagem é enviada para o reconciliador a fim de notificá-lo de que uma compra foi realizada e envia-se na mensagem o identificador da compra. A fila de mensagens utilizada é local e está no mesmo computador que processa a transação de compra. Aqui é usada uma fila de mensagens com o objetivo de desacoplar o reconciliador do restante do sistema.

Assim que receber a mensagem, o reconciliador de dados atualizará o contador de livros em estoque. Note que para o reconciliador será preciso o uso de uma transação distribuída para receber a mensagem e atualizar o contador, mas, como isso é feito pelo reconciliador, um processo que executa em segundo plano e de maneira assíncrona do restante da transação, não haverá muito impacto no desempenho e escalabilidade.

Agora é preciso tratar do problema do uso de transações distribuídas. Pela listagem 3.4 é

possível identificar os seguintes participantes na transação distribuída:

1. Sistema de aprovação de crédito;
2. *Shard* que armazena as transações;
3. Fila de mensagens;
4. *Shard* que armazena os dados dos usuários;
5. Sistema de cobrança;
6. Sistema de entrega.

A transação precisa se comunicar com 5 participantes e ter a concordância de todos para ser completada. Neste caso, existem três problemas: desempenho ruim devido à quantidade de participantes que devem ser contatados e sincronizados; disponibilidade baixa, pois se qualquer um dos 5 participantes estiver indisponível a transação falhará e uma venda deixará de ser realizada; alto acoplamento entre os participantes.

O primeiro participante a ser retirado da transação é o *shard* que armazena os dados dos usuários. Na listagem 3.4, na linha 5, é atualizado o valor total das compras realizadas pelo usuário. Este é um dado utilizado por motivos de desempenho, como se fosse um cache de alguns valores da tabela T_COMPRAS, e, como qualquer outro dado, ele deve ser consistente. A solução, neste caso, é mover a responsabilidade de manter este dado consistente para o reconciliador, já que ele é o responsável por manter os dados consistentes.

Por algum tempo, durante a janela de inconsistência, o valor total das compras do usuário ficará desatualizado, mas eventualmente o valor será atualizado e propagado para o restante do sistema. Para diminuir ainda mais a quantidade de participantes na transação distribuída, aplica-se a mesma solução novamente: as responsabilidades de aprovação de crédito, notificação do sistema de cobrança e notificação do sistema de entrega passam a ser do reconciliador. Estas mudanças são ilustradas na listagem 3.5.

```
1 begin transaction
2     INSERT INTO T_COMPRAS(
3         $idTransacao, $idComprador, $valor, $qtdeItensComprados);
4     enviarMensagemParaReconciliador($idTransacao);
5 end transaction
```

Listagem 3.5: Retirando participantes da transação distribuída

Os participantes da transação distribuída agora são o *shard* que armazena as transações e a fila de mensagens. Entretanto, é possível transformar a transação distribuída em uma transação local colocando a fila de mensagens no mesmo computador do *shard* que armazena as transações de compra e assim evitar o uso de 2PC.

Para finalizar o exemplo, falta entender como funciona o reconciliador. Ao longo do exemplo, o reconciliador de dados acumulou responsabilidades, e agora deve reconciliar os dados,

finalizar a transação de compra e iniciar ações compensatórias quando necessário. As responsabilidades do reconciliador são implementadas na listagem 3.6.

```

1  begin transaction
2      while (existem mensagens na fila)
3          $msg = receberMensagem();
4          $res = aprovarCréditoParaCompra($msg.idComprador);
5          if ( $res == true ) {
6              $livrosEmEstoque = SELECT qtdeLivros FROM T_ESTOQUE FOR UPDATE;
7              UPDATE T_ESTOQUE
8                  SET qtdeLivros = qtdeLivros - $qtdeItensComprados;
9              if ( ($livrosEmEstoque - $msg.qtdeItensComprados) < 0 ) {
10                 // notificar sistema de estoque que
11                 // mais livros devem ser adquiridos
12                 ...
13             }
14             UPDATE T_USUARIOS
15                 SET valorTotalDeCompras = valorTotalDeCompras + $msg.valor
16                 WHERE idUsuario = $msg.idUsuario;
17             notificarSistemaDeCobrança($msg.idTransacao);
18             notificarSistemaDeEntrega($msg.idTransacao);
19         } else {
20             // cancelar compra
21             DELETE FROM T_COMPRAS WHERE idTransacao = $msg.idTransacao;
22             // notificar usuário que crédito não foi aprovado
23             ...
24         }
25     end while
26 end transaction

```

Listagem 3.6: Reconciliador

Na linha 1, o reconciliador inicia uma transação e, nas linhas 2 e 3, as mensagens de notificação de compras são recebidas. Em seguida, linha 4, é feita a aprovação de crédito do cliente para que este possa realizar a compra. Este é o primeiro ponto onde pode ser necessária uma ação compensatória, caso o crédito não seja aprovado. Este fato é compensado na linha 21 onde a compra é cancelada e o usuário é notificado que a compra foi cancelada por problemas de crédito.

Se se estivesse utilizando uma consistência forte dos dados, a compra nem mesmo deveria ter sido completada para o usuário. Com o crédito aprovado, linhas 5 a 13, trava-se o contador da quantidade de livros em estoque na tabela T_ESTOQUE, o contador de livros em estoque é atualizado e verifica-se se há livros suficientes em estoque para cumprir a compra.

Caso não haja livros suficientes em estoque, uma ação compensatória é iniciada para notificar o sistema de controle de estoque, indicando que é preciso comprar mais livros. Na linha 14, o acumulador de valor de compras do usuário é atualizado. Depois se notifica os sistemas

de cobrança e de entrega, nas linhas 15 e 16.

Nesta implementação do reconciliador é utilizada uma transação distribuída. Como o reconciliador é um componente interno, que desempenha suas responsabilidades assincronamente em segundo plano, utilizou-se transações distribuídas para conveniência de implementação em troca de desempenho e escalabilidade menores. Isto é aceitável por ser um componente interno, desacoplado do fluxo principal da compra de livros.

Além disso, problemas de disponibilidade devido à transação distribuída não são visíveis aos usuários. De qualquer maneira, é possível implementar o reconciliar para minimizar, até mesmo evitar, o uso de transações distribuídas, bastando continuar a se aplicar a técnica de ter uma ação compensatória para cada falha que possa ocorrer e tratar outras situações específicas. O problema deste caminho é que o sistema se tornará extremamente complexo e difícil de ser implementado (o uso de Sagas, 3.5, pode auxiliar a implementação).

Uma conclusão que se pode tirar da discussão de BASE é que transações ACID e transações distribuídas não serão substituídas ou deixarão de ser utilizadas. ACID e transações distribuídas têm duas grandes vantagens difíceis de serem ignoradas. Primeiro, garantem consistência forte dos dados, e há muitas situações onde não se pode fugir dessa necessidade. Segundo, são extremamente convenientes e fáceis de serem usadas por programadores. No exemplo, caso o reconciliador fosse implementado sem o uso de transações distribuídas, ele se tornaria excessivamente complexo e difícil de ser implementado.

Outra conclusão é que quanto mais dinâmico for o sistema, mais propenso ele é a aceitar BASE. Em um sistema dinâmico, os dados estão naturalmente com seu estado em fluxo. Uma consistência forte e imediata dos dados é mera ilusão nesta situação. Do ponto de vista dos usuários, quando o sistema diz que há livros em estoque para serem entregues amanhã, esta é uma condição válida apenas por um tempo, enquanto o usuário vê esta informação a situação já pode ter mudado e não haver mais livros em estoque.

Para implementação do padrão BASE, as seguintes diretrizes são sugeridas:

- *Identifique os pontos onde a consistência pode ser relaxada:* Faça um estudo tomando como base as regras de negócio do sistema, as quais mostrarão os dados que não precisam ter uma consistência forte e onde pode-se utilizar uma consistência eventual e dados ligeiramente desatualizados. Procure por dados que naturalmente terão seu estado em constante fluxo;
- *Procure oportunidades de relaxar a consistência principalmente entre domínios funcionais:* Oportunidades de relaxar a consistência são mais fáceis de serem encontrados entre domínios funcionais do que dentro de um mesmo domínio funcional;
- *Uma vez encontrados os dados que terão uma consistência mais relaxada, defina a estratégia de reconciliação e a janela de inconsistência:* Definir em detalhes a estratégia de reconciliação é extremamente importante, pois é ela que garantirá a consistência eventual dos dados. O tamanho da janela de inconsistência também deve ser definido - uma

janela muito grande pode não ser aceitável e uma janela muito curta pode ser difícil de implementar. O tamanho da janela de inconsistência define a periodicidade da reconciliação, pois quanto menor a frequência de reconciliação, maior a diferença entre dados provisórios e reais;

- *Não faça agora o que pode ser feito depois:* Identifique entre os requisitos, tarefas que podem ser feitas mais tarde e delegue estas tarefas para o reconciliador ou outro subsistema se oportuno;
- *A inconsistência temporal não tem como ser escondida dos usuários:* Portanto, o uso de dados inconsistentes deve ser feito de comum acordo. A consistência está nos olhos de quem vê;
- *Use eventos para notificar os interessados quando os dados estiverem consistentes:* Com janelas de inconsistência grandes, é importante que todos os interessados saibam que os dados foram consistidos. Dê expectativas de conclusão da consistência e notifique os usuários mais tarde;
- *Desacople o reconciliador do restante do sistema:* Quanto mais desacoplado do restante do sistema mais fácil será escalar o reconciliador e o restante do sistema separadamente. O reconciliador deve ser desacoplado na maneira como se comunica com o resto do sistemas e desacoplado nos tempo e espaço.
- *Use o padrão BASE com parcimônia:* BASE melhora o desempenho e escalabilidade, mas pode tornar o sistema excessivamente complexo e difícil de implementar, escolha bem os pontos onde usá-lo;

Apesar do exemplo e implementação de BASE terem sido feitos com banco de dados, deve-se perceber que BASE é um conceito que pode ser aplicado a qualquer tipo de dado ou armazenamento e não é restrito ao uso com banco de dados.

Variantes

Uma possível variante de BASE são Sagas (ver 3.5). Sagas tem um foco menor, que lida apenas com como evitar transações distribuídas, mas pode ser usada como alternativa a BASE em alguns casos.

Qualquer sistema que possua replicação de dados pode ser uma variante de BASE. Um sistema que possui um banco de dados mestre e replica os dados para um escravo está tentando solucionar quase os mesmos problemas que BASE: disponibilidade, escalabilidade e desempenho. Também é eventualmente consistente e usa estado *soft*, pois sempre haverá uma janela de inconsistência até que os dados sejam replicados.

Usos conhecidos

Talvez o uso mais conhecido do padrão BASE seja o sistema DNS (*Domain Name System*). Os dados são particionados entre vários servidores. Trata-se de um sistema que está sempre disponível e os dados são eventualmente consistentes. Quando há uma alteração em um nome de domínio há um tempo para que a nova informação se propague pelo sistema.

Empresas que fazem uso de BASE: eBay [eBay] [Floyd Marinescu], Amazon [Amazon], [Vogels 2007].

Consequências

As seguintes **vantagens** são obtidas:

Minimiza e evita que acesso intenso a dados afetem o sistema: Os *hot spots* (pontos quentes) de dados são pelo menos minimizados e podem até ser evitados;

Minimiza e evita o uso de transações distribuídas: Com a reconciliação dos dados é possível evitar o uso de transações distribuídas pois agora há quem trate das possíveis falhas e inconsistência;

Aumenta o desempenho e a escalabilidade do sistema: Sem *hot spots* e sem transações distribuídas é mais fácil escalar o sistema sem causar efeitos colaterais que acabam por prejudicar a própria escalabilidade e desempenho;

As vantagens do padrão *sharding* se aplicam (ver 3.3).

Como consequência tem-se as seguintes **desvantagens**:

As complexidade e dificuldade de implementação do sistema aumentam: O fato de ter de construir um reconciliador de dados, ter de lidar com dados ligeiramente inconsistentes e a falta de transações distribuídas torna o sistema mais complexo e difícil de implementar;

Não é aplicável a qualquer situação: Algumas situações exigem uma consistência forte dos dados e não se pode aplicar BASE;

Não é um modelo transparente para os desenvolvedores: Os desenvolvedores devem lidar com situações onde os dados podem não ser os mais atuais;

Não é transparente para os usuários: Não há como esconder a inconsistência temporal dos dados dos usuários;

As desvantagens do padrão *sharding* se aplicam (ver 3.3).

Veja também

Sharding (ver 3.3), para implementação do particionamento dos dados. Sagas (ver 3.5), para evitar uso de transações distribuídas e ações compensatórias.

3.5 Padrão: Sagas

Resumo

O padrão Sagas proporciona melhoria da escalabilidade e desempenho, evitando o uso de transações distribuídas e mantendo a consistência dos dados. Transações ACID longas são divididas em transações menores e ações compensatórias são definidas para preservar a consistência dos dados.

Exemplo

Suponha um site de comércio eletrônico que vende livros. Devido ao grande volume de usuários e vendas o sistema é distribuído e escalado na horizontal. Entre as várias regras de negócio e de consistência de dados, as seguintes regras se aplicam para a realização de uma aquisição de livros: (1) “Deve haver livros suficientes em estoque antes de realizar uma remessa para cumprir uma ordem de compra”; (2) “Se o cliente pagar com cartão de crédito, deve-se garantir que o cliente tenha crédito para a compra através da aprovação da compra pela operadora de cartão de crédito”; (3) “Quando uma ordem de compra for completada deve-se notificar os sistemas de cobrança e entrega”.

Para garantir a consistência dos dados são usadas transações ACID (Atomicidade, Consistência, Isolamento, Durabilidade). À medida que a carga de trabalho do sistema aumenta, as transações de ordem de compra ficam cada vez mais lentas. Escalar o sistema horizontalmente não produz efeitos benéficos e para algumas operações há uma piora do desempenho.

Observação: De propósito é utilizado o mesmo exemplo utilizado em BASE, 3.4. A intenção é mostrar como uma parte do problema pode ser resolvido de maneira diferente.

Contexto

- Sistemas onde a escalabilidade e o desempenho são prejudicados pelo uso de transações distribuídas.

Problema

É comum que processos de negócio sejam longos e complexos, e devido a esta complexidade, um processo de negócio acaba por ter que acessar muitos dados e outros sistemas. Processos de negócios como este são implementados com transações ACID.

Esta abordagem tem os problemas de: (1) é uma transação de vida longa que demora algum tempo para terminar devido a natureza do próprio problema, durante sua execução será bloqueado o acesso a vários dados; (2) com o acesso a dados bloqueados por um longo tempo aumenta a quantidade de possíveis *deadlocks*; (3) é uma transação distribuída, será preciso acessar vários bancos de dados e sistemas.

Todos os problemas têm impacto na escalabilidade do sistema. Usar travas de acesso a dados no banco de dados e usar um protocolo de coordenação e sincronização como 2PC (*two phase commit*) em um cenário de alta carga e distribuído degradará bastante o desempenho e a escalabilidade. No exemplo, durante todo o processo de fechamento de uma ordem de compra, os dados acessados possuem travas, utilizadas pelo banco de dados para garantir as propriedades ACID da transação.

As seguintes Forças devem ser consideradas:

- As restrições de integridade, ditadas pelos requisitos do sistema, devem ser obedecidas;
- Deve-se minimizar, ou evitar, o uso de transações distribuídas;
- Deve-se minimizar o tempo que os dados ficam bloqueados para acesso;
- A implementação da solução não deve deixar o sistema excessivamente complexo; e
- A implementação da solução não deve ser complexa do ponto de vista técnico.

Solução

Para solucionar este problema transforma-se a transação de vida longa (LLT, *Long Lived Transaction*) em uma Saga [Garcia-Molina e Salem 1987]. A LLT é transformada em uma Saga quebrando-a em sub-transações menores e independentes que juntas constituem uma LLT lógica. Cada uma das sub-transações menores é uma transação ACID.

Para manter a consistência dos dados, para cada sub-transação é definida uma ação compensatória correspondente, executada na ocorrência de falha, que tem o objetivo de desfazer as ações da sub-transação e trazer os dados novamente a um estado consistente.

Estrutura

Sagas são compostas de sub-transações e ações compensatórias. As sub-transações são transações ACID que executam parte das ações da transação a partir da qual a Saga foi derivada. Juntas, as sub-transações têm o mesmo significado semântico que a transação que originou a Saga. Para cada sub-transação há uma ação compensatória correspondente que desfaz, do ponto de vista semântico, as ações realizadas pela sub-transação.

O objetivo de uma ação compensatória é compensar os casos de falhas e trazer o estado dos dados de volta a um estado consistente. Elas não necessariamente trazem os dados para o mesmo estado em que estavam antes da execução da sub-transação correspondente.

Como cada sub-transação é uma transação ACID separada e as ações compensatórias provavelmente também o são, e outras transações ACID podem estar em execução (como parte de Sagas ou não), todas estas transações serão executadas de maneira intercalada.

A Figura 3.22 ilustra a relação entre Sagas, sub-transações e ações compensatórias.

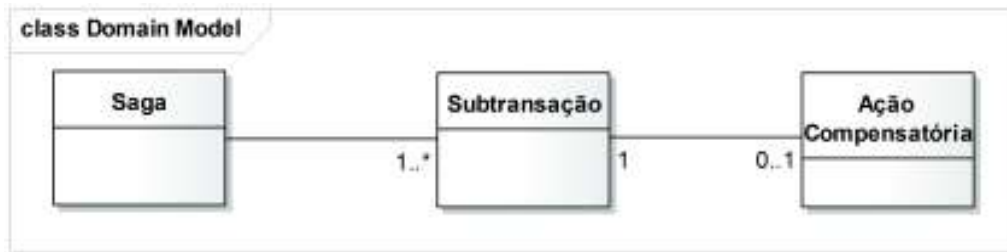


Figura 3.22: Modelo de domínio de Sagas

Utilizando o exemplo da ordem de compra, a LLT pode ser dividida nas sub-transações: armazenar a ordem de compra (T_1); atualizar o estoque (T_2); notificar o sistema de cobrança (T_3) e assim por diante.

Como cada uma das sub-transações é uma transação ACID, se alguma falhar, os dados ficarão inconsistentes. Portanto, para cada sub-transação da Saga uma ação compensatória correspondente é definida (provavelmente outra transação ACID) para desfazer as ações realizadas, do ponto de vista lógico, e trazer os dados de volta a um estado consistente. Seria definida a ação compensatória C_1 para apagar a ordem de compra criada por T_1 , C_2 para atualizar o estoque e indicar que os itens estão novamente disponíveis para compensar T_2 e assim por diante para as outras transações.

Um exemplo de que uma ação compensatória não pode simplesmente voltar os dados para o estado anterior é a execução de C_2 . Não se pode atualizar a quantidade de livros em estoque com a quantidade que havia quando T_2 foi executada, pois outras transações podem ter sido executadas e a quantidade de livros em estoque alterada.

A ação compensatória de uma sub-transação é opcional, podendo haver situações onde uma ação compensatória pode não ser necessária. As sub-transações, geralmente, não são totalmente independentes umas das outras, existe alguma relação entre elas. Não é necessário que todas as sub-transações vejam o mesmo estado consistente dos dados para que seja possível executar a Saga.

Dinâmica

Para executar a Saga, as sub-transações são executadas em sequência: $T_1, T_2, T_3, \dots$. Caso alguma das sub-transações falhe, as ações compensatórias são executadas em ordem inversa

àquela das transações. Se as ações compensatórias são C_1, C_2, C_3 e executando a Saga T_1, T_2, T_3 a transação T_3 falhou, as ações compensatórias executadas são C_2 e C_1 , nesta ordem.

As garantias oferecidas pelas Sagas são as seguintes. Dadas as sub-transações $T_1, T_2, \dots, T_n$ e as correspondentes ações compensatórias $C_1, C_2, \dots, C_{n-1}$, podem ocorrer duas possíveis seqüências de execução. Será executada a seqüência

$$T_1, T_2, \dots, T_n$$

ou será executada a seqüência

$$T_1, T_2, \dots, T_j, C_j, C_{j-1}, \dots, C_1$$

para algum $0 < j < n$.

Também é possível executar as transações de uma Saga em paralelo, o que é útil quando se trabalha com dados distribuídos como em *sharding* (3.3). Devido à sua dinâmica, Sagas podem ser vistas como uma maneira de implementar *workflows* multi-transações.

Implementação

Para a implementação de Sagas, a maior dificuldade é construir o mecanismo que executa as sub-transações e as respectivas ações compensatórias, particularmente no que se refere ao tratamento de erros. Com o objetivo de facilitar o uso de Sagas foi feita uma definição em Java de código livre (*open source*) de uma API para Sagas.

A Figura 3.23 apresenta um diagrama UML que descreve a API disponível para se programar com Sagas.

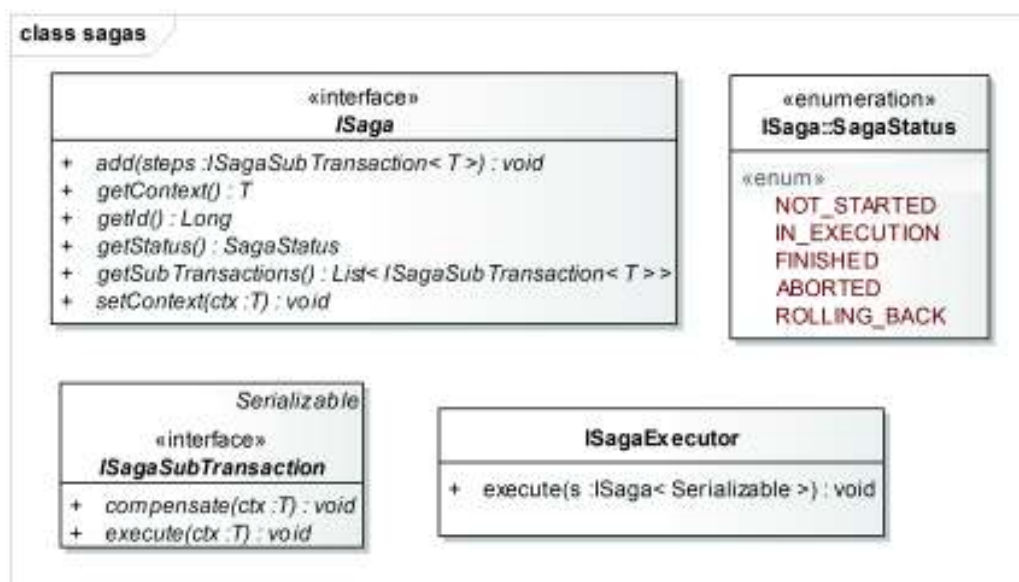


Figura 3.23: API para Sagas

A interface `ISaga` representa uma Saga, composta de sub-transações, que, por sua vez, são representadas pela interface `ISagaSubTransaction`. O método `add` da `ISaga` adiciona uma sub-transação à Saga, o método `getId` retorna o identificador da Saga, o método `getStatus` retorna o status atual da Saga, e os métodos `getContext` e `setContext` são usados para retornar e atribuir, respectivamente, o contexto da Saga.

Uma Saga pode estar em diversos estados, que são representados pela enumeração `SagaStatus`. O estado `NOT_STARTED` indica que a Saga ainda não foi executada, o estado `IN_EXECUTION` indica que a Saga está em execução, `FINISHED` denota uma Saga que já foi executada com sucesso, `ABORTED` caracteriza uma Saga que já foi executada mas que ocorreu um erro em alguma sub-transação e todas as ações compensatórias foram executadas, finalmente `ROLLING_BACK` indica que uma Saga foi executada, houve um erro em alguma sub-transação e as ações compensatórias estão sendo executadas.

Toda Saga possui um contexto que é compartilhado por todas as sub-transações. O contexto e seu tipo são definidos pelo programador, e geralmente o contexto armazena dados que são comuns ou utilizados por todas as sub-transações. O contexto pode ser visto como uma área “global” de armazenamento utilizada pelas sub-transações para compartilhar dados.

A interface `ISagaSubtransaction` representa uma sub-transação. O método `execute` executa a sub-transação e recebe como parâmetro o contexto da Saga. Durante a execução da sub-transação, caso `execute` lance uma exceção, a Saga é abortada e inicia-se a execução das ações compensatórias. O método `compensate` executa a ação compensatória correspondente da sub-transação. Durante a execução de uma Saga, uma transação é iniciada antes da execução do método `execute` e finalizada depois que o método é finalizado. Caso não tenha sido lançada uma exceção para o método, então a transação é consolidada. Caso contrário a transação é abortada. O mesmo ocorre para o método `compensate`.

A interface `ISagaExecutor` representa o motor de execução das Sagas e seu método `execute` é utilizado para executar a Saga. Ao invés da própria Saga saber como executar suas sub-transações, foi definida esta interface para deixar o mais separado possível, a definição das Sagas de como elas são executadas, para que seja possível trocar a implementação do executor de Sagas com grande facilidade.

Para ilustrar o uso da API será feita a implementação de alguns passos do exemplo, ou seja, da compra de livros. Como nas outras listagens apresentadas anteriormente, foram retirados os tratamentos de erros para tornar os exemplos mais concisos e fáceis de entender. A implementação a ser feita é a mesma seqüência de passos feita na listagem 3.1, sendo que a listagem 3.5 apresenta o início da implementação.

```

1 ISaga< HashMap< Object, Object > > s =
2     new Saga< HashMap< Object, Object > >();
3
4 ISagaSubTransaction< HashMap< Object, Object > > step =
5     new ISagaSubTransaction< HashMap< Object, Object > >() {
6

```

```
7      public void execute( HashMap< Object, Object > ctx )
8          throws Exception {
9
10         // verificar qtde de livros em estoque
11         Integer qtdeLivrosEstoque = getQtdeLivrosEstoque();
12         Integer qtdeLivrosComprados =
13             (Integer) ctx.get( "qtdeLivrosComprados" );
14
15         if ( (qtdeLivrosEstoque - qtdeLivrosComprados) < 0 ) {
16             throw new Exception(
17                 "Quantidade de livros em estoque insuficientes" );
18         }
19     }
20
21     public void compensate( HashMap< Object, Object > ctx )
22         throws Exception {}
23 };
24 s.add( step );
25
26 step = new ISagaSubTransaction< HashMap< Object, Object > >() {
27
28     public void execute( HashMap< Object, Object > ctx )
29         throws Exception {
30
31         Integer idComprador = (Integer) ctx.get( "idComprador" );
32         CartaoCredito Cartao = (CartaoCredito) ctx.get( "nroCartao" );
33
34         if ( ! aprovarCompraComCartaoDeCredito( idComprador, cartao ) ) {
35             throw new Exception(
36                 "Compra por cartão de crédito não aprovada" );
37         }
38     }
39
40     public void compensate( HashMap< Object, Object > ctx )
41         throws Exception { }
42 };
43 s.add( step );
```

Na linha 1, uma Saga é criada, neste exemplo é utilizado como contexto para a Saga e um mapa para suas sub-transações, onde as chaves e seus valores são objetos do tipo `Object`. Nas linhas 4 e 5, é declarada e iniciada a definição da sub-transação que verifica se há livros suficientes em estoque para serem comprados. O método de execução da sub-transação é definido na linha 7 e é aceito como parâmetro o contexto da Saga, no caso um mapa de objetos.

Na linha 11, busca-se a quantidade de livros em estoque e, em seguida se obtém do contexto da Saga, a quantidade de livros que estão sendo comprados. Depois é verificado, na linha 15, se há livros em estoque suficientes para serem comprados. Caso não haja livros suficientes,

é lançada uma exceção, na linha 16, para indicar que a Saga deve ser abortada. Na linha 21, é definido o método de compensação, sendo que neste caso não há nada a ser feito. A sub-transação é adicionada à Saga na linha 24.

Em seguida, nas linhas 26 a 43, é definida a sub-transação para aprovação da compra por cartão de crédito. Sua implementação é análoga à da sub-transação para verificar se há livros em estoque e portanto não será discutida aqui. No caso específico destas duas sub-transações, como ambas não possuem ações compensatórias, elas poderiam ter sido implementadas em apenas uma sub-transação.

A listagem 3.5 mostra a implementação da sub-transação para armazenar a compra na no banco de dados e a sub-transação para atualizar a quantidade de livros em estoque.

```

1  step = new ISagaSubTransaction< HashMap< Object, Object > >() {
2
3      public void execute( HashMap< Object, Object > ctx )
4          throws Exception {
5
6          Integer idComprador = (Integer) ctx.get( "idComprador" );
7          BigDecimal valor = (BigDecimal) ctx.get( "valor" );
8          Integer qtdeItens = (Integer) ctx.get( "qtdeItens" );
9          Integer idTransacao = GeradorId.gerarIdTransacao();
10
11          String sql =
12              "INSERT INTO T_COMPRAS(" +
13                  idTransacao + ", " +
14                  idComprador + ", " +
15                  valor + ", " +
16                  qtdeItens + ")";
17
18          Connection con = DB.getConexao();
19          Statement stmt = con.createStatement();
20          stmt.executeUpdate( sql );
21
22          ctx.put( "idTransacao", idTransacao );
23      }
24
25      public void compensate( HashMap< Object, Object > ctx )
26          throws Exception {
27
28          Connection con = DB.getConexao();
29          Statement stmt = con.createStatement();
30          String sql =
31              "DELETE FROM T_COMPRAS " +
32              "WHERE idTransacao = " +
33              ctx.get( "idTransacao" );
34          stmt.executeUpdate( sql );
35      }

```

```

36 };
37 s.add( step );
38
39 step = new ISagaSubTransaction< HashMap< Object, Object > >() {
40
41     public void execute( HashMap< Object, Object > ctx )
42         throws Exception {
43
44         String sql =
45             "UPDATE T_ESTOQUE SET qtdeLivros = qtdeLivros - " +
46             (Integer) ctx.get( "qtdeItens" );
47
48         Connection con = DB.getConexao();
49         Statement stmt = con.createStatement();
50         stmt.executeUpdate( sql );
51     }
52
53     public void compensate( HashMap< Object, Object > ctx )
54         throws Exception {
55
56         Connection con = DB.getConexao();
57         Statement stmt = con.createStatement();
58         String sql =
59             "UPDATE T_ESTOQUE SET qtdeLivros = qtdeLivros - " +
60             ctx.get( "qtdeItens" );
61         stmt.executeUpdate( sql );
62     }
63 };
64 s.add( step );

```

Na linha 1, é criada e definida a sub-transação para armazenar a compra no banco de dados. Na linha 3, é definido o método `execute`, que inicialmente (linhas 6 a 8) busca no contexto alguns dados da compra, e cria um novo identificador para a transação, na linha 9. O comando SQL para armazenar os dados é criado nas linhas 11 a 16. Nas linhas 18 a 20 é invocada a execução do comando SQL via conexão ao banco de dados. Na linha 22 o identificador da transação é armazenado no contexto.

A ação compensatória da sub-transação é apagar do banco de dados a transação de compra criada. A ação é implementada no método `compensate` na linha 25. Primeiro se conecta ao banco de dados, define o comando SQL para apagar a compra, através do identificador da transação colocado no contexto da Saga pelo método `execute`, e executa-se o comando SQL. Estas ações são feitas nas linhas 28 a 34 e, na linha 37, a sub-transação é adicionada à Saga. A sub-transação para atualizar a quantidade de livros em estoque é definida entre as linhas 39 a 64, e é análoga a outra sub-transação e portanto não será discutida aqui.

A listagem 3.5 mostra como a Saga é executada.

```
1 // criar e popular o contexto com os dados da compra
2 HashMap< Object, Object > ctx = new HashMap< Object, Object >();
3 ctx.put( "idComprador", idComprador );
4 ...
5 s.setContext( ctx );
6
7 // executar a Saga
8 ISagaExecutor executor = Sagas.getDefaultExecutor();
9 executor.execute( s );
10 System.out.println( "Status: " + s.getStatus() + ", Id: " + s.getId() );
```

Na linha 2 cria-se o contexto da Saga, e em seguida, nas linhas 3 a 5, o contexto é populado com os dados da compra para que sejam utilizados pelas sub-transações. Na linha 8 é criado o executor de Sagas e, em seguida na linha 9, a Saga é executada. Para finalizar, na linha 10, são impressos o status e o identificador da Saga.

O uso da API de Sagas não é complexo, mas, como pode ser visto pelas listagens, é bem mais trabalhoso programar com Sagas do que com transações distribuídas. O código fonte da implementação completa de Sagas pode ser encontrado no apêndice A.

Para implementação de Sagas, sugere-se as seguintes diretrizes (retiradas de [Garcia-Molina e Salem 1987]):

- *Antes de tudo identifique quais processos são realmente LLTs:* Nem todo processo é uma LLT. É preciso identificar quais processos são uma LLT e dessas LLTs quais têm impacto significativo no desempenho do sistema para serem candidatas a Sagas;
- *Para identificar as sub-transações procure por pontos de divisão naturais:* LLTs representam processos do mundo real, assim procure nos processos do mundo real os pontos onde naturalmente ocorrem subdivisões do trabalho a ser executado. Geralmente os processos de negócios são naturalmente divididos em vários passos, estes passos são candidatos a serem sub-transações de uma Saga;
- *Para identificar as sub-transações procure por pontos de divisão entre domínios funcionais:* No caso do fechamento de uma ordem de compra é preciso atualizar o estoque, notificar o sistema de cobrança e notificar o sistema de entrega. Estes são domínios funcionais distintos: estoque; cobrança; entrega. Ações que lidam com domínios funcionais distintos possuem boa oportunidades de serem sub-transações.

Variantes

BASE (Basically Available, Soft State, Eventual Consistency), 3.4, pode ser considerada uma variante de Sagas, mas possui um escopo mais amplo. Sagas podem ser utilizadas conjuntamente com BASE.

Usos conhecidos

O eBay [eBay] é conhecido por não utilizar transações distribuídas em seus sistemas [Fowler 2007] [Pritchett 2007b], em seu lugar os desenvolvedores devem implementar ações compensatórias para os casos de erro, o que acaba por caracterizar o uso do conceito de Sagas.

Consequências

As seguintes **vantagens** são obtidas:

Evita o uso de transações distribuídas: Com o uso de sub-transações, ações compensatórias e um mecanismo que assegure a execução em caso de erros, evita-se o uso de transações distribuídas;

Aumento de desempenho e escalabilidade: Dividindo a LLT em transações menores evita-se que travas de acesso a dados sejam mantidas por um longo tempo, possibilitando que outras transações acessem os dados.

Como consequência tem-se as seguintes **desvantagens**:

Aplica-se apenas a situações específicas: Há situações onde o uso de Sagas não se aplica;

A implementação do sistema pode tornar-se complexa: Apesar de tecnicamente, com a ajuda da implementação fornecida, o uso de Sagas se tornar simples, devido à natureza do problema, o sistema pode tornar-se muito complexo;

Garantir a consistência e integridade dos dados passa a ser responsabilidade do sistema:
Com o uso de Sagas, a garantia de que os dados estão consistentes é toda do sistema.

Veja também

BASE, 3.4, para implementação de uma solução com escopo mais amplo.

3.6 Padrão: Camada de Caches Distribuídos

Resumo

O padrão Camada de Caches Distribuídos proporciona melhoria da escalabilidade horizontal e desempenho através do uso de vários caches com um particionamento dos dados entre os caches, evitando duplicação desnecessária de dados em memória.

Exemplo

Suponha um site de comércio eletrônico que vende produtos dos mais variados tipos (livros, computadores, televisores, telefones, etc.). A página inicial apresenta muitas informações e é dinâmica, apresentando aleatoriamente as promoções, em função dos produtos mais vendidos e sugestões de itens baseados nas preferências dos usuários. Para melhorar o desempenho do site foi construída uma Arquitetura *Shared Nothing*. Devido ao grande acesso de dados, em cada instância, é feito cache dos dados mais requisitados. O uso de memória de cada computador é alto devido ao cache de dados. Quando algum dado muito requisitado sofre alteração, como uma mudança de preço ou uma nova promoção, a invalidação dos dados nos caches é feita através da expiração do dado no cache. Quando o tempo de expiração é curto o desempenho piora devido ao constante acesso ao banco de dados para carregar novos dados em todos os caches. Quando o tempo de expiração é longo os dados apresentados no site ficam desatualizados por muito tempo.

Contexto

- Sistemas com grande acesso a dados, onde as requisições feitas pelos usuários têm como consequência muitos acessos ao banco de dados e o tempo de resposta da aplicação deve ser pequeno;
- Sistemas com problemas de desempenho e escalabilidade onde escalar na vertical ou na horizontal é difícil, inviável ou caro financeiramente;
- Sistemas que fazem uso de cache local e uso intenso de memória, prejudicando o sistema, sendo que os dados dos caches são praticamente os mesmos;
- Sistemas onde há vários caches e invalidar todos os caches é difícil.

Problema

Em aplicações com grande acesso a dados é comum que, para cada requisição de usuário, vários acessos de leitura sejam feitos no banco de dados. Dependendo da natureza do sistema, as requisições de diferentes usuários acabam por acessar os mesmos dados, duplicando esforço. O sistema tem seu desempenho e sua escalabilidade prejudicados devido ao grande volume de operações de I/O.

Com o uso tradicional de caches locais a cada instância, o problema é atenuado. Entretanto, em sistemas de grande volume de usuários e de dados, o uso de cache local ocupa muita memória, o que acaba prejudicando o sistema. Além disso, se cada instância tem seu cache: (1) haverá desperdício de memória, pois os mesmo dados estarão presentes em vários caches e são os dados mais acessados; (2) a invalidação de dados fica mais difícil em vários caches, com exceção da invalidação por tempo de expiração.

As seguintes Forças devem ser consideradas:

- A solução deve oferecer uma solução de armazenamento de dados que comporte qualquer tipo de dados;
- A solução deve ser capaz de ser escalada separadamente do restante do sistema;
- A consistência dos dados deve ser mantida pela solução;
- A durabilidade dos dados deve ser preservada.

Solução

A solução para o problema é utilizar um cache distribuído e particionado para armazenar dados frequentemente acessados. Ao invés de cada instância possuir seu cache local, utiliza-se um subsistema a parte com a exclusiva função de cache, este subsistema é, geralmente, fisicamente separado do restante do sistema.

Para evitar desperdício de memória, com o armazenamento de dados duplicados, os dados são particionados entre vários caches, assim como se faz com *sharding* de base de dados (3.3). Para maximizar o potencial do cache distribuído, faz-se cache de todos os dados possíveis, não apenas de dados frequentemente acessados no banco de dados e de instâncias de objetos utilizados com frequência pela aplicação.

Estrutura

A Figura 3.24 ilustra a estrutura da solução.

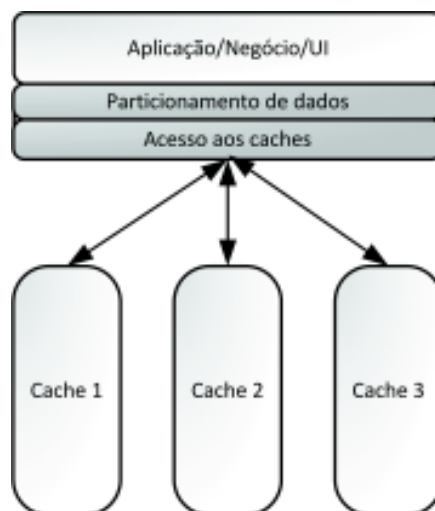


Figura 3.24: Estrutura de caches distribuídos e particionados

Os principais participantes da solução são: os caches, que armazenam os dados; uma camada de particionamento, que distribui os dados entre os caches; uma camada de acesso aos caches responsável pela comunicação com as várias instâncias de caches; e um protocolo de rede para comunicação com os caches (não mostrado na figura).

A camada de particionamento tem as mesmas responsabilidades da camada de particionamento da estrutura do padrão *Sharding* (3.3), mas possui menos responsabilidades e, portanto, é mais simples.

Utiliza-se o conceito de camadas para descrever e agrupar as responsabilidades, pois esta representação é abstrata o suficiente para permitir qualquer tipo de implementação (por exemplo, talvez seja mais fácil implementar apenas um componente de particionamento de dados nos caches e não toda uma camada, mas as responsabilidades são as mesmas).

Pode-se utilizar os caches de duas maneiras, em *Sideline* ou como uma camada de abstração do banco de dados. A Figura 3.25 ilustra o esquema *Sideline*.

No esquema *Sideline*, os caches são tratados como um subsistema a parte. O restante do sistema sabe que o cache existe, e quais são suas responsabilidades, e implementa a lógica de manipulação e acesso aos caches.

Outro esquema é utilizar a camada de cache como uma abstração do banco de dados como ilustrado na Figura 3.26.

Neste caso, os caches formam uma camada situada acima dos bancos de dados e abaixo do restante do sistema. Os caches abstraem, para o restante da aplicação, o acesso ao banco de dados e é responsabilidade dos caches ler e escrever nos bancos de dados. Para o restante da aplicação, a camada de cache é transparente, sendo que os acessos a dados seguem como se fossem feitos diretamente no banco de dados, mas na verdade são intermediados pela camada de cache.

Continuando, para o exemplo em tela, poder-se-ia utilizar um cache em *Sideline* para armazenar os dados apresentados na página inicial do site, que são as promoções, os produtos

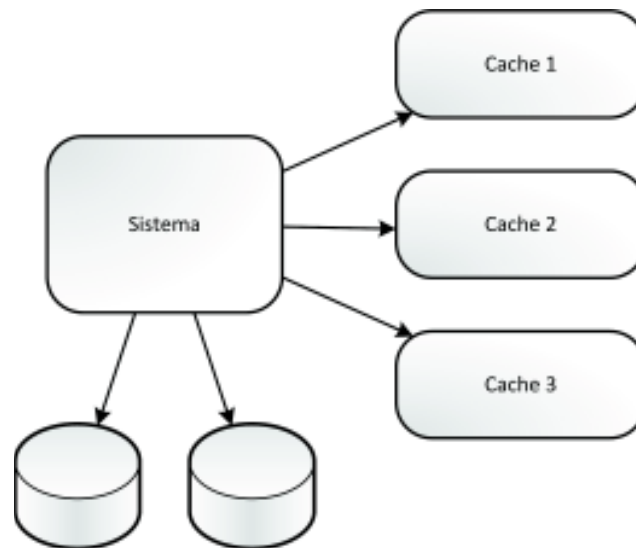
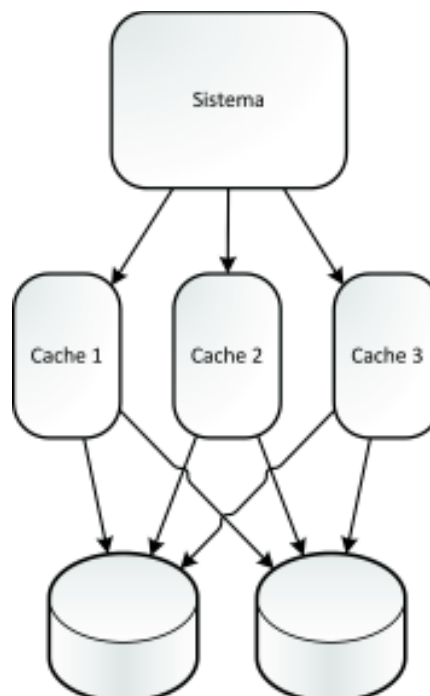
Figura 3.25: Caches em *sideline*

Figura 3.26: Caches como uma camada de abstração

mais vendidos e sugestões de itens baseado nas preferências do usuário.

O uso de um cache distribuído e particionado abre outras possibilidades de armazenamento de dados. Uma funcionalidade comum em sites para a Internet é a chamada sessão do usuário. A sessão do usuário é utilizada para armazenar dados referentes ao uso do sistema pelo usuário. Ela armazena dados como preferências, escolhas feitas no site e que têm influência no uso de outras funcionalidades, etc.

Uma maneira comum de implementar a sessão do usuário é armazená-la em um cache local da instância. Entretanto, isto introduz o problema de que apenas a instância que tem a sessão do usuário pode atender às suas requisições. Se outra instância atender a requisição, ela não terá a sessão do usuário.

Esta amarração do usuário a apenas um computador causa um desbalanceamento de carga entre os computadores já que o comportamento dos usuários são diferentes. Alternativas de implementação de sessão de usuário são: replicar a sessão dos usuários entre todas as instâncias; armazenar a sessão no sistema de arquivo; armazenar a sessão em banco de dados; ou, armazenar a sessão em um *cookie*.

Nenhuma destas alternativas apresenta bom desempenho e escalabilidade (algumas apresentam uma ou a outra). Uma boa solução é armazenar a sessão do usuário no cache distribuído. Como não há dados duplicados nos caches devido ao particionamento, qualquer instância poderá atender à requisição de qualquer usuário, bastando buscar a sessão no cache.

Dinâmica

A dinâmica de funcionamento de um cache em *Sideline* se dá como ilustrado na figura 3.27.

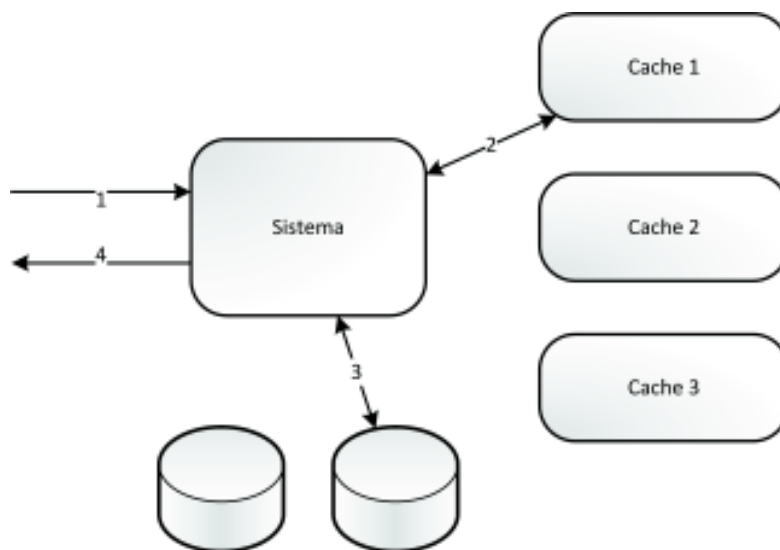


Figura 3.27: Dinâmica de uso de cache em *sideline*

Primeiro, quando é preciso acessar algum dado, é verificado o cache. Se o dado estiver no cache basta utilizá-lo. Se o dado não estiver no cache, é feito um acesso ao banco de dados para ler o dado e em seguida o dado é armazenado no cache. Note que neste esquema, o cache não possui inteligência alguma.

Ponto importante no uso de cache é a política de invalidação dos dados. A maneira mais simples é realizar uma expiração por tempo. Cada dado armazenado fica no cache por um período de tempo predeterminado. Uma política melhor é a aplicação assumir a responsabilidade de invalidar o cache. Quando um dado for atualizado no banco de dados a aplicação verifica se o dado está presente no cache e caso esteja o invalida, assim a versão mais atual do dado será armazenada no cache quando for requisitada.

Para o uso de cache como uma camada de abstração aos dados a figura 3.28 ilustra como uma leitura é feita.

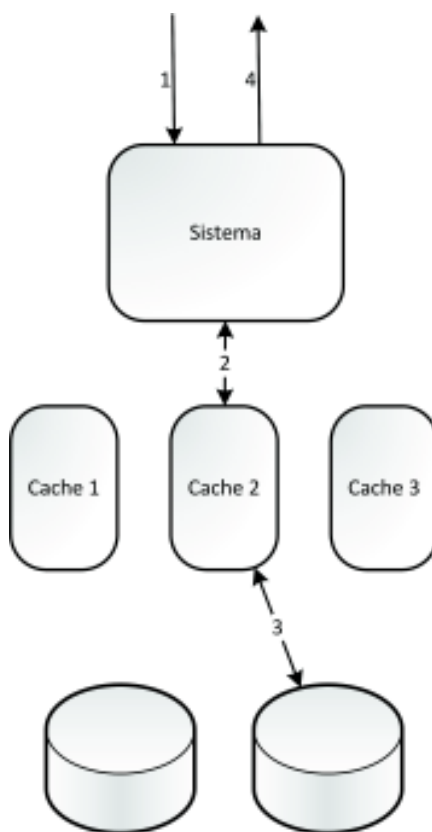
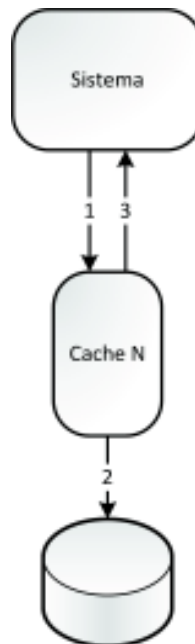


Figura 3.28: Dinâmica de caches como uma camada de abstração aos dados

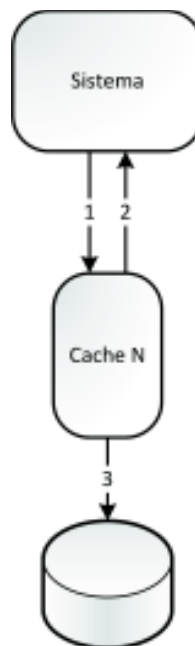
Quando uma leitura de dados é feita, ela é requisitada ao cache, que verifica se o dado está armazenado em sua memória. Se estiver, o dado é retornado; senão, o próprio cache acessa o banco de dados, lê o dado, armazena em memória e retorna.

Para o uso de cache como uma camada de abstração aos dados há duas maneiras de funcionamento para escritas, *write-through* e *write-back*. A Figura 3.29 ilustra o funcionamento *write-through*.

Figura 3.29: Escrita *write-through*

Quando é feita uma escrita ela é feita no cache (1), que realiza a escrita no banco de dados (2) e caso o dado esteja presente em memória atualiza o dado e então retorna uma resposta de sucesso (3).

Para o modo *write back*, a Figura 3.30 ilustra o funcionamento.

Figura 3.30: Escrita *write-back*

Quando é feita uma escrita ela é feita no cache (1), que retorna uma resposta imediatamente para o requisitante da Escrita (2). O dado atualizado é mantido em memória, ou em armazena-

mento local, e mais tarde (por exemplo, no caso de substituição) o próprio cache escreve o dado no banco de dados (3).

No exemplo do site, quando a página inicial é acessada, todos os dados a serem apresentados são buscados no cache e então mostrados. Dados que não estão nos caches são carregados do banco de dados e armazenados nos caches. O mesmo acontece quando é preciso utilizar os dados da sessão do usuário.

Implementação

Antes de ser discutida a implementação, são abordadas as vantagens e desvantagens dos dois tipos de uso dos caches. Além das diferenças estruturais entre os dois esquemas de cache discutidos, há um obstáculo, pois, uma camada de cache para abstrair o banco de dados, é difícil de implementar, enquanto que um esquema *Sideline* é fácil de implementar.

O uso de cache como uma camada de abstração acaba por transformar os caches quase que em um banco dados, assim o uso de cache em *Sideline* é recomendado para a maioria das situações. Devido a este fato é discutida apenas a implementação do cache em *Sideline*.

A implementação do cache em *Sideline*, no que diz respeito ao particionamento dos dados, segue a mesma linha da implementação de *Sharding* (3.3), e portanto não será repetida aqui. Ressalte-se, entretanto, que o modelo de armazenamento dos dados é diferente do *Sharding*, no qual os dados são armazenados em bancos de dados relacionais, enquanto que no cache os dados são armazenados em memória e a organização dos dados é mais simples (os dados são armazenados como pares (chave, valor)).

A chave identifica o dado e o valor é tratado como uma sequência opaca de bytes. A interpretação do significado dos bytes (dado) é de total responsabilidade do usuário. Devido a esta organização bem mais simples dos dados no cache o esquema de particionamento recomendado é o particionamento por chave ou *hash*.

Apesar da implementação de um cache distribuído e particionado não ser complexa, atualmente a implementação se tornou mais fácil devido a disponibilidade de software, inclusive de código aberto (*open source*). Com o uso de um software que implementa a camada de particionamento, a camada de acesso ao cache, o protocolo de comunicação e o armazenamento dos dados em memória, o sistema fica responsável apenas por implementar a política de uso dos caches.

Uma boa opção de software é o *memcached* [Danga], que usa uma tabela de *hash* distribuída [Balakrishnan et al. 2003] para implementar um cache distribuído e particionado. Outras opções de software que podem ser utilizadas são [JBoss b], [JBoss a], [Oracle], [GigaSpaces].

Existem várias (boas) opções de software, sendo que para ilustrar este caso, será utilizada uma implementação do *memcached* com a API *Spymemcached*. A listagem 3.7 ilustra como os itens da página inicial do site seriam recuperados utilizando-se um cache distribuído.

```
1 MemcachedClient c = new MemcachedClient(
2     new InetSocketAddress("127.0.0.1", portNum));
3
4 List<Produto> promos = getPromocoes(c);
5 List<Produto> maisVendidos = getProdutosMaisVendidos(c);
6 Session sessao = getSessaoUsuario(c, idUsuario);
7 List<Produto> sugestoes = getSugestoes(c, sessao);
```

Listagem 3.7: Exemplo de implementação utilizando memcached

Na linha 1 é criado um cliente para acessar o memcached, especificando o endereço IP e número da porta. Em seguida, nas linhas 4 e 5, são carregados os produtos que estão em promoção e os produtos mais vendidos. Depois, nas linhas 6 e 7, é recuperada a sessão do usuário e buscadas sugestões de produtos para ele.

Para buscar os produtos em promoção, a implementação seria a da listagem 3.8.

```
1 public List<Produto> getPromocoes(MemcachedClient c) {
2     List<Produto> produtos = c.get("promocoes");
3     if (produtos == null) {
4         produtos = lerPromocoesDoBancoDeDados();
5         c.put("promocoes", produtos, 60 * 10);
6     }
7     return produtos;
8 }
```

Listagem 3.8: Buscando por promoções

Na linha 2 acessa-se o cache para obter as promoções. Caso as promoções não estejam no cache, linha 3, as promoções são carregadas do banco de dados utilizando alguma regra de negócio para selecionar as promoções e em seguida as novas promoções são armazenadas no cache, linha 5, sob uma chave de identificação `promocoes` e com um tempo de expiração de 10 minutos.

Toda a responsabilidade de implementar o esquema de particionamento e o acesso ao cache através de um protocolo de comunicação fica a cargo da API fornecida pelo memcached, restando para a aplicação a responsabilidade de implementar a política de uso do cache. O método para buscar os produtos mais vendidos é análogo a este.

A implementação para lidar com a sessão do usuário é bem parecida com a implementação para buscar as promoções (listagem 3.9).

```
1 public Session getSessaoUsuario(MemcachedClient c, int idUsuario) {
2     Session sessao = c.get("sessao." + idUsuario);
3     if (sessao == null) {
4         sessao = criarNovaSessao(idUsuario);
5         c.put("sessao." + idUsuario, sessao, 60 * 60);
6     }
7     return sessao;
```

```
8  }
9
10 public List<Produto> getSugestoes(MemcachedClient c, Session s) {
11     List<Produto> sugestoes = c.get("sugestoes." + s.idUsuario);
12     if (sugestoes == null) {
13         sugestoes = calcularSugestoes(s);
14         c.set("sugestoes." + s.idUsuario, sugestoes, 60 * 5);
15     }
16     return sugestoes;
17 }
```

Listagem 3.9: Buscando a sessão do usuário

O método `getSessaoUsuario`, linha 1, e o método `getSugestoes`, linha 10, ambos são análogos ao método `getPromocoies`.

Nas listagens apresentadas até aqui, não foi incluído o tratamento de erro para deixar os exemplos mais claros e fáceis de serem compreendidos. Entretanto, há um tratamento de erro importante, que não deve ser ignorado durante a implementação, e que diz respeito às falhas dos caches. É preciso lembrar que os dados estão particionados entre os caches e que não há dados duplicados entre os caches (a não ser que a duplicação tenha sido feito propositalmente) e que os dados sejam particionados por um esquema de particionamento.

O esquema de particionamento mapeará um dado em particular para um, e apenas um, dos caches e o mapeamento será sempre o mesmo para aquele dado, senão não se conseguiria encontrar o dado nos caches pois não haveria como saber em qual cache ele está.

Quando um cache falha e é preciso acessar ou armazenar um dado mapeado para o cache que falhou, é preciso saber como agir. Algumas opções são:

- Ignorar a falha e carregar os dados diretamente de sua fonte (do banco de dados por exemplo) até que o cache volte a funcionar. Quando o cache voltar a funcionar ele volta a armazenar dados para o restante do sistema. Esta política funciona bem quando se espera que os caches fiquem inacessíveis por períodos curtos de tempo;
- Ignorar a falha, carregar os dados diretamente de sua fonte (do banco de dados por exemplo) e utilizar apenas os caches disponíveis para realizar o mapeamento dos dados para os caches. O problema aqui é que a quantidade de caches disponíveis mudou e portanto é preciso redistribuir os dados nos caches. Com a quantidade de caches disponíveis alterada, um esquema de mapeamento baseado em chave ou *hash* mapeará os dados para caches diferentes e portanto uma redistribuição de todos os dados nos caches seria preciso. Neste caso, o fato de utilizar apenas os caches disponíveis para realizar o mapeamento forçará a redistribuição dos dados nos caches, os dados serão carregados novamente do banco de dados e armazenados no cache e com o tempo os dados serão redistribuídos novamente. Os dados antigos, que estavam nos caches, e que agora são mapeados para outro cache, expirarão com o tempo (se houver objetos que não expiram no cache deve-se lidar com

este caso em particular). Para evitar a espera pela expiração dos dados, uma opção é reiniciar os caches. Quando o cache que falhou voltar a funcionar este processo ocorrerá novamente;

- Utilizar uma função de *hash* consistente [Karger et al. 1997] para fazer o mapeamento de dados para caches e ter o mesmo comportamento do item anterior. Com o *hashing* consistente não será preciso redistribuir todos os dados apenas uma parte deles.

Um opção para minimizar os efeitos quando ocorre falha em um dos caches é armazenar cada dado em mais de um cache. A camada de particionamento mapeia neste caso um dado para mais de um cache. Quando um dado é armazenado, ele é armazenado em todos os caches mapeados. Quando um dado é lido tenta-se ler o dado de todos os caches nos quais o dado foi mapeado.

Outro problema com caches distribuídos é o efeito “manada”, que ocorre quando há a expiração ou invalidação de algum dado no cache. Imagine um sistema composto de 20 instâncias e uma camada de caches distribuídos. Alguns dados armazenados nos caches são muito requisitados e todos as 20 instância sempre estão buscando estes dados nos caches. Quando algum destes dados expira, todos as instâncias que o acessam tentarão atualizar o dado no cache e, de repente, muitas consultas iguais serão feitas no banco de dados para buscar o dado e atualizá-lo no cache.

Este efeito ocorre por que o acesso aos caches não é transacional, isto é: buscar um dado no cache; verificar que o dado não se encontra lá; buscar o dado no banco de dados; e armazená-lo no cache, não é atômico e não existe controle de concorrência entre os acessos aos dados.

É importante notar que esse fenômeno ocorre apenas com caches não transacionais, como o memcached ou qualquer outra implementação de cache distribuído (outros software como JBoss Cache e GigaSpaces possuem suporte transacional).

O motivo pelo qual este efeito ocorre é a condição de corrida entre detectar que um dado não está no cache e o tempo necessário para buscar o dado no banco de dados e atualizá-lo no cache. Portanto, uma solução para minimizar este efeito é diminuir a janela de tempo entre detectar a não presença do dado no cache e atualizá-lo.

Uma possível implementação para diminuir este problema é apresentada na listagem 3.10 para o uso do memcached (a idéia pode ser utilizada com qualquer cache):

```
1 public class ObjetoCacheado {
2     Object objetoReal; // o objeto que na verdade é cacheado
3     long tempoExpiracaoReal; // a hora em que o objeto expirará no cache
4     boolean atualizando; // indica se o objeto foi atualizado
5 }
6
7 public Object get(MemcachedClient c, String key) {
8
9     ObjetoCacheado o = (ObjetoCacheado) c.get( key );
10    if (o == null) {
```

```
11         return null;
12     }
13
14     if (System.currentTimeMillis() > o.tempoExpiracaoReal
15         && !o.atualizando) {
16
17         set(key, o.objetoReal, 30, true);
18         return null;
19     }
20     return o.objetoReal;
21 }
22
23 public boolean set(
24     MemcachedClient c,
25     String key,
26     Object o,
27     int timeout,
28     boolean atualizando) {
29
30     ObjetoCacheado cached = new ObjetoCacheado();
31     cached.objetoReal = o;
32     cached.atualizando = atualizando;
33     long now = System.currentTimeMillis();
34     cached.tempoExpiracaoReal = now + timeout;
35
36     int thirtyDays = 60 * 60 * 24 * 30;
37     return c.set(key, now + thirtyDays, cached);
38 }
```

Listagem 3.10: Buscando a sessão do usuário

A idéia geral da implementação é armazenar o dado no cache, mas informando ao cache um tempo de expiração bem grande, muito maior que o tempo real de expiração do dado, e armazenar juntamente com o dado o seu tempo real de expiração. A expiração do dado agora é feita pela aplicação. Na linha 1, é definida a classe dos objetos, *ObjetoCacheado*, que são armazenados no cache. O membro *objetoReal* é uma referência ao objeto que o usuário do cache quer armazenar. O membro *tempoExpiracaoReal* é o tempo real de expiração do dado. O membro *atualizando* indica, quando verdadeiro, que alguém está atualizando o dado no cache.

Na linha 7 define-se o método *get*, utilizado para buscar dados no cache. Uma vez buscado o dado no cache, linha 9, verifica-se em seguida se o objeto estava presente no cache. Com o objeto presente no cache verifica-se, linha 14, se o dado expirou e se há outro alguém que já esteja atualizando o dado no cache.

Caso o dado tenha expirado e não haja mais ninguém a atualizá-lo, o dado é imediatamente armazenado no cache com um tempo de expiração de 30 segundos, informando que agora há

alguém que atualizará o dado no cache, linha 17, e retorna-se a quem chamou o método `get` que o dado requisitado não está no cache (o que forçará sua atualização) na linha 18.

Caso o dado não tenha expirado, ou tenha expirado, mas haja alguém atualizando-o, retorna-se a versão atual do dado, linha 20. Na linha 23 é definido o método `set`, utilizado para armazenar dados no cache. Na linha 30 um objeto do tipo `ObjetoCacheado` é criado para ser armazenado no cache, logo em seguida, associa-se a ele o objeto real e indica-se se o objeto está sendo atualizado no cache.

Depois atribui-se o tempo real de expiração na linha 34. O objeto é armazenado no cache, na linha 37, com um tempo de expiração de um mês. O tempo de um mês foi escolhido arbitrariamente. Esta solução não resolve completamente o problema, ainda há condição de corrida entre as linhas 10 e 18, entretanto a quantidade de instâncias (ou processos, ou *threads*) que tentarão atualizar o dado ao mesmo tempo diminuirá bastante.

Um ponto importante para uso de caches distribuídos é lembrar-se do seu objetivo de uso que é minizar I/O e melhorar o desempenho. Entretanto, deve-se planejar o uso do cache de maneira equilibrada para que o sistema não dependa dele para funcionar. Sistemas onde o cache deve estar funcional para que o restante do sistema funcione tem sua disponibilidade prejudicada, pois qualquer operação nos caches, como rebalanceamento de dados, reinicilização, etc., tornará o sistema indisponível. Deve-se planejar o uso do cache de tal maneira que se o cache ficar indisponível a aplicação continue a funcionar normalmente.

Variantes

Uma variante possível é utilizar uma hierarquia de caches. Além da camada de caches cada nó possui um cache local. Quando é preciso acessar algum dado, primeiro é verificado o cache local e depois a camada de caches. Este esquema funciona bem para dados que são apenas para leitura (ou atualizados com frequência muito baixa). Apesar do uso de memória não ser o mais eficiente possível, pois existirão dados duplicados nos caches locais, há um ganho de desempenho que pode justificar esse uso adicional de memória.

Uma alternativa é utilizar *sharding* ao invés de uma camada de cache, à medida que se tem cada vez mais *Shards*, eles se tornam cada vez menores possibilitando que todos os dados do *Shard* sejam mantidos em memória, o que acaba por tornar o *Shard* em um cache. Outra opção ainda é utilizar uma camada de cache em conjunto com *sharding*.

Usos conhecidos

Camada de caches distribuídos e particionados é feito por várias empresas, como YouTube [highscalability.com 2008], Facebook [Facebook] [Jack Dorsey 2008], e outros (<http://www.danga.com/memcached/users.bml>).

Consequências

As seguintes **vantagens** são obtidas:

Aumento do desempenho: Acesso a dados utilizados com frequência é mais rápido;

Uso mais eficiente de memória: A memória dos nós da camada de cache está totalmente disponível para armazenar dados em cache e não há dados duplicados sendo mantidos nos caches;

Facilidade de escalar a camada de caches: Por ser um subsistema a parte, com baixo acoplamento com o restante do sistema é fácil escalar a camada de caches;

Uso de cache evita que se precise escalar outras partes mais difíceis do sistema: Entre as alternativas disponíveis para escalar um sistema, o uso de caches pode ser a mais fácil e barata. Em um sistema que já está construído é mais fácil adicionar uma camada de caches do que implementar *sharding* ou BASE, por exemplo.

Como consequência tem-se as seguintes **desvantagens**:

Introdução de latência (*penalty*) caso a taxa de acerto seja baixa: Caso a taxa de acerto nos caches seja baixa para alguns dados, eventualmente, haverá aumento no tempo de resposta das requisições;

Torna o sistema mais complexo e difícil de implementar: Com o uso de cache deve-se implementar a administração dos dados armazenados nos caches;

Pode não funcionar bem com dados que são muito atualizados: Se os dados do sistema forem atualizados com grande frequência o uso de caches distribuídos não trará benefícios;

Pode se tornar um problema se não usado corretamente: Se feito de maneira incorreta, o sistema pode ficar dependente do cache para funcionar, como discutido na seção de Implementação;

Veja também

Sharding, 3.3, para possíveis maneiras de particionar os dados.

3.7 Uma Pequena Linguagem de Padrões

Um único padrão não viabiliza a elaboração e a implementação de toda uma arquitetura de sistema. Ele apenas ajuda a projetar um aspecto do sistema. Mesmo projetando um aspecto corretamente, pode ocorrer de a arquitetura como um todo não atender a seus requisitos. Portanto, é preciso um conjunto de padrões que possam ser utilizados e que cubram vários problemas de projeto [Buschmann et al. 2007b].

Padrões não existem sozinhos, eles interagem entre si, se relacionam. Eles podem se complementar, podem colaborar para resolver problemas, podem ser sinônimos onde a diferença entre eles é sutil, podem competir com ou estender outros padrões.

Os padrões apresentados neste trabalho compõem, até o momento, um catálogo de padrões. Entretanto, existe relacionamentos entre os padrões, como foi apresentado nos próprios padrões, mas estes relacionamentos não estão documentados de uma maneira que deixa claro o relacionamento entre eles.

Uma linguagem de padrões organiza, documenta e deixa claro o relacionamento entre os padrões. Uma linguagem de padrões pode ser definida como:

Uma *Linguagem de Padrões* define uma rede de padrões que dão suporte uns aos outros, tipicamente uma árvore ou um grafo orientado, de tal maneira que um padrão pode opcionalmente ou necessariamente mover-se para outro, elaborando um projeto em maneira particular, respondendo a Forças específicas, tomando diferentes caminhos quando apropriado [Buschmann et al. 2007a].

Outra maneira, talvez mais fácil, de compreender uma linguagem de padrões, é como um conjunto de padrões, organizados em um diagrama, onde são evidenciados quais padrões possuem relacionamentos uns com os outros e quais são esses relacionamentos. Uma linguagem de padrões é uma ferramenta, que oferece orientação em como criar um tipo particular de sistema, que deve ser usada durante o projeto de um sistema para guiar o arquiteto na busca por soluções.

A partir dos padrões apresentados neste capítulo, é possível identificar uma pequena linguagem de padrões para construção de sistemas escaláveis. Não é uma linguagem completa, os padrões apresentados formam apenas um conjunto inicial de padrões para escalabilidade, entretanto é importante documentar seus relacionamentos. A Figura 3.31 ilustra a linguagem formada pelos padrões apresentados.

O padrão *Shared Nothing* possui relacionamento com os padrões *Sharding*, *BASE* e *Camada de Caches Distribuídos*. Estes padrões utilizam o padrão *Shared Nothing* como um meio para obter escalabilidade linear. O padrão *Shared Nothing* também possui um relacionamento com camadas de caches para redução de I/O. Para evitar o uso de transações distribuídas em um sistema que é parcialmente *Shared Nothing* pode-se utilizar o padrão *Sagas*.

Sharding possui relacionamento próximo com outros padrões que endereçam problemas e aspectos que *Sharding* não trata. O padrão *Sagas* pode ser utilizado para evitar o uso de

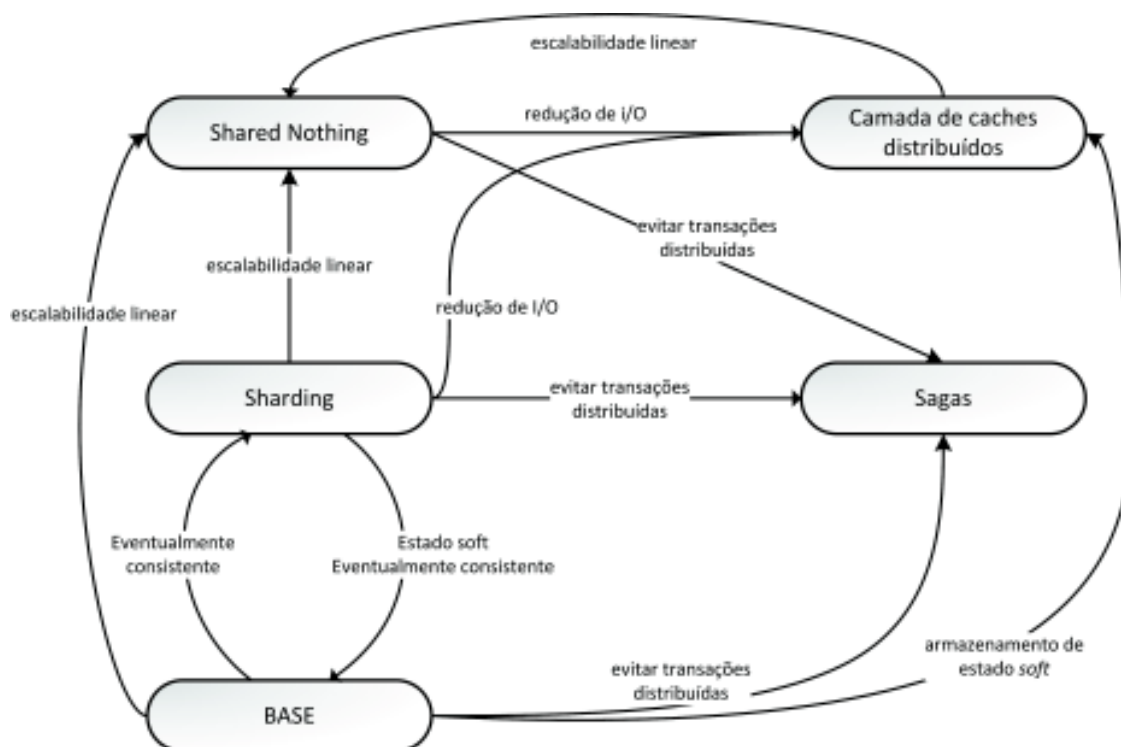


Figura 3.31: Linguagem de padrões

transações distribuídas quando se utiliza dados particionados por *Sharding*. Para melhorar o desempenho de um *Sharding* é possível utilizar uma Camada de Caches Distribuídos. *Sharding* também pode utilizar BASE para obter consistência eventual dos dados.

O padrão BASE possui um relacionamento com *Shared Nothing*, que pode ser utilizado para se ter um sistema quase linearmente escalável. Para implementar estado *soft* dos dados é utilizado a Camada de Caches Distribuídos. Para ter um sistema basicamente disponível utiliza-se *Sharding* dos dados.

Sagas são usadas para evitar o uso de transações distribuídas. Os padrões Sagas e Camada de Caches Distribuídos têm um escopo menor que os demais padrões, e como existem por si só e são utilizados para complementar os demais, entretanto não são complementados por outros padrões apresentados neste trabalho.

Uma linguagem de padrões é um grafo, entretanto, na Figura 3.31 não há indicação do ponto por onde se pode iniciar o uso dos padrões. Isso se deve aos fatos de que não há uma ordem definida que deve ser seguida para o uso dos padrões e de que é possível aplicar, a um mesmo problema, mais de um padrão em ordens diferentes. Para iniciar o uso dos padrões a principal sugestão é identificar os problemas enfrentados e verificar quais padrões podem ser aplicados. Outra sugestão, a ser utilizada quando ainda não se têm bem definidos os problemas enfrentados ou está-se iniciando o projeto arquitetural de um novo sistema, é aplicar os padrões que são mais genéricos e tem escopo amplo, como *Shared Nothing* e BASE.

Apesar de pequena, a linguagem apresentada é útil, pois através dela é mais fácil entender o

relacionamento entre os padrões e utilizá-los. Através da aplicação de um padrão em um sistema pode-se com mais facilidade se escolher padrões complementares para auxiliar na resolução de problemas de projeto ainda não resolvidos.

Capítulo 4

Diretrizes Arquiteturais para Escalabilidade

Este capítulo tem o objetivo de apresentar diretrizes para o projeto e a construção de arquitetura de sistemas escaláveis. Diretrizes provêm técnicas e estratégias, e mesmo conselhos, gerais que podem ser usados para guiar o projeto de um sistema pelo caminho correto para se alcançar escalabilidade. As diretrizes geralmente têm um escopo amplo e cobrem várias áreas, como partionamento de funcionalidades, partionamento de dados, uso de transações, etc. Diferente dos padrões de projetos, as diretrizes não são, todas as vezes, soluções para problemas específicos e recorrentes, elas dão suporte aos padrões, que muitas vezes as aplicam e complementam.

Na próxima seção são apresentadas as diretrizes, para cada uma diretriz é apresentada uma frase que resume a diretriz e uma breve discussão. Como todo o trabalho, as diretrizes são relacionadas à escalabilidade de sistemas. Há muitas outras diretrizes que podem se aplicadas referentes à melhoria de desempenho e paralelismo de sistemas que não estão inclusas aqui pois o foco do trabalho é a escalabilidade.

4.1 Diretrizes

Não deixe a escalabilidade para depois. Escalabilidade é uma prática difícil de ser realizada, não é algo que pode ser pensado ou feito depois. Deve-se projetar e construir um sistema já visando sua escalabilidade para que, quando pronto, pela adição de mais recursos, tenha-se aumento, ou pelo menos manutenção, do desempenho frente ao aumento da carga de trabalho. Um sistema no qual não houve preocupação com a escalabilidade poderá ser escalado verticalmente até algum ponto indeterminado, e provavelmente será muito difícil, ou até impossível, de escalar horizontalmente.

Particione o sistema funcionalmente. Funcionalidades relacionadas devem ficar juntas, funcionalidades que não são relacionadas devem ficar separadas [Shoup 2008]. Preferencial-

mente, as funcionalidades não relacionadas devem ter um baixo acoplamento para permitir que sejam escaladas com independência. Após identificados e particionados os domínios funcionais do sistema, a estratégia é criar *pools* de computadores que hospedem os subsistemas (instâncias) que ofereçam serviços coesos a outros *pools* de computadores. Por exemplo, pode-se ter um conjunto de instâncias que oferece serviços relacionados à busca, outro conjunto que ofereça serviços relacionados a gerenciamento de estoque, outro de serviços de cobrança, e assim por diante. O objetivo de se ter *pools* de instâncias com serviços coesos é a possibilidade de escalá-los de maneira independente de acordo com a demanda de trabalho e uso específico de recursos de cada funcionalidade. Esta diretriz pode ser aplicada aos dados e à camada de acesso a dados através do padrão *sharding* (ver 3.3).

Particione o sistema horizontalmente para distribuir a carga. Mesmo com o uso de particionamento funcional haverá um momento onde apenas um computador não será capaz de atender à demanda de trabalho [Shoup 2008]. Assim, é preciso dividir a carga de trabalho entre várias instâncias da aplicação. Para conseguir “expandir” o sistema na horizontal, a melhor maneira é através da construção de sistemas sem estado (*stateless*), aplicando por exemplo uma Arquitetura *Shared Nothing* (ver 3.2). Com isso, para atender a maior demanda por trabalho, adiciona-se mais instâncias do sistema. O particionamento horizontal, no que diz respeito aos dados e à camada de acesso a dados, é possível através de *sharding* (ver 3.3).

Particione o sistema funcionalmente e horizontalmente. Utilize as diretivas de particionamento funcional e horizontal em conjunto sempre que possível para maximizar os ganhos de escalabilidade e desempenho.

Faça balanceamento da carga de trabalho. De nada adianta particionar horizontalmente se a carga de trabalho entre os computadores é desigual. Faça um balanceamento de carga entre todos os computadores para obter uma distribuição de carga igual ou, ainda melhor, proporcional a capacidade de processamento de cada computador. Por exemplo, um computador com 8 processadores deve atender a mais requisições do que outro com 4 processadores.

Faça com que os nós de processamento sejam o mais independentes possível. Com uma carga de trabalho alta será muito difícil obter um bom desempenho se houver dependência forte entre os nós para processar uma requisição. Faça com que os nós sejam os mais autônomos possíveis, para que consigam tomar decisões apenas com base em seu estado local [Vogels 2007].

Construa o sistema objetivando um baixo acoplamento. Acoplamento [Pressman 2004] é o grau de dependência entre módulos de um sistema. Um módulo pode ser um subsistema, uma camada do sistema, outro sistema, ou qualquer outro componente que faça parte de um sistema de software. Acoplamento é um conceito bem conhecido e bem compreendido.

Módulos possuem acoplamento alto, quando há uma dependência forte entre eles, que tem

como consequências: uma alteração em um dos módulos implica em alteração no outro módulo; os módulos são difíceis de compreender sem conhecimento do outro módulo; um módulo é difícil de ser reutilizado, pois módulos acoplados devem ser utilizados em conjunto [Pressman 2004]. Módulos possuem um acoplamento baixo quando a dependência entre eles é a menor possível e assume-se o mínimo sobre o outro. [Pressman 2004].

Construir um sistema de acoplamento baixo não o faz escalar, mas torna os módulos mais independentes, fazendo com que seja mais fácil escalar cada um individualmente, sem forçar mudanças em outros módulos. Se torna mais fácil, e seguro, aplicar as outras diretrizes e técnicas discutidas neste trabalho.

Em uma situação onde um módulo A possui alto acoplamento com um módulo B, para escalar A será preciso escalar B. Por exemplo, suponha que há um módulo A que faz chamadas para um módulo B, que é responsável por enviar e-mails seguindo instruções de outros módulos. As chamadas feitas para o módulo B são síncronas, o que aumenta o acoplamento. Como A e B são altamente acoplados, para escalar A deve-se escalar B. Uma maneira de fazer os dois módulos menos acoplados é trocar a chamada síncrona por uma fila de mensagens, assim o módulo A pode enviar uma mensagem para a fila e ir fazer outra atividade, enquanto a mensagem será consumida pelo módulo B quando for apropriado a ele. Os dois módulos trabalham o mais rápido que puderem e o desempenho e escalabilidade de um não está amarrada à do outro, pode-se escalar A e B de maneira independente.

Use comunicação assíncrona. Sempre que possível use comunicação assíncrona para auxiliar a desacoplar funcionalidades. Se um módulo A comunica-se com um módulo B de maneira síncrona, A e B estão fortemente acoplados e isso impacta na escalabilidade, pois para escalar A é preciso escalar B. O mesmo impacto ocorre na disponibilidade, se B estiver indisponível então A também estará. Se A e B comunicam-se de maneira assíncrona então é possível escalar A e B de maneira independente. O mesmo efeito ocorre sobre a disponibilidade. Para integrar A e B de maneira assíncrona pode-se utilizar filas de mensagens ou um processo *batch*, por exemplo [Shoup 2008].

A comunicação assíncrona é uma maneira eficaz de se conseguir baixo acoplamento e pode, e deve, ser aplicada internamente ao sistema para comunicação entre componentes. Um exemplo de técnica para implementar isto é o uso de SEDA (*Staged Event-driven Architecture*) [Welsh et al. 2001].

Além do ganho em escalabilidade há ganhos de desempenho, já que com o uso de chamadas assíncronas não há bloqueio do chamador, que fica livre para realizar outras tarefas enquanto espera por uma resposta.

Evite o uso de transações distribuídas. O uso de transações distribuídas, como o protocolo 2PC, que é um protocolo pessimista, tem um custo alto de coordenação e latência. O aumento de custo do protocolo 2PC aumenta geometricamente em relação ao aumento da quantidade de participantes da transação distribuída (este efeito ocorre com qualquer protocolo que requer

concordância de todos os seus participantes). Além do problema de desempenho, a disponibilidade é afetada, pois para que uma transação distribuída seja completada com sucesso todos os participantes devem estar funcionando. Se apenas um dos participantes não estiver funcionando não haverá como completar a transação. À medida que se escala horizontalmente um sistema, a probabilidade de que algum participante falhe é cada vez maior. Alternativas para trabalhar sem transações distribuídas são BASE (ver 3.4) e Sagas (ver 3.5). Outra maneira de evitar o uso de transações distribuídas é desnormalizar o banco de dados para que seja possível executar uma transação em um só banco de dados.

Faça a maior quantidade de trabalho possível de maneira assíncrona. Quanto mais trabalho puder ser feito depois, melhor. Deixe a maior quantidade de trabalho possível para ser realizada mais tarde, de maneira assíncrona. Quando uma requisição for recebida faça apenas o essencial e retorne rapidamente uma resposta, deixe o restante do trabalho sendo feito, ou para fazer depois em segundo plano, de maneira assíncrona e desacoplada da requisição. Com isso, diminui-se o tempo de resposta do sistema. Também é uma estratégia que torna o custo de hardware do sistema menor, pois o processamento de requisições de maneira síncrona força o uso de hardware para suportar a mais alta carga de trabalho esperado. Com o uso de processamento assíncrono, a infra-estrutura necessária para realizar o processamento pode ser dimensionada para uma carga de trabalho média, pois agora se tem mais tempo para processar as requisições [Shoup 2008].

Faça um uso correto de cache. Cache deve ser utilizado para minimizar I/O e aumentar o desempenho. O uso de cache deve ser avaliado para cada sistema, levando-se em consideração as restrições de armazenamento, disponibilidade e tolerância a dados desatualizados. O ponto importante aqui é o uso incorreto de cache, onde o sistema depende de tal forma do cache que não é possível funcionar sem ele. O cache deve auxiliar na melhoria do desempenho e escalabilidade, mas não deve ser uma dependência capaz de tornar o sistema indisponível, com o cache fora de funcionamento o sistema deve continuar a funcionar normalmente mesmo com um desempenho inferior do que com o cache.

Não desconsidere a latência, adapte-se a ela. A existência da latência é um fato, e como diz a segunda falácia dos sistemas distribuídos [Rotem-Gal-Oz 2007], a latência não é zero. Além disso, deve-se lembrar que a latência não desaparecerá e quase sempre não está sob controle do sistema. Baseando-se nestes fatos não se deve tentar projetar e implementar um sistema que tenta ignorar ou tenta dar a impressão de que a latência é zero, a latência deve ser aceita e deve-se trabalhar com ela. Assuma que a latência será grande, pois se o sistema funcionar com uma latência grande com certeza funcionará com uma latência pequena [Pritchett 2007a]. Técnicas para lidar com a latência são o uso de assincronismo e BASE.

Separe a política do mecanismo. A separação entre política e mecanismo é uma boa prática

conhecida e pode ajudar a aumentar a escalabilidade. A partir da separação das funcionalidades de política e mecanismo faz-se particionamento funcional e desacopla-se as funcionalidades, possibilitando escalar cada uma de maneira independente. Um exemplo desta separação é com o uso do memcached [Danga] (ver 3.6), que oferece apenas o mecanismo de cache e a política de uso deve ser implementada pelo restante do sistema.

Separe a aplicação de seu estado. O mesmo princípio da separação de política e mecanismo se aplica para a aplicação e seu estado. Separe o estado da aplicação para escalá-los de maneira independente. Aqui, estado se refere a dados como sessão de usuários, dados que precisam ser mantidos entre requisições, etc.

Entre escalabilidade e desempenho, escolha escalabilidade. Muitas vezes há situações onde se deve escolher entre escalabilidade e desempenho, escolha escalabilidade. Por exemplo, se tiver que escolher entre utilizar *stored procedures* de banco de dados ou realizar as consultas pela aplicação, escolha esta última opção. O uso de *stored procedures* não escala, pois estão centralizadas no banco de dados, mas a realização das consultas na aplicação escala. Aumentando o desempenho aumenta-se a escalabilidade mas aumentando cada vez mais o desempenho, por muitas vezes não se vai deixar os usuários mais satisfeitos. Aumentando a escalabilidade os usuários ficarão mais satisfeitos, pois o sistema será capaz de atendê-los sempre, mesmo que sejam muitos usuários.

Se não for possível escalar então escale ao redor. Há situações onde não é possível escalar alguma parte ou algum componente de um sistema, por não ser tecnicamente viável, ou por ser muito difícil, ou muito caro. Nestes casos, construa subsistemas escaláveis ao redor do que não pode ser escalado para que este não se torne um gargalo. Considere o caso de um banco de dados legado que não consegue mais atender a carga de trabalho atual e que não pode ser modificado para aplicar um *sharding*. Pode-se construir, então, subsistemas escaláveis ao redor do banco de dados que utilizem Arquiteturas *Shared Nothing*, BASE e uma Camada de Cache Distribuído, diminuindo a carga do banco de dados.

Implemente e disponibilize serviços de granularidade alta. Procure utilizar serviços de granularidade alta, isto favorece a realização de mais processamento em um mesmo local (no mesmo processo, na mesma memória local, etc.).

Planeje para um ambiente heterogêneo. À medida que se escala horizontalmente, ao longo do tempo, com certeza haverá diferenças entre os hardware utilizados. Deve-se construir a arquitetura do sistema levando este ponto em consideração para que o sistema continue funcional e mantenha suas características não funcionais.

Use operações idempotentes. Operações idempotentes são operações que podem ser executadas inúmeras vezes sempre com o mesmo efeito. Por exemplo, não importa quantas vezes se

requisita o cancelamento de uma compra, o cancelamento será feito apenas uma vez. O uso de operações idempotentes evita que o sistema tenha que saber se determinada operação já foi aplicada e evita que a operação seja executada novamente. Sem ter que controlar várias execuções repetidas das operações há aumento do desempenho.

Discuta os requisitos de negócio antes de tomar decisões arquiteturais sobre escalabilidade. Quem define os requisitos de escalabilidade são os requisitos de negócios, portanto primeiro é preciso conhecê-los antes de tomar decisões arquiteturais com o objetivo de ter escalabilidade. Deve-se lembrar que escalabilidade é um entre muitos requisitos funcionais e não funcionais e que a arquitetura do sistema deve ser escalável mas também deve atender a todos os outros requisitos.

Um sistema escalável deve ser composto de subsistemas escaláveis. No projeto e implementação de sistemas escaláveis é preciso atenção para que todas as partes do sistema sejam escaláveis. Uma parte do sistema que não é escalável se tornará um gargalo em algum momento. Só é possível fazer um sistema escalável quando todas as suas partes são escaláveis. Caso alguma parte não seja escalável, devem ser aplicadas técnicas para construir um “invólucro” escalável ao redor destas partes.

Escale apenas o que deve ser escalável. Muitas das vezes não é necessário escalar o sistema todo, apenas parte dele. Em um sistema *web* desenvolvido pelos autores, quando um usuário A realizava algumas ações em particular, outros usuários, que deveriam estar usando o sistema no mesmo momento, e que tinham especial interesse no usuário A, eram notificados sobre as ações.

Para implementar esta funcionalidade, o navegador do usuário realizava verificações periódicas no sistema para buscar eventos endereçados a ele referentes a usuários de seu interesse. O intervalo entre as verificações era de alguns segundos, mas com algumas centenas de usuários utilizando o sistema ao mesmo tempo isto gerou uma quantidade de requisições de verificação de eventos proporcional. As requisições de verificação estavam sobrecarregando os computadores e prevenindo o uso das outras funcionalidades do sistema.

Este requisito em particular era o único que foi identificado como sendo necessário de ser escalável, as outras funcionalidades do sistema podiam ser tratadas com uma arquitetura padrão *web* em 3 camadas (3.2) com uso combinado de cache. Para solucionar o problema foi construído um novo subsistema de eventos apenas para receber e tratar as requisições de verificação de eventos e uma camada de cache para armazenar em memória os eventos. As requisições de verificação de eventos eram direcionadas aos computadores do subsistema de eventos, outras requisições eram enviadas a outros computadores. Quando um usuário A realizava alguma ação em particular, um evento era publicado para este novo subsistema e armazenado em memória caso houvesse interessados no evento utilizando o sistema naquele momento. Quando o navegador de um outro usuário realizava requisições de verificação de eventos os computadores do

subsistema de eventos precisavam apenas acessar em memória os eventos. Com esta solução foi possível distribuir a carga do sistema entre o subsistema de eventos e o restante do sistema.

Este é um exemplo que ilustra a necessidade de conhecer bem os requisitos de escalabilidade do sistema para que se saiba quais serão os desafios de escalabilidade que serão enfrentados e onde eles estão. Nem sempre é necessário fazer todo o sistema escalável, talvez apenas algumas partes precisem ser escaláveis, foque os esforços onde é realmente importante.

Capítulo 5

Exemplo de uma Arquitetura de um Sistema Escalável

Neste capítulo é apresentado um exemplo de uma arquitetura de um sistema escalável. O objetivo é apresentar a arquitetura deste sistema como um exemplo da aplicabilidade dos padrões e diretrizes discutidos neste trabalho. São apresentados os usos dos padrões e das diretrizes e os motivos e decisões que levaram ao seu uso.

O sistema apresentado é um sistema real, onde o autor foi um dos arquitetos responsáveis, desenvolvido para uma grande empresa nacional. O desenvolvimento do sistema foi realizado durante o ano de 2009 e o sistema encontra-se em produção.

5.1 Requisitos Funcionais

O objetivo do projeto era construir um sistema que realiza o processamento de dados de transações enviados pelos seus usuários e que tivesse suporte a implementações de regras de processamento personalizadas pelos usuários.

Os usuários do sistema podem utilizá-lo de duas maneiras. A primeira maneira é através de *Web Services* para realização de processamento de transações e consultas aos processamentos já realizados. A segunda maneira é através de aplicações *web* para administração.

O leque dos principais requisitos funcionais é bem pequeno, composto de apenas duas operações. O principal requisito é realizar o processamento das transações. Para tal os passos de uso são os seguintes:

1. Um sistema externo, pertencente ao usuário, envia informações para processamento utilizando um *Web Services* disponibilizado pelo sistema;
2. O sistema recebe as informações e aplica vários algoritmos e mecanismos internos, e externos, para processar a requisição;
3. O sistema retorna o resultado do processamento para o usuário junto com um identificador único do processamento.

O segundo requisito principal é consultar, posteriormente, os resultados dos processamentos através de seu identificador único. Durante o restante da discussão deste exemplo, a experiência deter-se-á apenas a estes requisitos.

5.2 Requisitos Não Funcionais

Os principais requisitos não funcionais do sistema são:

- Desempenho:
 - Vazão (capacidade): deve suportar uma carga de trabalho inicial de 100 transações/segundo, com planejamento para 300 transações/segundo ao final de um ano;
 - Tempo de resposta: as requisições dos clientes devem ser respondidas em no máximo 1 segundo, para toda a faixa de valores possíveis de vazão.
- Escalabilidade: o sistema deve ser facilmente escalável na horizontal para aumentar seu desempenho. O objetivo é ser capaz de aumentar a vazão o quanto for desejado ou necessário mantendo o tempo de resposta;
- Disponibilidade: 99% de disponibilidade;
- Extensibilidade: o sistema deve ser facilmente extensível para inclusão de novas funcionalidades;
- Segurança: toda a comunicação via *Web Services* deve ser criptografada. O sistema deverá armazenar dados sensíveis criptografados. Todas as requisições devem ser autenticadas e autorizadas.

5.3 Arquitetura e Funcionamento

Para elaboração da arquitetura do sistema, o primeiro passo foi a análise detalhada dos requisitos funcionais e não funcionais. Depois disso foi feito um particionamento funcional do sistema e chegou-se à seguinte arquitetura (Figura 5.1).

O módulo **Receptor** é quem disponibiliza os *Web Services* aos usuários e é responsável por: receber as requisições *Web Services*; validar todos os dados da requisição, garantindo que o sistema processará apenas requisições bem formadas; realizar a autenticação e autorização dos usuários; realizar a criptografia dos dados na rede; e criptografar os dados das requisições.

O **Receptor** tem uma outra responsabilidade que é verificar os dados das requisições contra uma lista negra que contém muitas informações, tais como por exemplo endereços IP. Quando uma requisição é verificada contra a lista negra e algum dado da requisição é encontrado na

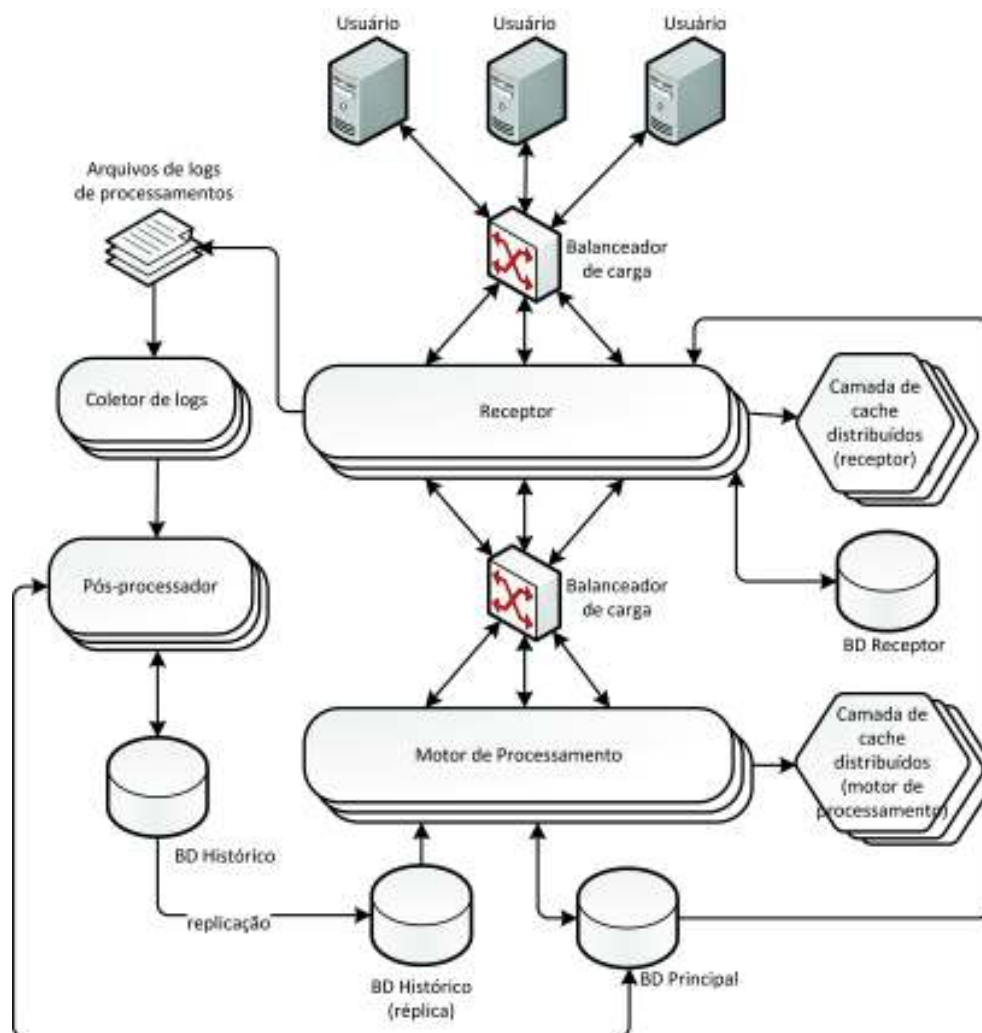


Figura 5.1: Arquitetura do sistema de exemplo

lista negra, imediatamente é retornada uma resposta informando que dados da requisição foram encontrados na lista negra.

Como a quantidade de informações da lista negra é grande, e são atualizadas constantemente, elas são armazenadas em uma camada de caches distribuídos. A lista negra é armazenada de maneira persistente em um banco de dados (de nome “BD Receptor”). Esta camada de cache também armazena credenciais dos usuários e um cache negativo de autenticação. As credenciais dos usuários são lidas do banco de dados principal do sistema (de nome “BD Principal”).

Outra responsabilidade do Receptor é fazer controle de inundação de requisições, limitando a quantidade de requisições que um usuário pode fazer por período de tempo e evitar ataques de negação de serviços. Uma vez recebida uma requisição pelo Receptor e executadas todas suas responsabilidades, a requisição é delegada a outro módulo para realização do processamento propriamente dito. Uma vez que o processamento é finalizado, o Receptor recebe a resposta, grava os dados da requisição e o resultado do processamento em um arquivo em disco, de maneira assíncrona, e retorna a resposta para o usuário. As transações gravadas em disco serão

posteriormente processadas por outro módulo.

O módulo **Motor de Processamento** recebe requisições do Receptor e realiza o processamento da requisição, feita através da aplicação de algoritmos matemáticos e várias regras definidas através de uma linguagem própria desenvolvida especialmente para descrição das regras de processamento do sistema.

Os modelos matemáticos utilizados requerem uma grande base de dados de histórico de análises de transações para serem eficientes. O histórico das análises fica armazenado em banco de dados, de onde o Motor de Processamento lê o histórico, e é composta dos dados das transações, do resultado das análises e de muitos outros dados calculados a partir de análises já realizadas.

O Motor de Processamento também tem a responsabilidade de contabilizar os processamentos realizados pelos usuários. Esta contabilização deve ser feita de maneira online, pois um usuário tem um limite de processamentos que pode realizar e após ultrapassar este limite o sistema não deve mais processar requisições do usuário.

Para toda requisição, os contadores de acesso do usuário são atualizados para garantir que o usuário não ultrapasse o limite que lhe é permitido. Todas estas informações, dados de histórico, regras a serem aplicadas no processamento de transações e contadores de contabilização de processamentos são armazenados em uma camada de caches distribuídos. Todas as requisições enviadas pelo Receptor ao Motor de Processamento são balanceadas por um hardware balanceador de requisições de rede.

As informações referentes a contadores de contabilização e regras para processamento são armazenadas no banco de dados Principal. Depois de feito um processamento, o Motor de Processamento retorna o resultado para o Receptor, isto é, ele não armazena o resultado em banco de dados.

O subsistema **Coletor de Logs** é responsável por coletar os arquivos de logs, produzidos pelo Receptor, que contém as requisições e o resultados de seus processamentos, e distribuí-los para pós-processamento. O módulo **Pós-processador** é responsável por ler os arquivos de logs e realizar duas tarefas.

A primeira é armazenar os dados, referentes à requisição, e o resultado do processamento, no banco de dados de histórico. A segunda é utilizar os dados das requisições e os resultados dos processamentos para recalcular os dados da base de históricos. Para cada requisição e sua respectiva resposta é feito o recálculo de várias informações de histórico.

Pela Figura 5.1 pode-se ver que há 2 balanceadores de carga presentes, contudo esta não é a quantidade real. Os balanceadores de cargas podem se tornar gargalos e para evitar isto é feito o uso de mais de um roteador em cada situação (para receber requisições de clientes e entre os Receptores e Motores de Processamento) e várias rotas de rede.

Para deixar claro o funcionamento do sistema, são resumidas aqui as ações tomadas, e quem as toma, para realizar o processamento de uma requisição:

1. O usuário envia uma requisição para processamento para o sistema através de um *Web*

Service;

2. O módulo Receptor recebe a requisição e realiza validação dos dados recebidos, autenticação e autorização do usuário, e criptografia de dados;
3. O módulo Receptor delega a requisição ao módulo de Motor de Processamento;
4. O módulo Motor de Processamento recebe a requisição realiza o processamento, atualiza os contadores de acesso do usuário e retorna uma resposta ao Receptor;
5. O Receptor recebe a resposta, salva em disco a requisição e o resultado, e retorna uma resposta para o usuário;
6. O Coletor de Logs coleta o arquivo de logs do Receptor e envia para o Pós-processador;
7. O Pós-processador armazena as requisições e as análises resultantes no banco de dados e recalcula a base de dados históricos.

5.4 Aplicabilidade dos Padrões e Diretrizes

Nesta seção são discutidos os Padrões e as Diretrizes utilizados pelo sistema de exemplo e como eles resultaram em um sistema escalável. Para as Diretrizes, primeiramente é listada a ‘Frase de Resumo da Diretriz’ e em seguida ela é discutida no contexto do sistema.

5.4.1 Aplicabilidade das Diretrizes

Particione o Sistema Funcionalmente. Particionamento Funcional foi um dos primeiros passos realizados. Foram identificadas as funcionalidades mais relacionadas que foram agrupadas com o objetivo de criar subsistemas coesos que pudessem ser escalados separadamente. Os principais subsistemas identificados foram o Receptor e o Motor de Processamento, que juntos têm responsabilidade pela maioria dos requisitos funcionais.

Ao invés de dois subsistemas poder-se-ia ter feito apenas um módulo que agregasse todas as funcionalidades, entretanto decidiu-se separar as funcionalidades consideradas de suporte ao processamento no módulo Receptor e o processamento em si em outro módulo, pois as necessidades computacionais das atividades realizadas em cada módulo são diferentes; o processamento dos dados das requisições exige mais memória e capacidade de processamento.

A intenção também era que o Receptor fosse uma barreira de proteção para o módulo de Motor de Processamento. Os módulos Coletor de Logs e Pós-processador não surgiram a partir do particionamento funcional, mas sim como uma resposta à necessidade de desempenho e escalabilidade (estes módulos serão discutidos mais a frente).

Particione o Sistema Horizontalmente para distribuir a carga. Todos os módulos do sistema foram construídos para não armazenarem estado, assim qualquer módulo é capaz de atender a

uma requisição, sem a necessidade dos usuários estarem “amarrados” a uma instância. Na seção sobre a aplicação do padrão *Shared Nothing* haverá uma mais detalhada discussão deste tópico.

Particione o sistema funcionalmente e horizontalmente. Como foi apresentado nas duas diretrizes anteriores o sistema foi particionado funcional e horizontalmente, dando ao sistema o melhor dos dois.

Faça balanceamento da carga de trabalho. Para balancear as requisições, decidiu-se pelo uso de um hardware balanceador de cargas. Seu uso é feito para balancear as requisições dos usuários enviadas ao Receptor e as requisições delegadas pelos Receptores para os Motores de Processamento. O módulo Coletor de Logs é o responsável pelo balanceamento de carga dos Pós-processadores.

Faça com que os nós de processamento sejam os mais independentes possíveis. Para processar uma requisição, a única dependência reside entre os módulos ‘Receptor’ e ‘Motor de Processamento’, que mesmo assim possui baixo acoplamento. O Receptor apenas delega a requisição. Cada um dos módulos é capaz de tomar decisões sem a necessidade de entrar em acordo com outros módulos. Por exemplo, para realizar um processamento o Motor de Processamentos não precisa se comunicar com outras instâncias dele mesmo ou algum outro módulo para então poder tomar uma decisão sobre o processamento de uma transação.

Construa o sistema objetivando um baixo acoplamento. Todos os módulos possuem baixo acoplamento uns com os outros. O acoplamento entre Receptor e Motor de Processamento é apenas para que um delegue trabalho para outro. O Receptor salva em disco os logs dos processamentos que são depois consumidas pelo Coletor de Logs e Pós-processador de transações. Os caches distribuídos são utilizados apenas como armazenamento em memória.

Use comunicação assíncrona. Todas as comunicações entre Receptor e Motor de Processamento, Receptor e camada de caches e Motor de Processamento e caches distribuídos são assíncronas. A única comunicação síncrona é com os bancos de dados.

Evite o uso de transações distribuídas. Todas as transações realizadas são locais, transações distribuídas são utilizadas em um único local, no Pós-processador. O Receptor e o Motor de Processamentos têm integração com alguns outros sistemas externos (que não podem ser apresentados aqui) e nenhuma destas transações é distribuída. No Pós-processador o uso de transações distribuídas é feito devido à facilidade de implementação e não há a necessidade de se ter o mais alto desempenho possível, já que é um módulo que processa as transações em segundo plano de maneira assíncrona e desacoplado dos processamentos de requisições e não há problema se este processamento não for imediato.

Faça a maior quantidade de trabalho possível de maneira assíncrona. Duas atividades importantes foram retiradas do fluxo principal do sistema e são feitas de maneira assíncrona: o armazenamento das transações e a atualização da base histórica de informações. Estas duas operações têm um custo computacional e de I/O altos e, do ponto de vista dos requisitos de negócios, fazê-las no momento do atendimento de uma requisição não afetaria o negócio. Além disso, o requisito de ter um tempo de resposta pequeno é mais importante, por isso estas atividades são feitas em segundo plano de maneira assíncrona.

Faça um uso correto de cache. As duas camadas de cache do sistema são utilizadas apenas como um mecanismo de armazenamento distribuído em memória, evitando grande parte dos acesso aos bancos de dados. O sistema funciona normalmente sem as camadas de cache.

Separe a política do mecanismo. No uso dos caches distribuídos, estes são usados apenas como mecanismo, toda a política é implementada pelos usuários dos caches. No caso do Pós-processador, a política de balanceamento de carga foi separada em um outro módulo.

Se não for possível escalar então escale ao redor. O sistema integra-se com outros módulos, que não podem ser apresentados aqui, e que têm escalabilidade e desempenho imprevisíveis. Assim, o sistema foi projeto para escalar independentemente destes sistemas externos, utilizando como principais estratégias, realizar o cache dos dados destes sistemas e realizar um processamento em segundo plano assíncrono.

Planeje para um ambiente heterogêneo. O sistema foi desenvolvido em linguagem Java para que pudesse funcionar em qualquer ambiente que suporte a plataforma Java.

Use operações idempotentes. O Pós-processador é capaz de processar o mesmo arquivo de log várias vezes, sempre com o mesmo efeito, se um resultado já foi armazenado e o histórico atualizado com seus dados, pode-se processá-lo novamente com o mesmo efeito.

Escale apenas o que deve ser escalável. Dos requisitos do sistema há muitos outros que não têm a necessidade de serem altamente escaláveis. Os subsistemas *web* de administração, por exemplo, são aplicações *web* comuns com uma arquitetura de 3 camadas (Apresentação, Negócios, Acesso a dados), onde não foram aplicados padrões de escalabilidade além de uma arquitetura *Shared Nothing* e caches locais. O módulo de Pós-processador é outro que possui requisitos de escalabilidade e desempenho mais fracos, portanto foi permitido a ele o uso de transações distribuídas.

5.4.2 Aplicabilidade dos padrões

Shared Nothing

O padrão *Shared Nothing* foi aplicado em todos os subsistemas e foi o principal padrão para tentar atingir o objetivo de ter escalabilidade horizontal linear. Ele foi utilizado como o primeiro padrão a ser aplicado para o projeto do sistema devido a ser o padrão mais genérico e amplo dos padrões apresentados neste trabalho. Todos os módulos são independentes, não há estado mantido em memória - que faça com que um usuário tenha de sempre ser atendido por determinado computador (ou instância do subsistema), não há dependência ou qualquer relação entre as requisições dos clientes, não há necessidade de sincronização de estado entre as instâncias e os recursos disponíveis para o sistema são utilizados exclusivamente com tarefas que não são sobretaxa.

A interação entre Receptor e Motor de Processamento é uma dependência, mas, contudo, de baixo acoplamento, assíncrona, e é apenas uma delegação de trabalho.

Com a independência entre os subsistemas, a criação de *pools* de instâncias se torna fácil e para escalar o sistema na horizontal basta então adicionar mais instâncias, como ilustrado na figura 5.2.

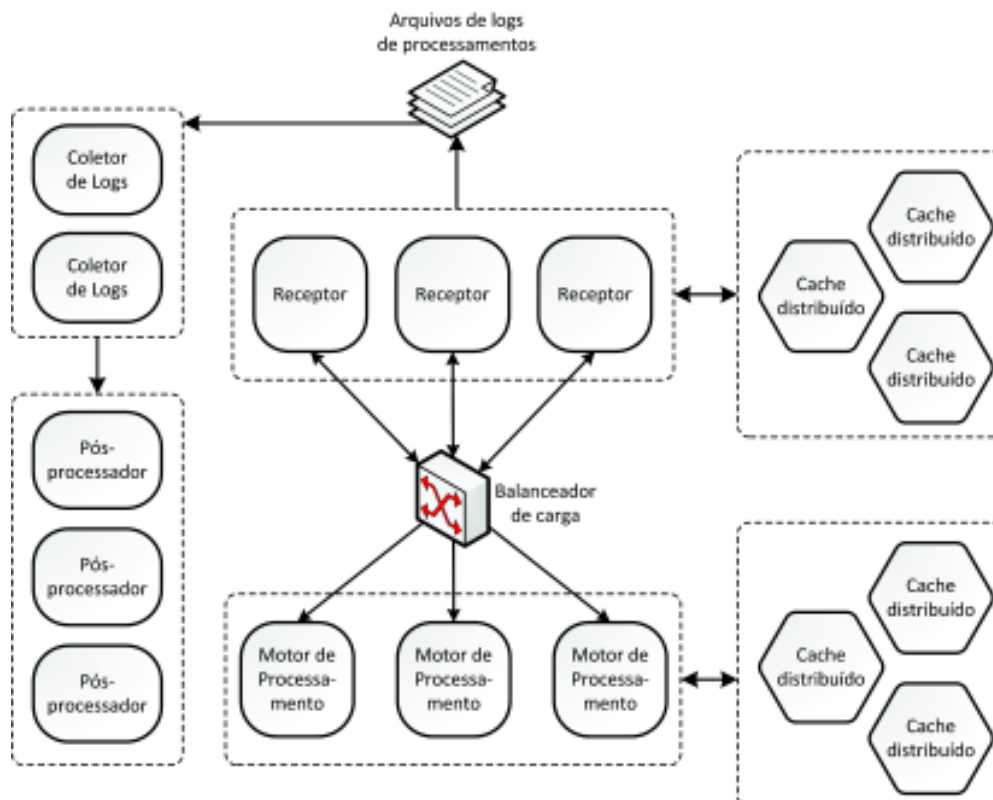


Figura 5.2: Construção de *pools* de instâncias para o sistema exemplo

Esta figura apresenta um exemplo de implantação do sistema em computadores. O tamanho dos *pools* de instâncias não possui um limite determinado, é possível adicionar quantas instân-

cias forem precisos até o limite suportado pelo sistema. Outra vantagem da independência entre os subsistemas é no aumento da disponibilidade já que qualquer instância é capaz de processar uma requisição e tem-se várias instâncias à disposição (há mais de um balanceador de carga, apesar de não representado na figura).

Um dos problemas do uso do *Shared Nothing* é a dificuldade de projetar um sistema que possui apenas um módulo de processamento que acumula todas as funcionalidades do sistema. Para aliviar este problema foi usado como suporte o particionamento funcional para que o sistema fosse composto de subsistemas *Shared Nothing* que são escaláveis e que juntos formam um sistema escalável. Com a arquitetura *Shared Nothing* e o particionamento funcional, é possível, ao mesmo tempo, escalar todo o sistema ou apenas uma parte dele que precise ser escalada.

Sharding

Para que fosse possível escalar os dados do sistema, foi utilizado *Sharding*, onde foram considerados os seguintes fatos: (1) haverá uma enorme quantidade de dados históricos no sistema, esperando-se que inicialmente se tenha por volta de 4 bilhões de registros; (2) a atualização dos dados históricos requer o manuseio de grande quantidade de dados e grande quantidade de operações de leitura e escrita; (3) as análises realizadas pelo Motor de Processamento requerem uma grande quantidade de dados históricos, mas estes dados são apenas para leitura, durante o processamento não é feita a atualização dos dados históricos utilizados; (4) os dados históricos são independentes de outros dados, como dados de usuários e regras que são aplicadas no processamento; (5) os dados históricos utilizados por um processamento não precisam ser os mais atuais; (6) os dados históricos devem ser continuamente atualizados.

Em função destes fatos foi feito um *Sharding* Funcional, onde se criou três domínios funcionais, um domínio para dados históricos, um domínio para dados do Receptor e um domínio para o restante dos dados. O objetivo foi evitar que o uso dos dados históricos se tornasse um gargalo durante os processamentos.

Na Figura 5.1 há 4 bancos de dados. O banco de dados “BD Histórico (mestre)” armazena os dados históricos, ele é utilizado apenas pelo Pós-processador, que armazena as transações e faz o recálculo dos dados históricos. O banco de dados “BD Histórico (réplica)” é uma réplica do banco mestre de dados históricos e é utilizado apenas pelo Motor de Processamento, este é um banco apenas para leitura. Com este esquema evita-se que o acesso aos dados históricos se torne um gargalo, pois a atualização dos dados históricos não interfere no acesso a dados necessário para processamento de requisições.

O banco de dados “BD Principal” armazena os dados dos usuários, suas configurações e as regras que dirigem o processamento de transações. Apesar de não ilustrado na Figura 5.1 este banco de dados é replicado. O banco de dados “DB Receptor” armazena os dados da lista negra. No caso deste banco de dados seus dados também estão disponíveis nos caches distribuídos para evitar que este banco se torne um gargalo (discutido mais adiante).

O uso de *Sharding* horizontal não foi necessário. A quantidade de usuários do sistema não

será muito grande, apenas um banco de dados sem particionamento é capaz de suportar o volume de usuários. Em relação ao banco de dados do Receptor o volume de informações é grande, mas ainda não justifica um *Sharding* horizontal, ainda mais se for levado em consideração que seus dados estão armazenados em cache.

O uso de *Sharding* horizontal para o banco de dados de históricos é justificável, entretanto, analisando os fatos relativos a este domínio funcional (como apresentado no início desta seção), viu-se que fazer uma replicação dos dados solucionaria o problema deste banco se tornar um gargalo de maneira mais fácil e mais barata do que com um *Sharding* horizontal. O uso de *Sharding* horizontal não é descartado no futuro, e como é usado *Sharding* vertical o impacto de uma mudança como esta é mais localizado.

Camada de Caches Distribuídos

O sistema possui duas Camadas de Caches Distribuídos que armazenam dados dos bancos de dados e de outros sistemas externos (estes últimos não são apresentados nas figuras). O cache distribuído do Receptor armazena os dados da lista negra, que ficam em armazenamento persistente no banco “DB Receptor”.

O objetivo de armazenar a lista negra em cache foi evitar que o banco de dados que armazena a lista se tornasse um gargalo. A decisão de armazenar os dados da lista negra no cache pode ser questionada, já que para verificar se um dado está na lista negra bastaria fazer uma consulta por chave primária no banco de dados, entretanto alguns motivos levaram a evitar o acesso a banco de dados.

O primeiro é que uma consulta no banco de dados, mesmo que seja por chave primária, é mais lenta que uma consulta ao cache, no banco de dados é preciso acessar o índice da tabela, no cache não é preciso, apenas verifica-se se determinada chave está presente, o particionamento dos dados e *hashing* da chave são feitos pelo cliente de acesso ao cache e não no próprio cache.

Segundo, utilizando um cache evita-se que ataques de negação de serviço comprometam o banco de dados.

Terceiro, com o cache se pode ter em memória apenas os dados mais acessados, isto ajuda a evitar acessos ao banco de dados no caso de uma situação onde, por exemplo, uma nova lista de cartões de créditos roubados surgiu na Internet, utilizando o cache se pode manter estes números em memória.

Quarto, como o volume de dados é muito grande, os bancos de dados não conseguiriam manter em seus caches tantas informações quanto os caches distribuídos. Além da lista negra estes caches armazenam dados de autenticação e autorização e dados para controlar a quantidade de requisições realizadas por período de tempo por usuário para evitar ataques de inundação.

O cache distribuído do Motor de Processamento armazena dados históricos, contadores de acesso dos usuários, dados de sistemas externos, regras a serem aplicadas nos processamentos e configurações dos usuários. Neste caso foi utilizado o cache distribuído com o objetivo de

diminuir o acesso ao banco de dados. Os dados armazenados são aqueles mais utilizados para realizar processamento de transações.

Além do uso do cache distribuído para minimizar o acesso a banco de dados, outro motivo foi o volume de dados potencialmente muito grande. Com um cache distribuído particionado evita-se que dados duplicados sejam armazenados em memória.

No caso da lista negra, por exemplo, pode-se ter alguns milhões de registros de dados. A decisão de utilizar dois caches distribuídos foi influenciada pelo particionamento funcional do sistema o que acabou por determinar a separação dos dados em dois grupos já que os dados não possuem relacionamento. Outras influências foram a disponibilidade e o volume de dados. Na indisponibilidade de um dos caches, a quantidade de dados a ser redistribuídos é menor e um cache não perturbará o outro quando isso ocorrer.

As duas camadas de cache foram usadas em *Sideline*, a invalidação dos dados nos caches é responsabilidade dos subsistemas usuários dos caches. O sistema foi projetado e implementado para não depender do cache para funcionar. Se as duas camadas de cache forem retiradas o sistema funcionará normalmente, entretanto ficará mais lento. Para implementação da camada de cache foi utilizado o memcached [Danga], com *hash* consistente das chaves do cache. Quando algum dos caches se torna indisponível, a estratégia implementada foi ignorar o erro, carregar os dados do banco de dados e armazená-los em algum cache disponível, iniciando um rebalanceamento dos dados.

BASE

O uso do padrão BASE se deu principalmente pelo motivo que o sistema em questão se retroalimenta com seus próprios dados. Os dados gerados pelo sistema, os resultados dos processamentos das transações, são usadas para atualizar e criar mais dados para o sistema, os dados históricos, que por sua vez são usados para novas análises. Pode-se então visualizar que os dados estarão em constante fluxo de alteração e de nada adianta tentar trabalhar com dados que são os mais atuais, pois mudarão assim que outra requisição for processada e os dados históricos atualizados, o que provavelmente ocorrerá daqui a um instante.

Para ter um sistema basicamente disponível, na camada de dados, foi feito *Sharding Vertical* e utilizada replicação dos bancos de dados. Para cache de dados foram utilizadas camadas de caches distribuídos e particionados. Nos outros subsistemas o uso de *Shared Nothing* permitiu a criação *pools* de instâncias.

O uso de dados *soft* é feito em várias operações. No Receptor, os dados de autenticação e autorização do usuário não são os mais atuais (há um algoritmo que detecta este fato e realiza a autenticação e autorização corretamente com os dados mais atuais quando necessário), o mesmo se aplica aos dados da lista global.

No Motor de Processamento, praticamente todos os dados utilizados para processamento não são os mais atuais, a exceção são os contadores de acesso dos usuários e as regras a serem aplicadas. Os dados armazenados em cache, que fazem o papel de dados provisórios, são at-

ualizados no cache em sua grande maioria pelo próprio sistema quando são atualizados pelos usuário através de outros meios, o restante dos dados nos caches possuem um tempo de vida e em algum momento expirarão e serão atualizados.

O ponto mais evidente da consistência eventual do sistema é o armazenamento dos resultados dos processamentos realizados e atualização dos dados históricos. Quando um processamento é realizado, se imediatamente a seguir for feita uma consulta a ele, o sistema retornará uma resposta dizendo que o processamento não existe. O resultados do processamento só será armazenado algum tempo mais tarde pelo Pós-processador. O usuário sabe desta eventual consistência e, portanto, esta situação é esperada. Este mesmo comportamento acontece quando um usuário gera relatórios através das interfaces *web* de Administração.

Os dados históricos são atualizados à medida que o resultados dos processamentos são armazenados, assim, eventualmente todos os dados históricos estarão atualizados (tanto no banco mestre quando em suas réplicas). Neste sistema de exemplo, a consistência eventual dos dados é garantida pelo Pós-processador, que faz o papel de reconciliador de dados, apesar de que neste caso não foi preciso implementarações compensatórias. Um ponto que foi relaxado, do ponto de vista do padrão BASE, foi o uso de transações distribuídas pelo Pós-processador como já foi discutido.

5.5 Análise da Escalabilidade

Nesta seção é feita uma avaliação da escalabilidade do sistema de exemplo com o objetivo de avaliar sua escalabilidade horizontal. Ressalta-se que avaliar o desempenho do sistema não é um objetivo desta avaliação, e, portanto, as avaliações apresentadas aqui não se preocupam com isso. O objetivo desta avaliação é a escalabilidade.

Duas avaliações serão realizadas para avaliar como o desempenho do sistema se comporta e avalia-se duas métricas de desempenho, tempo de resposta e vazão. O primeiro avalia o comportamento do tempo de resposta à medida que o sistema é escalado horizontalmente. O segundo avalia o comportamento da vazão à medida que o sistema é escalado horizontalmente. A partir dessas avaliações é possível entender a escalabilidade do sistema, verificar o quanto o desempenho é afetado e verificar o quão próxima a escalabilidade do sistema é da escalabilidade linear.

Para realização das análises foram utilizados computadores com processador Intel Pentium Core 2 Duo de 2GHz, 2 GB de memória RAM, discos de 5.400 RPM e sistema operacional Windows XP. Em cada um dos nós foram hospedadas uma instância do módulo Receptor e uma instância do módulo Motor de Processamento, a instância do módulo Receptor foi configurado para sempre delegar requisições para o módulo local do Motor de Processamento.

Com esta configuração foi preciso balancear apenas as requisições enviadas para os Receptores. Para o balanceamento destas requisições foi utilizado um servidor Apache com módulo de *proxy balancer* que foi posicionado à frente de todos os Receptores. Para execução dos testes

foi utilizado o JMeter [JMeter]. Esta organização é ilustrada na Figura 5.3.

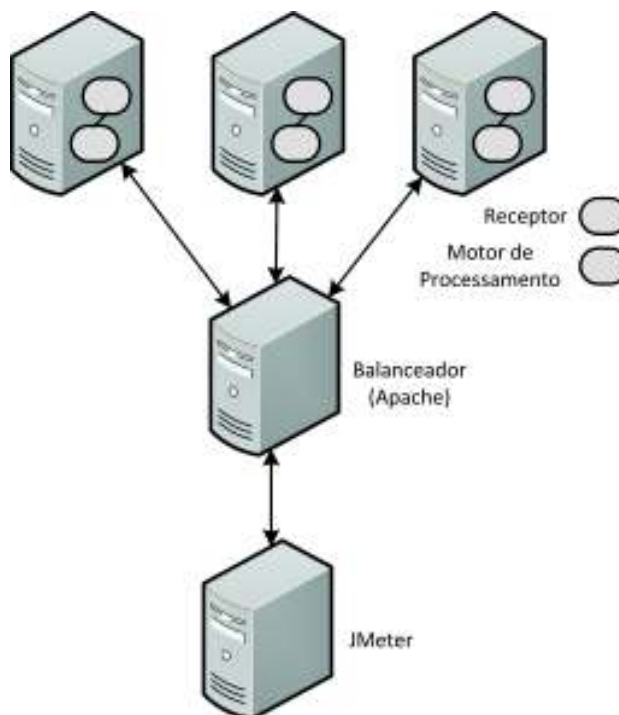


Figura 5.3: Organização dos módulos do sistema

Observe que os computadores utilizados para os testes não possuem a configuração mínima recomendada para execução do sistema em um ambiente de produção, assim os valores apresentados nas análises não obedecem os requisitos de desempenho descritos anteriormente na seção 5.2.

A primeira avaliação consiste em avaliar como o tempo de resposta é afetado à medida que o sistema é escalado horizontalmente. O objetivo desta avaliação não é saber o quão rápidas são as respostas ou qual é a vazão do sistema, o objetivo é saber se, à medida que mais nós são adicionados, o tempo de resposta aumenta, diminui ou continua constante. Para realização desta avaliação foi simulado um usuário enviando requisições para análise durante 3 minutos. Este teste foi repetido variando-se a quantidade de nós de processamento, e a primeira execução do teste foi descartada e apenas a segunda foi considerada.

O gráfico 5.4 ilustra os resultados dos testes, os pontos na curva são a média dos tempos de respostas das requisições durante os 3 minutos de testes para cada quantidade de nós. O eixo vertical indica o tempo de resposta em milissegundos, o eixo horizontal indica a quantidade de instâncias do sistema.

A tabela 5.5 apresenta um resumo das informações obtidas no teste: os tempos de resposta são em milissegundos; e a vazão em requisições/minuto.

Observando-se o gráfico e os dados da tabela verifica-se que há um pequeno aumento no tempo médio de resposta à medida que se aumenta a quantidade de nós, enquanto a vazão diminui. Duas possibilidades poderiam causar o aumento no tempo de resposta.



Figura 5.4: Avaliação do tempo de resposta

Qtde Nós	Tempo de Resp.	Variação do		
	Médio	Tempo de Resp.	Vazão	Variação da Vazão
1	495	0,00%	121	0,00%
2	521	5,25%	114	6,14%

Figura 5.5: Resumo dos resultados da primeira avaliação

A primeira possibilidade é sobretaxa de comunicação e sincronização entre os nós. Entretanto, o sistema foi construído para que os nós sejam independentes e não se comuniquem durante o processamento de requisições. Portanto, esta possibilidade foi descartada.

A segunda possibilidade é a presença de algum gargalo. O sistema possui 3 componentes que poderiam se tornar um gargalo: o banco de dados; o cache distribuído; ou o balanceador de carga. Durante os testes, o componente mais exigido dos 3 foi o balanceador de carga, entretanto apenas por este fato não se pode concluir que ele foi a causa do aumento no tempo de resposta. Pode-se então concluir que estes 3 componentes, em conjunto, causaram aumento no tempo de resposta. Todavia, deve-se notar que na instalação do sistema que está em produção a variação foi menor do que a dos testes realizados aqui, pois em ambiente de produção utiliza-se hardware adequado para todos os componentes do sistema (módulos Receptor e Motor de Processamento, memcached, banco de dados, rede, etc.).

A segunda avaliação consiste em avaliar a capacidade de processamento do sistema à medida que se escala horizontalmente. Para realização desta avaliação foi simulada uma quantidade crescente de usuários enviando requisições para análise por 3 minutos. Inicia-se simulando 1 usuário até que se chegue a 30 usuários simultâneos e repete-se o teste variando a quantidade de nós de processamento. O objetivo é saber como o tempo de resposta varia em função da carga de trabalho à medida que a quantidade de nós é aumentada. A Figura 5.6 ilustra os resultados dos testes com 1 nó.

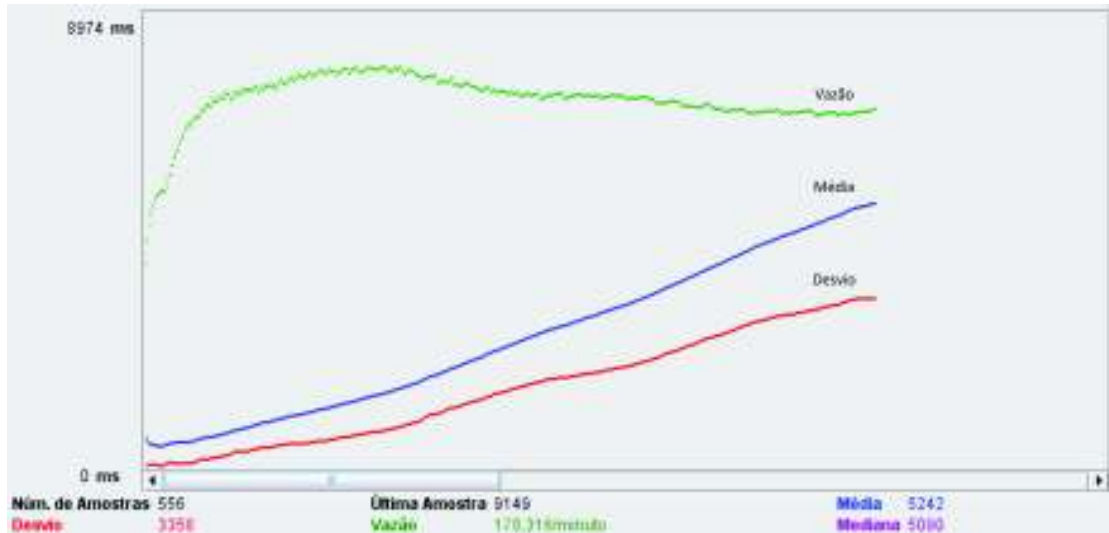


Figura 5.6: Variação do tempo de resposta por vazão, 1 nó

A Figura 5.7 ilustra os resultados dos testes com 2 nós.



Figura 5.7: Variação do tempo de resposta por vazão, 2 nós

A tabela 5.8 apresenta um resumo das informações obtidas no teste. Os tempos de resposta são em milissegundos; e a vazão em requisições/minuto.

Qtde Nós	Tempo de Resp. Médio	Variação do Tempo de Resp.	Vazão	Variação da Vazão
1	5242	0,00%	178	0,00%
2	2633	-50,23%	353	198,31%

Figura 5.8: Resumo dos resultados da segunda avaliação

Os números obtidos no segundo cenário estão dentro do esperado. À medida que o número de nós foi aumentado o tempo de resposta diminuiu proporcionalmente a quantidade de nós e a

vazão aumenta na mesma medida.

As avaliações realizadas indicam que o sistema possui uma escalabilidade quase linear. À medida que o sistema cresce, não houve impacto significativo no tempo de resposta, indicando que seu crescimento gera pouca sobretaxa, e com uma carga de trabalho crescente é possível verificar que o sistema tem melhoras no tempo de resposta e vazão, proporcionais a quantidade de nós adicionados.

Contudo, há uma pergunta que não pode ser respondida: até quando o sistema será linearmente escalável? Para responder a esta pergunta, seria necessário realizar os testes aumentando a quantidade de nós até verificar-se que não há melhora no desempenho ou que as melhoras são diferentes do esperado, entretanto, para realizar tal cenário é preciso ter a disposição uma grande quantidade de hardware, o que não foi possível.

Capítulo 6

Conclusão e Trabalho Futuros

6.1 Conclusão

Recentemente muita importância tem sido dada a sistemas escaláveis, pois praticamente todo sistema desenvolvido nos dias de hoje tem entre seus requisitos não funcionais escalabilidade. Nesta dissertação foram apresentadas definições de escalabilidade, padrões arquiteturais e de projeto e diretrizes que podem ser utilizados para a construção e implementação de sistemas escaláveis.

Escalabilidade foi definida detalhadamente, assim como as duas principais estratégias para escalar um sistema, escalabilidade horizontal e vertical. A escalabilidade foi classificada em 3 categorias: linear; sublinear; e superlinear. Cada categoria é caracterizada pelo seu fator de escalabilidade. Através destas definições é possível discutir objetivos de escalabilidade com mais clareza.

Técnicas para construção de arquiteturas e implementação de sistemas escaláveis foram apresentadas e organizadas em padrões arquiteturais e diretrizes. O padrão Arquitetura *Shared Nothing* objetiva a construção de sistemas com escalabilidade linear através do uso de várias instâncias independentes do sistema evitando o aparecimento de gargalos. O padrão *Sharding* é uma alternativa para escalar sistemas que lidam com grandes volumes de dados, os quais são umas das partes mais difíceis de serem escaladas horizontalmente. BASE é um padrão amplo que ataca vários problemas, onde se troca uma consistência forte dos dados por disponibilidade e tolerância a partições e ganha-se em consequência escalabilidade e desempenho. Sagas são uma maneira de evitar o uso de transações distribuídas e ao mesmo tempo conseguir manter a consistência dos dados. Camadas de cache distribuídos ampliam o uso tradicional de caches para auxiliar no aumento da escalabilidade, evitando o armazenamento de dados duplicados nos caches ao mesmo tempo que se evita operações de I/O desnecessárias em dispositivos mais lentos (como banco de dados).

Os padrões apresentados foram organizados em uma linguagem de padrões, onde foram documentados os relacionamentos entre os padrões e como eles podem ser utilizados em con-

junto para a construção de sistemas. A linguagem de padrões se torna uma ferramenta útil para ser utilizada por arquitetos de sistemas e auxiliar na tomada de decisões arquiteturais.

Várias diretrizes foram apresentadas que podem ser seguidas para se obter sistemas escaláveis. As diretrizes foram discutidas e são outra ferramenta para ser utilizada que complementam os padrões.

Para exemplificar o uso dos padrões e das diretrizes foi apresentada e discutida a arquitetura de um sistema escalável. O sistema apresentado como exemplo é um sistema real, que conseguiu atingir seus requisitos de escalabilidade através das técnicas descritas neste trabalho.

Para entender a escalabilidade do sistema exemplo, foram feitos ensaios de laboratório para analisar o comportamento do sistema em relação à sua escalabilidade. Os resultados obtidos indicam que os padrões e diretrizes apresentados por este trabalho foram importantes para se atingir a escalabilidade.

Em essência, os resultados indicaram que há pertinência nas diretrizes e na linguagem de padrões para projeto de sistemas escaláveis. Infelizmente, não foi possível chegar-se aos limites dos testes, para validar a implementação em sua amplitude, em função das limitações laboratoriais. Contudo, para o espaço amostrado, a implementação do sistema mostrou-se próxima a escalabilidade linearmente.

Com a estruturação das técnicas de escalabilidade em padrões arquiteturais e de projeto, a sua organização em uma linguagem de padrões, e a apresentação das diretrizes, atingimos o objetivo principal deste trabalho, ou seja, prover ferramentas para que arquitetos, projetistas e desenvolvedores possam construir e implementar sistemas escaláveis desde sua concepção.

Até o momento, não é conhecido, pelo menos em língua portuguesa, e após exaustiva pesquisa, um trabalho com a envergadura e o propósito do apresentado neste projeto. A estruturação das técnicas em padrões, sua organização em uma Linguagem de Padrões e a identificação e discussão das diretrizes são uma importante contribuição para a área de arquitetura de sistemas distribuídos que carece de tais ferramentas para escalabilidade. Com estas apropriadas e úteis ferramentas possibilita-se a construção de sistemas escaláveis, disseminando a experiência acumulada de anos de experiência de arquitetos e desenvolvedores, especialmente os não tão experientes, contribuindo para que sistemas que ainda serão construídos atendam seus requisitos de escalabilidade e tenham uma maior qualidade.

6.2 Trabalhos Futuros

Com a realização deste trabalho vislumbramos alguns trabalhos futuros que podem ser feitos na área de escalabilidade. O principal trabalho seria a identificação e catalogação de mais técnicas para escalabilidade e sua estruturação em padrões arquiteturais e de projeto. Este trabalho pode ser feito através do estudo das arquiteturas de sistemas escaláveis reais.

Outro futuro trabalho é o estudo dos padrões apresentados aqui sob a óptica de definições formais de escalabilidade e de técnicas para prever a escalabilidade de sistemas. A definição

formal feita em [Duboc et al. 2007] parece ser particularmente interessante para tal estudo.

Entre os padrões apresentados aqui, o padrão BASE merece ainda mais estudo. BASE não é apenas um padrão, é um conceito, e é possível utilizar outros padrões e técnicas para implementá-lo e suportá-lo além dos apresentados neste trabalho.

Transações ACID é um conceito sedimentado e utilizado por todos os bancos de dados. O padrão BASE pode ser uma alternativa interessante e uma ampla área de pesquisa nesta matéria.

Ainda um outro trabalho futuro, é o desenvolvimento de uma ontologia para criar um modelo de referência para arquiteturas escaláveis.

Referências Bibliográficas

- [Alexander 1979] Alexander, C. (1979). *The Timeless Way of Building*. Oxford University Press.
- [Amazon] Amazon. Amazon. <http://www.amazon.com>. Acessado em 20/09/2009.
- [Amdahl 1967] Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. In *AFIPS '67 (Spring): Proc. of the April 18-20, 1967, spring joint computer conference*, pp. 483–485.
- [Anderson 1999] Anderson, K. M. (1999). Supporting industrial hyperwebs: lessons in scalability. In *ICSE '99: Proc. of the 21st intl. conference on Software engineering*, pp. 573–582.
- [Balakrishnan et al. 2003] Balakrishnan, H., Kaashoek, M. F., Karger, D., Morris, R., e Stoica, I. (2003). Looking up data in P2P systems. *Commun. ACM*, 46(2):43–48.
- [Beck e Cunningham 1987] Beck, K. e Cunningham, W. (1987). Using Pattern Languages for Object-Oriented Program. In *OOPSLA '87: workshop on Specification and Design for Object-Oriented Programming*.
- [Bertolino e Mirandola 2004] Bertolino, A. e Mirandola, R. (2004). Software performance engineering of component-based systems. In *WOSP '04: Proc. of the 4th intl. workshop on Software and performance*, pp. 238–242.
- [Bondi 2000] Bondi, A. B. (2000). Characteristics of scalability and their impact on performance. In *WOSP '00: Proc. of the 2nd intl. workshop on Software and performance*, pp. 195–203.
- [Borden et al. 1989] Borden, T. L., Hennessy, J. P., e Rymarczyk, J. W. (1989). Multiple operating systems on one processor complex. *IBM Syst. J.*, 28(1):104–123.
- [Brataas e Hughes 2004] Brataas, G. e Hughes, P. (2004). Exploring architectural scalability. In *WOSP '04: Proc. of the 4th intl. workshop on Software and performance*, pp. 125–129.
- [Brewer 2000] Brewer, E. A. (2000). Towards robust distributed systems (abstract). In *PODC '00: Proc. of the nineteenth annual ACM symposium on Principles of distributed computing*, p. 7.
- [Buschmann et al. 2007a] Buschmann, F., Henney, K., e Schmidt, D. (2007a). *Pattern-Oriented Software Architecture: A Pattern Language for Distributed Computing (Wiley Software Patterns Series)*. John Wiley & Sons.
- [Buschmann et al. 2007b] Buschmann, F., Henney, K., e Schmidt, D. (2007b). *Pattern-Oriented Software Architecture: A Pattern Language for Distributed Computing (Wiley Software Patterns Series)*. John Wiley & Sons.

- [Clements et al. 2002] Clements, P., Garlan, D., Bass, L., Stafford, J., Nord, R., Ivers, J., e Little, R. (2002). *Documenting Software Architectures: Views and Beyond*. Pearson Education.
- [Crispin 1996] Crispin, M. (1996). Internet Message Access Protocol - Version 4rev1. RFC 2060 (Proposed Standard). Obsoleted by RFC 3501.
- [Danga] Danga. memcached. <http://www.danga.com/memcached/>. Acessado em 20/09/2009.
- [Demers et al. 1994] Demers, A., Petersen, K., Spreitzer, M., Terry, D., Theimer, M., e Welch, B. (1994). The Bayou Architecture: Support for Data Sharing among Mobile Users.
- [Duboc et al. 2007] Duboc, L., Rosenblum, D., e Wicks, T. (2007). A framework for characterization and analysis of software system scalability. In *ESEC-FSE '07: Proc. of the the 6th joint meeting of the European software engineering conference*, pp. 375–384.
- [eBay] eBay. eBay. <http://www.ebay.com>. Acessado em 20/09/2009.
- [Eckerson 1995] Eckerson, W. W. (1995). Three Tier Client/Server Architecture: Achieving Scalability, Performance, and Efficiency in Client Server Applications. In *Open Information Systems 10*.
- [Facebook] Facebook. <http://www.facebook.com>. Acessado em 20/09/2009.
- [Fitzpatrick 2007] Fitzpatrick, B. (2007). LiveJournal: Behind The Scenes. http://danga.com/words/2007_yapc_asia/yapc-2007.pdf. Acessado em 20/09/2009.
- [Flickr] Flickr. <http://www.flickr.com>. Acessado em 20/09/2009.
- [Floyd Marinescu] Floyd Marinescu, C. H. Trading Consistency for Scalability in Distributed Architectures. <http://www.infoq.com/news/2008/03/ebaybase>. Acessado em 20/09/2009.
- [Fowler 2007] Fowler, M. (2007). Transactionless. <http://martinfowler.com/bliki/Transactionless.html>. Acessado em 20/09/2009.
- [Friendster] Friendster. <http://www.friendster.com>. Acessado em 20/09/2009.
- [Gamma et al. 1995] Gamma, E., Helm, R., Johnson, R., e Vlissides, J. (1995). *Design patterns: elements of reusable object-oriented software*. Addison-Wesley Professional.
- [Garcia-Molina e Salem 1987] Garcia-Molina, H. e Salem, K. (1987). Sagas. *SIGMOD Rec.*, 16(3):249–259.
- [GigaSpaces] GigaSpaces. GigaSpaces. <http://www.gigaspaces.com/>. Acessado em 20/09/2009.
- [Gilbert e Lynch 2002] Gilbert, S. e Lynch, N. (2002). Brewer's Conjecture and the Feasibility of Consistent Available Partition-Tolerant Web Services. In *In ACM SIGACT News*, p. 2002.
- [Gray e Cheriton 1989] Gray, C. e Cheriton, D. (1989). Leases: an efficient fault-tolerant mechanism for distributed file cache consistency. In *SOSP '89: Proceedings of the twelfth ACM symposium on Operating systems principles*, pp. 202–210.

- [Gustafson 1988] Gustafson, J. L. (1988). Reevaluating Amdahl's law. *Commun. ACM*, 31(5):532–533.
- [Herlihy e Wing 1990] Herlihy, M. P. e Wing, J. M. (1990). Linearizability: a correctness condition for concurrent objects. *ACM Trans. Program. Lang. Syst.*, 12(3):463–492.
- [highscalability.com 2008] highscalability.com (2008). YouTube Architecture. <http://highscalability.com/youtube-architecture>. Acessado em 20/09/2009.
- [Hill 1990] Hill, M. D. (1990). What is scalability? *SIGARCH Comput. Archit. News*, 18(4):18–21.
- [Jack Dorsey 2008] Jack Dorsey, B. S. (2008). It's Not Rocket Science, But It's Our Work. <http://blog.twitter.com/2008/05/its-not-rocket-science-but-its-our-work.html>. Acessado em 20/09/2009.
- [JBoss a] JBoss. Infinispan. <http://www.jboss.org/infinispan>. Acessado em 20/09/2009.
- [JBoss b] JBoss. JBoss Cache. <http://www.jboss.org/jboss-cache/>. Acessado em 20/09/2009.
- [JMeter] JMeter. JMeter. <http://jakarta.apache.org/jmeter/>. Acessado em 20/09/2009.
- [Joglekar e Woodside 2000] Joglekar, P. e Woodside, M. (2000). Evaluating the Scalability of Distributed Systems. *IEEE Trans. Parallel Distrib. Syst.*, 11(6):589–603.
- [Karger et al. 1997] Karger, D., Lehman, E., Leighton, T., Panigrahy, R., Levine, M., e Lewin, D. (1997). Consistent hashing and random trees: distributed caching protocols for relieving hot spots on the World Wide Web. In *STOC '97: Proceedings of the twenty-ninth annual ACM symposium on Theory of computing*, pp. 654–663.
- [Kircher e Jain 2004] Kircher, M. e Jain, P. (2004). *Pattern-Oriented Software Architecture: Patterns for Resource Management*. John Wiley & Sons.
- [Klensin 2008] Klensin, J. (2008). Simple Mail Transfer Protocol. RFC 5321 (Draft Standard).
- [Lamport 1986a] Lamport, L. (1986a). On Interprocess Communication. Part I: Basic Formalism. *Distributed Computing*, 1(2):77–85.
- [Lamport 1986b] Lamport, L. (1986b). On Interprocess Communication. Part II: Algorithms. *Distributed Computing*, 1(2):86–101.
- [LiveJournal] LiveJournal. <http://www.livejournal.com>. Acessado em 20/09/2009.
- [Luke 1994] Luke, E. (1994). Defining and Measuring Scalability. In *In Scalable Parallel Libraries Conference*, pp. 183–186. IEEE Press.
- [McConnell 2004] McConnell, S. (2004). *Code Complete, Second Edition*. Microsoft Press.
- [Messerschmitt 1996] Messerschmitt, D. (1996). The convergence of telecommunications and computing: What are the implications today.

- [Neuman 1994] Neuman, B. C. (1994). Scale in Distributed Systems. In *Readings in Distributed Computing Systems*, pp. 463–489.
- [Oracle] Oracle. Oracle Coherence. <http://www.oracle.com/products/middleware/coherence/index.html>. Acessado em 20/09/2009.
- [Pattishall 2006] Pattishall, D. (2006). Unorthodox approach to database design Part 2:Friendster. <http://mysqldba.blogspot.com/2006/11/unorthodox-approach-to-database-design.html>. Acessado em 20/09/2009.
- [Pressman 2004] Pressman, R. (2004). *Software Engineering: A Practitioner's Approach*, 6 edition. McGraw-Hill Science/Engineering/Math.
- [Pritchett 2007a] Pritchett, D. (2007a). The Challenges of Latency. <http://www.infoq.com/articles/pritchett-latency>. Acessado em 20/09/2009.
- [Pritchett 2007b] Pritchett, D. (2007b). Dan Pritchett on Architecture at eBay. <http://www.infoq.com/interviews/dan-pritchett-ebay-architecture>. Acessado em 20/09/2009.
- [Pritchett 2008a] Pritchett, D. (2008a). BASE: An Acid Alternative. *Queue*, 6(3):48–55.
- [Pritchett 2008b] Pritchett, D. (2008b). Shard Lessons. http://www.addsimplicity.com/adding_simplicity_an_engi/2008/08/shard-lessons.html. Acessado em 20/09/2009.
- [Rosenthal 2003] Rosenthal, A. (2003). Toward Scalable Exchange of Security Information. Technical report, The MITRE Corporation.
- [Rotem-Gal-Oz 2007] Rotem-Gal-Oz, A. (2007). Fallacies of Distributed Computing Explained. <http://www.rgoarchitects.com/Files/fallacies.pdf>. Acessado em 20/09/2009.
- [Schmidt et al. 2000] Schmidt, D. C., Rohnert, H., Stal, M., e Schultz, D. (2000). *Pattern-Oriented Software Architecture: Patterns for Concurrent and Networked Objects*. John Wiley & Sons, Inc.
- [Shoup 2008] Shoup, R. (2008). Scalability Best Practices: Lessons from eBay. <http://www.infoq.com/articles/ebay-scalability-best-practices>. Acessado em 20/09/2009.
- [Smith 1987] Smith, R. (1987). Panel on design methodology. In *OOPSLA '87: Addendum to the proceedings on Object-oriented programming systems, languages and applications (Addendum)*, pp. 91–95.
- [Steen et al. 1998] Steen, M. V., Zijden, S. V. d., e Sips, H. J. (1998). Software Engineering for Scalable Distributed Applications. In *COMPSAC '98: Proc. of the 22nd intl. Computer Software and Applications Conference*, pp. 285–293.
- [Stonebraker 1986] Stonebraker, M. (1986). The Case for Shared Nothing. *Database Engineering*, 9:4–9.

- [Tharakan 2007] Tharakan, R. K. (2007). What is scalability? <http://www.royans.net/arch/2007/09/22/what-is-scalability/>. Acessado em 20/09/2009.
- [Vogels 2007] Vogels, W. (2007). Availability & Consistency. <http://www.infoq.com/presentations/availability-consistency>. Acessado em 20/09/2009.
- [Vogels 2008] Vogels, W. (2008). Eventually Consistent. *Queue*, 6(6):14–19.
- [Weinstock e Goodenough 2006] Weinstock, C. B. e Goodenough, J. B. (2006). On System Scalability. Technical report, SEI, Carnegie Mellon University, CMU/SEI-2006-TN-012.
- [Welsh et al. 2001] Welsh, M., Culler, D., e Brewer, E. (2001). SEDA: an architecture for well-conditioned, scalable internet services. *SIGOPS Oper. Syst. Rev.*, 35(5):230–243.
- [Williams e Smith 2004] Williams, L. G. e Smith, C. U. (2004). Web Application Scalability: A Model Based Approach. Technical report, Computer Measurement Group Conference (CMG'04).

Apêndice A

Sagas

```
1  /*
2   Copyright (C) 2009 Ivens Oliveira Porto.
3
4   This file is part of Sagas API.
5
6   Sagas API is free software: you can redistribute it and/or modify
7   it under the terms of the GNU General Public License as published by
8   the Free Software Foundation, either version 3 of the License,
9   or (at your option) any later version.
10
11   Sagas API is distributed in the hope that it will be useful,
12   but WITHOUT ANY WARRANTY; without even the implied warranty of
13   MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
14   GNU General Public License for more details.
15
16   You should have received a copy of the GNU General Public License
17   along with Sagas API. See COPYING. If not, write to
18   <sagasapi@gmail.com>.
19  */
20  package net.sagas;
21
22
23  import java.io.Serializable;
24  import java.util.List;
25
26
27  /**
28   * Interface para manipulação de uma saga.
29   *
30   * @author ivens
31   *
32   * @param <T>
33   *         classe do tipo do contexto da saga
```

```
34  */
35  public interface ISaga< T extends Serializable > {
36
37      // Possíveis estados de uma saga
38      enum SagaStatus {
39          NOT_STARTED,
40          IN_EXECUTION,
41          FINISHED,
42          ABORTED,
43          ROLLING_BACK
44      }
45
46
47      /**
48       * Adiciona uma subtransação a saga.
49       *
50       * @param subTx
51       *         uma sub-transação
52       */
53      void add( ISagaSubTransaction< T >... subTx );
54
55
56      /**
57       * @return as sub-transações da saga
58       */
59      List< ISagaSubTransaction< T > > getSubTransactions();
60
61
62      /**
63       * @return o contexto da saga
64       */
65      T getContext();
66
67
68      /**
69       * Atribui a saga o seu contexto.
70       *
71       * @param ctx
72       *         o contexto
73       */
74      void setContext( T ctx );
75
76
77      /**
78       * @return o identificador da saga.
79       */
80      Long getId();
```

```

81
82
83     /**
84      * @return o estado atual da saga
85      */
86     SagaStatus getStatus();
87 }

```

Listagem A.1: ISaga.java

```

1  /*
2   Copyright (C) 2009 Ivens Oliveira Porto.
3
4   This file is part of Sagas API.
5
6   Sagas API is free software: you can redistribute it and/or modify
7   it under the terms of the GNU General Public License as published by
8   the Free Software Foundation, either version 3 of the License,
9   or (at your option) any later version.
10
11   Sagas API is distributed in the hope that it will be useful,
12   but WITHOUT ANY WARRANTY; without even the implied warranty of
13   MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
14   GNU General Public License for more details.
15
16   You should have received a copy of the GNU General Public License
17   along with Sagas API. See COPYING. If not, write to
18   <sagasapi@gmail.com>.
19   */
20 package net.sagas;
21
22
23 import java.io.Serializable;
24
25
26 /**
27  * Uma sub-transação de uma saga.
28  *
29  * @author ivens
30  *
31  * @param <T>
32  *         classe do tipo de contexto utilizado pela sub-transação e sua
33  *         saga.
34  */
35 public interface ISagaSubTransaction< T extends Serializable >
36     extends Serializable {
37
38     /**

```

```

39      * Executa a sub-transação.
40      *
41      * @param ctx
42      *          o contexto da saga.
43      * @throws Exception
44      *          em caso de qualquer erro. A saga é abortada.
45      */
46      void execute( T ctx ) throws Exception;
47
48
49      /**
50      * Executa a ação compensatória da sub-transação.
51      *
52      * @param ctx
53      *          o contexto da saga.
54      * @throws Exception
55      *          em caso de qualquer erro. A saga é abortada.
56      */
57      void compensate( T ctx ) throws Exception;
58 }

```

Listagem A.2: ISagaSubTransaction.java

```

1  /*
2   Copyright (C) 2009 Ivens Oliveira Porto.
3
4   This file is part of Sagas API.
5
6   Sagas API is free software: you can redistribute it and/or modify
7   it under the terms of the GNU General Public License as published by
8   the Free Software Foundation, either version 3 of the License,
9   or (at your option) any later version.
10
11   Sagas API is distributed in the hope that it will be useful,
12   but WITHOUT ANY WARRANTY; without even the implied warranty of
13   MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
14   GNU General Public License for more details.
15
16   You should have received a copy of the GNU General Public License
17   along with Sagas API. See COPYING. If not, write to
18   <sagasapi@gmail.com>.
19  */
20
21  package net.sagas;
22
23
24  import java.io.Serializable;
25

```



```
26
27 /**
28  * Executor de sagas.
29  *
30  * @author ivens
31  *
32  */
33 public interface ISagaExecutor {
34
35     /**
36      * Executa uma saga.
37      *
38      * @param s
39      * a saga a ser executada.
40      */
41     public void execute( ISaga< Serializable > s );
42 }
```

Listagem A.3: ISagaExecutor.java